Writing Kernel with AI

K19G

2026-10-13

Table of contents

Writing Kernel with AI **9**
CPU to AI-Written Kernels . . . 9
Welcome to Kernel Engineering . . . 9
The Trajectory of This Book . . . 9
Who This Is For . . . 10
Formats . . . 10

Part I: Parallelism, from the CPU up **11**

Parallelism on the CPU **12**

Parallelism on the CPU **13**
1. Instruction-Level Parallelism (ILP) and Microarchitecture . . . 13
The Out-of-Order Engine . . . 14
2. SIMD: Vectorized Data Parallelism . . . 14
Vector Widths Across Hardware Architectures . . . 14
Explicit AVX2 Vector Operations in C++ . . . 15
Alignment and Vector Loads . . . 15
3. Thread-Level Parallelism: Heavyweight vs Lightweight . . . 16
The Cost of OS Threads . . . 16
User-Space Concurrency (M:N Models) . . . 16
4. Cache Lines and False Sharing . . . 16
The False Sharing Trap . . . 16
Remediation via Padding . . . 17
5. Architectural Contrast: CPU vs GPU Design Philosophy . . . 17

Roofline Lab: Predict Then Measure **19**

Roofline Lab: Predict Then Measure **20**
1. The Roofline Model Formulation . . . 20
Definitions . . . 20
The Ridge Point (I^*) . . . 21
2. Hardware Ridge Points Across Generations . . . 21
3. Dissecting Common Machine Learning Operations . . . 22
Case 1: Vector Addition ($y = x + z$) . . . 22
Case 2: LayerNorm / RMSNorm . . . 22
Case 3: Dense Matrix Multiplication (GEMM) . . . 23
4. The Multi-Tier Roofline (Hierarchy of Limits) . . . 23
5. Profiling Roofline Performance with NVIDIA Nsight Compute . . . 23
Key NCU Metrics to Inspect: . . . 24

The CUDA Programming Model & GPU Puzzles **25**

The CUDA Programming Model & GPU Puzzles **26**
1. The Logical vs Physical Hierarchy . . . 26
The Dimensions . . . 26
2. Multi-Dimensional Index Calculations . . . 27
1D Global Index Calculation . . . 27
2D Global Index Calculation (Matrix Coordinates) . . . 27
3. Warp Divergence and the SIMT Execution Mask . . . 27
The Cost of Divergence . . . 28
Best Practices to Eliminate Divergence . . . 28
4. Complete Worked Example: Vectorized FMA with Error Handling . . . 28
5. Inspecting Intermediate PTX and Machine SASS . . . 30

The Memory Hierarchy in Anger: Matrix Transpose **31**

The Memory Hierarchy in Anger: Matrix Transpose **32**
1. The GPU Memory Hierarchy . . . 32
Memory Space Properties . . . 33
2. Global Memory Coalescing: The 10x Lever . . . 33
The Golden Rule of Coalescing . . . 33
3. Shared Memory Bank Conflicts . . . 34
The Conflict Condition . . . 34
4. Case Study: Step-by-Step Optimization of Matrix Transpose . . . 34
Step 1: Naive Transpose (Uncoalesced Writes) . . . 34
Step 2: Coalesced Transpose via Shared Memory (With Bank Conflicts) . . . 35
Step 3: Conflict-Free Transpose with Shared Memory Padding . . . 36
Full Production Code . . . 36
5. Performance Comparison . . . 37

Part II: The GEMM Worklog: Naive to cuBLAS **38**

GEMM Worklog I: Naive to Tiled **39**

GEMM Worklog I: Naive to Tiled **40**
1. Dissecting the Naive GEMM: Why is it Slow? . . . 40
The Naive CUDA Kernel . . . 40
Performance Autopsy . . . 41
2. The Breakthrough: Shared Memory 2D Block Tiling . . . 41
Tile Dimensions and Reuse Factor . . . 42
3. The 2D Tiled CUDA Kernel . . . 42
4. Benchmark Results on NVIDIA A100 . . . 43
Why Did Performance Jump by 27x? . . . 44

GEMM Worklog II: Registers & Warp Tiling **45**

GEMM Worklog II: Registers & Warp Tiling **46**
Micro-Tiling in Registers . . . 46

Vectorized Memory Loads with `float4` . . . 46
The Warptiling Hierarchy . . . 46

Tensor Cores & WMMA: The Second Worklog **47**

Tensor Cores & WMMA: The Second Worklog **48**
The Tensor Core Advantage . . . 48
Direct Programming via WMMA (Warp Matrix Multiply Accumulate) . . . 48

Part III: Attention & Profiling **49**

Profiling & Debugging Like a Pro **50**

Profiling & Debugging Like a Pro **51**
The Professional Diagnostic Stack . . . 51
Debugging Sabotaged Kernels: 3 Classic Incident Scenarios . . . 51

launch_bounds and Register Pressure: What the Compiler Picks When You Say Nothing **52**

launch_bounds and Register Pressure: What the Compiler Picks When You Say Nothing **53**
1. Mental model: registers are divided, not shared . . . 53
2. The kernel . . . 54
3. Compile five ways without a GPU . . . 54
4. From register counts to occupancy . . . 56
5. Which build is fastest? Measure; don't infer . . . 57
6. The trap: "fixing" spills by removing the bounds . . . 57
7. Prompting an AI assistant for register work . . . 57
Pitfalls . . . 58
Checkpoint . . . 58
Try this . . . 58
Summary . . . 58
Sources (for further reading) . . . 58

Attention: The Kernel That Ate the World & FlashAttention v1 **59**

Attention: The Kernel That Ate the World & FlashAttention v1 **60**
The Breakthrough: Online Softmax & Tiling . . . 60
The Online Softmax Formula . . . 60

Part IV: The Modern Frontier **61**

FlashAttention from Scratch (FA1 → FA2 → FA3) **62**

FlashAttention from Scratch (FA1 → FA2 → FA3) **63**
FlashAttention Evolution . . . 63

Beating cuBLAS on an H100 **64**

Beating cuBLAS on an H100 **65**
1. Tensor Memory Accelerator (TMA) . . . 65
2. Warp Specialization . . . 65

Triton → CUTLASS → CuTe-DSL **66**

Triton → CUTLASS → CuTe-DSL **67**
OpenAI Triton . . . 67
CUTLASS & CuTe . . . 67

Inference-Serving Kernels **68**

Inference-Serving Kernels **69**
PagedAttention (vLLM) . . . 69

Blackwell & NVFP4 **70**

Blackwell & NVFP4 **71**
Blackwell Architectural Highlights . . . 71

A Blackwell GEMM by Hand in CuTe DSL: TMA, mbarriers, TMEM, and tcgen05 **72**

A Blackwell GEMM by Hand in CuTe DSL: TMA, mbarriers, TMEM, and tcgen05 **73**
1. Two regimes: prefill and decode . . . 73
2. The data path and the four primitives . . . 76
3. The kernel . . . 76
4. Compile for `sm_100a` without a GPU . . . 81
5. The ladder: what moved the number (reported) . . . 83
6. The swizzle . . . 84
7. Stages: the budget and the regime . . . 86
8. Two CTAs, then persistence . . . 86
9. Why decode loses, and what to do about it . . . 88
10. Driving an AI assistant through a ladder like this . . . 89
11. Verification and benchmark plan . . . 89
On a CPU (done here) . . . 89
On a Blackwell GPU (not done here) . . . 90
Pitfalls . . . 90
Checkpoint . . . 91
Try this . . . 91
Summary . . . 91
Sources (for further reading) . . . 92

Flash Attention 4 **93**

Flash Attention 4 **94**
The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier . . . 94

PagedAttention and the KV Cache **95**

PagedAttention and the KV Cache **96**
Mental model . . . 96
Why contiguous KV wastes memory . . . 96
Capacity sketch . . . 97
KV cache in Transformers (reminder) . . . 97
Block structure . . . 97
Worked layout numbers . . . 98
Block table and allocator . . . 98
Prefix sharing (copy-on-write intuition) . . . 98
Prefill vs decode under paging . . . 98
Implication . . . 99
Paged KV layouts (kernel writer view) . . . 99
Illustrative host-side table (not a production kernel) . . . 100
Implications for custom kernels . . . 101
Verification and benchmark notes . . . 101
Functional checks . . . 101
Performance / capacity checks . . . 101
Profiling tips . . . 102
Pitfalls . . . 102
Checkpoint . . . 102
Summary . . . 103
Sources (for further reading) . . . 103

Flash-Decoding: Split-KV Attention for Long-Context Decode **104**

Flash-Decoding: Split-KV Attention for Long-Context Decode **105**
1. Why decode starves the GPU . . . 105
Worked numbers . . . 106
2. Mental model: attention states and the merge operator . . . 106
3. Verify the algebra before writing a kernel . . . 107
The trap: the textbook merge overflows . . . 108
4. A two-kernel Triton implementation . . . 109
5. Compile for a GPU you do not have . . . 113
6. How many splits? . . . 114
7. Where the pattern shows up in production . . . 115
8. Verification and benchmark plan . . . 116
Functional . . . 116
Performance . . . 117
9. Prompting an AI assistant for a decode kernel . . . 117
Pitfalls . . . 117
Checkpoint . . . 118
Try this . . . 118
Summary . . . 118
Sources (for further reading) . . . 119

TurboQuant: Rotate-then-Quantize KV Caches at 3 Bits **120**

TurboQuant: Rotate-then-Quantize KV Caches at 3 Bits **121**
1. Why the KV cache is the thing to compress . . . 121
2. Mental model: make every coordinate look the same, then use one codebook . . . 123
3. The quantizer in NumPy . . . 123
4. Check the codebook against the paper . . . 126
5. Against the obvious 3-bit baselines . . . 128
6. The trap: unbiased is not the same as better . . . 130
7. Fractional bits and the outlier split . . . 132
8. Kernel view: never un-rotate a key . . . 132
9. Where it runs today . . . 135
10. Credit where it is due: RaBitQ and the older rotation literature . . . 136
11. Verification plan . . . 137
Functional (CPU) . . . 137
Quality (GPU) . . . 137
Performance (GPU) . . . 137
Pitfalls . . . 137
Checkpoint . . . 138
Try this . . . 138
Summary . . . 139
Sources (for further reading) . . . 139

Mixture-of-Experts Kernels: Routing, Permutation, and Grouped GEMM **141**

Mixture-of-Experts Kernels: Routing, Permutation, and Grouped GEMM **142**
1. Why MoE is a kernel problem . . . 142
2. Mental model: route, align, grouped GEMM, combine . . . 143
3. Ground truth first . . . 144
4. Routing semantics: the bias is for choosing, not for weighting . . . 146
5. The indirection table: histogram, pad, scan, scatter . . . 148
6. A fused grouped GEMM in Triton . . . 149
7. Compile for Hopper, and what the compiler tells you about decode . . . 153
8. Shapes decide everything: padding, stragglers, and the roofline . . . 154
9. Where this sits in production . . . 156
10. Verification and benchmark plan . . . 157
Functional . . . 157
Performance . . . 158
11. Prompting an AI assistant for an MoE kernel . . . 158
Pitfalls . . . 159
Checkpoint . . . 159
Try this . . . 159
Summary . . . 160
Sources (for further reading) . . . 160

Table-Batched Embeddings in Triton: Gather, Pool, and Run-Length Backward **161**

Table-Batched Embeddings in Triton: Gather, Pool, and Run-Length Backward **162**
1. Mental model: what TBE computes . . . 162
2. The kernels . . . 163

3. Verify on CPU . . . 168
4. Compile for GPUs you do not have . . . 169
5. The trap: one optimizer step per **row**, not per lookup . . . 170
6. Prompting an AI assistant for a TBE kernel . . . 171
Pitfalls . . . 171
Checkpoint . . . 172
Try this . . . 172
Summary . . . 172
Sources (for further reading) . . . 173

Part V: AI-Written Kernels **174**

DeepSeek: FlashMLA & DeepGEMM **175**

DeepSeek: FlashMLA & DeepGEMM **176**
FlashMLA: Multi-Head Latent Attention . . . 176
DeepGEMM: Clean Architecture over Brute Force . . . 176

LLM-Generated Kernels: The Agent + Profiler Loop **177**

LLM-Generated Kernels: The Agent + Profiler Loop **178**
The Agentic Optimization Loop . . . 178
Google DeepMind AlphaEvolve & KernelBench . . . 178
The Role of the Human Kernel Engineer . . . 178

Part VI: Kernel Engineering Capstone Projects **179**

Crusoe Capstone Specifications & Problem Tracks **180**

Crusoe Capstone Specifications & Problem Tracks **181**
Problem Track 1: Fused Sparse Attention . . . 181
Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization . . . 181
Problem Track 3: High-Throughput Speculative Decoding Verification Kernel . . . 181

Verification, Profiling, and Benchmarking Harness **182**

Verification, Profiling, and Benchmarking Harness **183**
1. Correctness Verification . . . 183
2. CUDA Event Benchmarking . . . 183

Demo Day Presentation & Portfolio **184**

Demo Day Presentation & Portfolio **185**
Deliverables Checklist . . . 185

Writing Kernel with AI

CPU to AI-Written Kernels

A comprehensive, production-grade guide to high-performance kernel engineering: from the fundamentals of CPU instruction parallelism to hand-tuned CUDA, FlashAttention, modern Hopper/Blackwell hardware architectures, and AI-driven kernel optimization loops.

Welcome to Kernel Engineering

Accelerated computing and modern artificial intelligence rest on a single engineering discipline: **extracting peak theoretical throughput from hardware**.

Whether training frontier foundation models or serving millions of concurrent inference tokens, the software stack converges onto specialized computational kernels. Writing fast kernels is not a matter of guesswork or trial-and-error—it is an empirical science governed by the physics of computation: arithmetic intensity, memory bandwidth, latency hiding, and hardware execution hierarchies.

The Trajectory of This Book

This curriculum spans six rigorous parts designed to take you from first principles to the cutting edge:

1. **Part I: Parallelism, from the CPU up:** Understand how modern CPUs achieve massive throughput via SIMD vectorization and multi-threading before stepping into the GPU execution hierarchy: warps, streaming multiprocessors, memory coalescing, and the Roofline model.
2. **Part II: The GEMM Worklog: Naive → cuBLAS:** Master General Matrix Multiply (GEMM)—the operation that comprises over 90% of deep learning FLOPs. Step by step, elevate a naive kernel achieving 1.3% of GPU capacity to a hierarchical warp-tiled and Tensor Core (WMMA) implementation rivaling NVIDIA's cuBLAS library.
3. **Part III: Attention & Profiling:** Diagnose hardware bottlenecks using NVIDIA Nsight Compute like a compiler engineer. Construct FlashAttention v1 from scratch, utilizing the online softmax algorithm to eliminate high-bandwidth memory bottlenecks.
4. **Part IV: The Modern Frontier:** Explore modern production kernels running on Hopper (H100) and Blackwell (B200) architectures: FlashAttention-2/3/4, TMA asynchronous copy, WGMMA, Triton, CUTLASS, CuTe-DSL, PagedAttention, and NVFP4.

5. **Part V: AI-Written Kernels:** Analyze DeepSeek's ultra-compact, high-performance kernels (FlashMLA and DeepGEMM). Step into the future of kernel development: using autonomous LLM agents (e.g., DeepMind AlphaEvolve, KernelBench) with profiler-in-the-loop guidance.
6. **Part VI: Kernel Engineering Capstone Projects:** Put your skills into practice with industry-grade capstone specifications developed with Crusoe, complete with verification harnesses, performance benchmarks, and demo-day technical presentations.

Who This Is For

- **ML Infrastructure & Systems Engineers:** Developers optimizing training clusters and inference backends (vLLM, SGLang, TensorRT-LLM).
- **GPU Programmers & Performance Engineers:** Engineers seeking mastery of modern GPU architectures (Hopper, Blackwell), memory coalescing, and async execution pipelines.
- **Software Engineers Transitioning to Accelerated Computing:** Developers wanting deep understanding of what happens when high-level PyTorch or JAX code hits the silicon.

Formats

This book is available in multiple formats via the library portal: - **HTML** — responsive interactive reading experience with syntax highlighting and light/dark theme toggle. - **PDF** — complete typeset document for offline review and reference. - **EPUB** — eBook format optimized for tablets and mobile e-readers.

Part I: Parallelism, from the CPU up

Parallelism on the CPU

Parallelism on the CPU

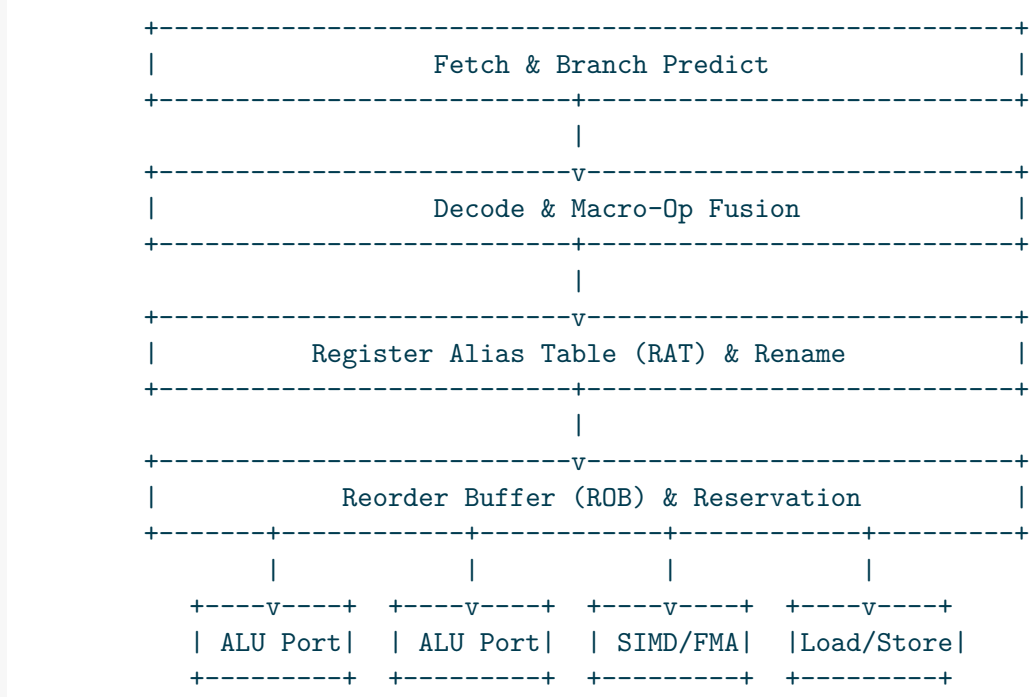
Before programming thousands of lockstep cores on a GPU, we must build a mechanical understanding of how modern CPUs execute concurrent and parallel work. High-performance computing is fundamentally an exercise in orchestrating physical execution units while feeding them data fast enough to prevent pipeline stalls.

Modern CPUs extract performance across three distinct tiers of parallelism: 1. **Instruction-Level Parallelism (ILP):** Pipelining, superscalar dispatch, out-of-order execution, and branch prediction within a single core. 2. **Data-Level Parallelism (SIMD):** Single Instruction, Multiple Data execution where one machine instruction operates over a register packed with multiple independent values. 3. **Thread-Level Parallelism (TLP):** Multiple independent execution streams running concurrently across physical and logical processor cores.

---

1. Instruction-Level Parallelism (ILP) and Microarchitecture

A modern CPU core is not a simple sequential state machine. It is an asynchronous processing pipeline designed to retire multiple micro-operations (uops) per clock cycle.



The Out-of-Order Engine

1. **Instruction Fetch & Branch Prediction:** The CPU fetches instructions ahead of the current program counter using branch target buffers (BTB) and direction predictors. When a branch is mispredicted, the entire speculative pipeline must be flushed, costing 15 to 20 cycles of wasted energy.
2. **Decode and Micro-op Generation:** Complex x86 instructions are broken down into simpler, RISC-like micro-operations (`uops`).
3. **Register Renaming & Reservation Stations:** To break false register dependencies (Write-After-Read and Write-After-Write hazards), the CPU maps architectural registers (e.g., `rax`, `ymm0`) to a much larger pool of physical registers.
4. **Out-of-Order (OoO) Dispatch:** Independent instructions whose operands are ready in the cache or register file execute out of program order across parallel execution ports.
5. **In-Order Retirement:** The Reorder Buffer (ROB) ensures that memory state updates and architectural register commits happen in strict program order, guaranteeing consistent exception semantics.

---

2. SIMD: Vectorized Data Parallelism

While ILP extracts parallelism implicitly from scalar instructions, SIMD (Single Instruction, Multiple Data) makes data parallelism explicit. Instead of performing arithmetic on a single 32-bit floating-point value, SIMD instructions load, operate upon, and store entire vectors of values simultaneously.

Vector Widths Across Hardware Architectures

Vector Extension	Vector Width	32-bit Floats per Vector	Target Architecture
x86 SSE2	128-bit	4	Legacy x86-64
x86 AVX2	256-bit	8	Intel Haswell+ / AMD Zen2+
x86 AVX-512	512-bit	16	Intel Skylake-SP / Emerald Rapids, AMD Zen4+
x86 AVX10	256 / 512-bit	8 or 16	Modern Intel Architecture
ARM Neon	128-bit	4	Apple Silicon, ARM Neoverse V1/V2, Cortex-A
ARM SVE / SVE2	Scalable (128–2048b)	Variable	Fujitsu A64FX, AWS Graviton3/4

Explicit AVX2 Vector Operations in C++

When performance is paramount, reliance on compiler autovectorization is risky. Compilers frequently fail to vectorize loops due to pointer aliasing concerns or unknown loop boundary alignments. Engineers write architecture-specific intrinsic functions:

```
#include <immintrin.h>
#include <iostream>
#include <vector>

void vector_fma_avx2(const float* __restrict__ a,
                     const float* __restrict__ b,
                     const float* __restrict__ c,
                     float* __restrict__ out,
                     size_t n) {
    size_t i = 0;

    // Process 8 floats per iteration using 256-bit AVX2 registers
    for (; i + 8 <= n; i += 8) {
        // Load vectors from memory
        __m256 va = _mm256_loadu_ps(&a[i]);
        __m256 vb = _mm256_loadu_ps(&b[i]);
        __m256 vc = _mm256_loadu_ps(&c[i]);

        // Fused Multiply-Add: out = (va * vb) + vc
        // Computes 8 FMAs (16 floating point operations) in 4-5 cycles latency
        __m256 vres = _mm256_fmadd_ps(va, vb, vc);

        // Store back to destination
        _mm256_storeu_ps(&out[i], vres);
    }

    // Scalar cleanup tail loop
    for (; i < n; ++i) {
        out[i] = a[i] * b[i] + c[i];
    }
}
```

Alignment and Vector Loads

- `_mm256_loadu_ps`: Unaligned load. Safe across any memory address, but incurs small penalty if crossing a 64-byte cache line boundary.
- `_mm256_load_ps`: Aligned load. Requires the source address to be strictly 32-byte aligned (`alignas(32)` or `posix_memalign`). A non-aligned pointer will trigger a hardware general protection fault (`SIGSEGV`).

---

3. Thread-Level Parallelism: Heavyweight vs Lightweight

To leverage multi-core CPUs, applications distribute work across threads. How threads are modeled dramatically impacts context switching overhead and memory footprint.

```
OS Heavyweight Threads:
  [Kernel Scheduler] ---> [Core 0: OS Thread (2-8 MB Stack)]  (Switch: ~1-3 us)
                     ---> [Core 1: OS Thread (2-8 MB Stack)]

Lightweight User-Space Green Threads / Fibers:
  [OS Thread / SM]    ---> [Fiber A (2 KB)] -> [Fiber B (2 KB)] -> [Fiber C (2 KB)]
```

The Cost of OS Threads

POSIX threads (`pthread_create`) or OpenMP workers (`#pragma omp parallel for`) allocate their own kernel task structure and a fixed memory stack (typically 2 MB to 8 MB). - When an OS thread context switches, the CPU must switch privilege rings into the kernel, save and restore architectural register state (including wide AVX-512 state), and suffer TLB and cache pollution. - Total latency: **1,000 to 3,000 CPU cycles** (1–3 microseconds).

User-Space Concurrency (M:N Models)

Modern programming environments decouple logical concurrency from physical OS threads: - **Go Goroutines:** Allocate a dynamic 2 KB stack that grows on demand. The runtime multiplexes thousands of goroutines across a fixed thread pool (`GOMAXPROCS`), switching in user space within ~10 to 50 nanoseconds. - **Java Virtual Threads (Project Loom):** Suspendable continuation objects stored on the garbage-collected heap. - **Rust Tokio:** Futures compiled into zero-allocation state machines driven by a lock-free work-stealing executor.

---

4. Cache Lines and False Sharing

CPU memory controllers do not fetch individual 4-byte floating-point numbers from system DRAM. All transfers between main memory and CPU caches occur in fixed chunks of **64 bytes**, called **cache lines**.

The False Sharing Trap

When two distinct threads running on separate CPU cores write to independent variables that happen to sit within the same 64-byte cache line, the CPU hardware coherence engine (e.g., MESI protocol) must invalidate that entire cache line across both cores back and forth:

```
                  64-Byte Cache Line
+-----------------------------------+-----------------------------------+
| Thread 0 Counter: uint64_t (8 B)  | Thread 1 Counter: uint64_t (8 B)  |
+-----------------------------------+-----------------------------------+
                  ^                                   ^
                  |                                   |
          Core 0 writes                       Core 1 writes
          (Invalidates Core 1)                (Invalidates Core 0)
```

This phenomenon—**false sharing**—can slow down multi-threaded code by a factor of 50x to 100x, reducing parallel throughput below that of a single-threaded loop.

Remediation via Padding

```cpp
// Bad: Threads write to adjacent elements in an array
struct BadThreadState {
    uint64_t counter; // 8 bytes -> adjacent elements share a 64B line!
};

// Good: Align to 64 bytes (the hardware cache line size)
struct alignas(64) GoodThreadState {
    uint64_t counter;
    // 56 bytes of implicit padding inserted by the compiler
};
```

---

5. Architectural Contrast: CPU vs GPU Design Philosophy

Why do GPUs dominate machine learning and scientific compute when CPUs have such sophisticated out-of-order execution engines?

```
CPU Architecture (Latency-Optimized):
+-------------------------------------------------------------+
| [Control: Branch Predictor, Out-of-Order Engine]            |
| [Large L1 / L2 / L3 Caches (Tens of Megabytes)]             |
| [A Few Powerful ALU / FMA Execution Cores (8-64 Cores)]     |
+-------------------------------------------------------------+

GPU Architecture (Throughput-Optimized):
+-------------------------------------------------------------+
| [Control] [Control] [Control] [Control]                     |
| [Thousands of Compact ALU / Tensor Cores]                   |
| [High-Bandwidth HBM3 / GDDR Memory Interface]               |
```

```
+------------------------------------------------------------+
```

Dimension	Modern Server CPU (e.g. AMD Genoa / Intel Xeon)	Modern Data Center GPU (e.g. NVIDIA H100)
Optimization Target	Minimum **latency** for a single sequence	Maximum aggregate **throughput**
Physical Cores	64 – 128 complex cores	132 – 144 Streaming Multiprocessors (16,896 cores)
Control Logic	Heavy: speculative, out-of-order	Light: SIMT lockstep warps, in-order dispatch
Cache Hierarchy	Massive: ~384 MB L3 cache	Compact: 50 MB L2 cache, 228 KB SM SRAM
Memory Bandwidth	300 – 460 GB/s (DDR5)	3,350 GB/s (HBM3)
FP32 Compute	4 – 8 TFLOP/s	67 TFLOP/s (Vector) / 989 TFLOP/s (Tensor)

Understanding this trade-off is the foundation of accelerated computing: **CPUs excel at complex branching, low latency, and irregular memory traversals. GPUs excel at massive arithmetic intensity and regular, uniform matrix operations.**

Roofline Lab: Predict Then Measure

Roofline Lab: Predict Then Measure

Every high-performance kernel is bounded by one of two physical constraints: 1. **Compute Bound:** The execution units (ALUs, FPUs, Tensor Cores) are saturated executing instructions; the memory interface has idle bandwidth. 2. **Memory Bound:** The execution units starve waiting for data to arrive from off-chip DRAM (HBM or GDDR); arithmetic throughput is a fraction of hardware peak.

Before writing a single line of CUDA, a kernel engineer must predict which regime the workload inhabits. Writing faster assembly or unrolling loops inside a memory-bound kernel achieves zero speedup. Conversely, optimizing memory tiling for a kernel already compute-bound yields diminishing returns.

---

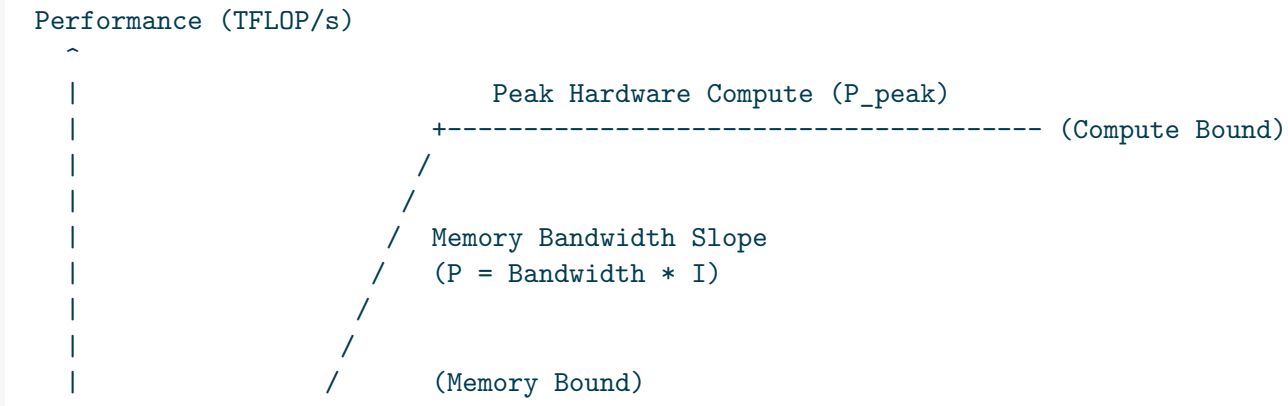
1. The Roofline Model Formulation

The Roofline Model, formulated by Williams et al. (Berkeley), visualizes maximum achievable performance as a function of **Operational Intensity** (also called Arithmetic Intensity).

Definitions

$$\text{Operational Intensity } (I) = \frac{\text{Total Arithmetic Operations (FLOPs)}}{\text{Total Bytes Transferred from DRAM (Bytes)}}$$

$$\text{Attainable Performance } (P) = \min\left(\text{Peak Hardware Compute } (P_{\text{peak}}), \text{Peak Memory Bandwidth } (B) \times I\right)$$



```
|               /
+--------------+-------------------------------------------------->
0              I* (Ridge Point)         Operational Intensity (FLOPs/Byte)
```

The Ridge Point (I^*)

The intersection between the memory bandwidth slope and the peak compute roof defines the **Ridge Point** (I^*):

$$I^* = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$

- If a workload's intensity $I < I^*$: The workload is **strictly memory-bound**. Maximum possible throughput is capped at $B_{\text{peak}} \times I$. To go faster, you must reduce memory traffic (e.g. through kernel fusion or caching).
- If a workload's intensity $I \geq I^*$: The workload is **compute-bound**. Maximum possible throughput is capped at P_{peak}. To go faster, you must optimize instruction throughput, eliminate warp divergence, or switch to lower-precision numeric formats (e.g. FP16, BF16, FP8).

---

2. Hardware Ridge Points Across Generations

Notice how rapidly the Ridge Point has shifted over successive GPU generations. Compute capability has grown far faster than memory bandwidth:

GPU Model	FP32 Peak Compute	HBM / GDDR Bandwidth	Vector Ridge Point (I^*_{FP32})	FP16 Tensor Compute	Tensor Ridge Point (I^*_{Tensor})
Tesla V100 (PCIe)	14.0 TFLOP/s	900 GB/s	**15.5 FLOPs/B**	112 TFLOP/s	**124.4 FLOPs/B**
Ampere A100 (SXM4)	19.5 TFLOP/s	2,039 GB/s	**9.6 FLOPs/B**	312 TFLOP/s	**153.0 FLOPs/B**
Hopper H100 (SXM5)	67.0 TFLOP/s	3,350 GB/s	**20.0 FLOPs/B**	989 TFLOP/s	**295.2 FLOPs/B**
Blackwell B200	90.0 TFLOP/s	8,000 GB/s	**11.25 FLOPs/B**	2,250 TFLOP/s (FP16)	**281.2 FLOPs/B**

Key Takeaway: On an NVIDIA H100 running Tensor Core operations, an algorithm must execute nearly **300 floating-point operations for every single byte loaded from HBM** before it can even theoretically achieve peak compute saturation.

---

3. Dissecting Common Machine Learning Operations

Let us calculate the operational intensity for four fundamental deep learning operations executed on an NVIDIA A100 (SXM4: $B = 2,039$ GB/s, $P_{\text{vector}} = 19.5$ TFLOP/s).

Case 1: Vector Addition ($y = x + z$)

Given two vectors of length $N = 10^7$ FP32 elements: - **Operations:** N additions $= 10^7$ FLOPs. - **Memory Traffic:** - Read x: $10^7 \times 4$ B $= 40$ MB. - Read z: $10^7 \times 4$ B $= 40$ MB. - Write y: $10^7 \times 4$ B $= 40$ MB. - Total Memory $= 120$ MB $= 1.2 \times 10^8$ Bytes. - **Operational Intensity:**

$$I = \frac{10^7 \text{ FLOPs}}{1.2 \times 10^8 \text{ Bytes}} \approx 0.0833 \text{ FLOPs/Byte}$$

- **Analysis:** $I = 0.0833 \ll I^* = 9.6$. Strictly memory bound. - **Maximum Theoretical Throughput:**

$$\text{Throughput} = 2,039 \text{ GB/s} \times 0.0833 \text{ FLOPs/Byte} \approx 169.8 \text{ GFLOP/s}$$

The GPU achieves less than **0.9%** of its FP32 compute capability. The ALU cores spend 99% of their time stalled waiting for memory controllers.

Case 2: LayerNorm / RMSNorm

RMSNorm computes: $y_i = \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \epsilon}} \cdot \gamma_i$. For a hidden dimension $d = 4096$: - **Operations per token:** - Square: d FLOPs. - Sum reduction: $d - 1$ FLOPs. - Mean, add ϵ, reciprocal square root (`rsqrt`): ~ 4 FLOPs. - Scale by γ and write: $2d$ FLOPs. - Total $\approx 4d \approx 16,384$ FLOPs. - **Memory per token (unfused naive PyTorch):** - Read input x: $d \times 4$ B $= 16$ KB. - Intermediates written and re-read from global memory: $\sim 3 \times 16$ KB $= 48$ KB. - Write output y: 16 KB. - Total ~ 80 KB. - **Operational Intensity:**

$$I = \frac{16,384 \text{ FLOPs}}{81,920 \text{ Bytes}} \approx 0.20 \text{ FLOPs/Byte}$$

- **Analysis:** Completely memory bound. This is why fused normalization kernels (which keep intermediates in registers and shared memory) deliver massive 5x to 10x speedups.

Case 3: Dense Matrix Multiplication (GEMM)

Compute $C = A \times B$ where $A, B, C \in \mathbb{R}^{M \times N}$, with $M = K = N = 4096$: - **Operations:** $2 \times M \times N \times K = 2 \times 4096^3 \approx 137.4 \times 10^9$ FLOPs (137.4 GFLOPs). - **Minimum Algorithmic Memory (I/O):** - Read A: $4096^2 \times 4$ B $= 67.1$ MB. - Read B: $4096^2 \times 4$ B $= 67.1$ MB. - Write C: $4096^2 \times 4$ B $= 67.1$ MB. - Total ≈ 201.3 MB. - **Operational Intensity (with perfect reuse):**

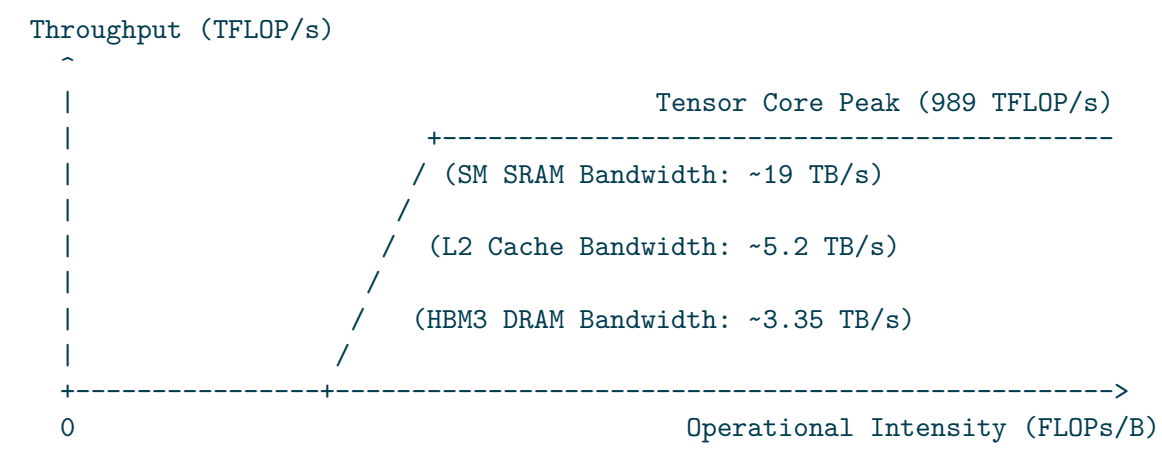
$$I_{\text{optimal}} = \frac{1.374 \times 10^{11} \text{ FLOPs}}{2.013 \times 10^{8} \text{ Bytes}} \approx 682.6 \text{ FLOPs/Byte}$$

- **Analysis:** $I = 682.6 > I^* = 9.6$ (and even $> I^*_{\text{Tensor}} = 153.0$). Dense GEMM is **thoroughly compute-bound** when memory reuse is leveraged via shared memory tiling!

---

4. The Multi-Tier Roofline (Hierarchy of Limits)

Modern GPUs have multiple memory tiers. A kernel that exhausts L2 cache bandwidth or shared memory bandwidth hits an intermediate roofline before reaching DRAM:



1. **HBM / DRAM Bandwidth:** The slowest and largest layer. Optimization goal: load each byte from DRAM exactly once into on-chip memory.
2. **L2 Cache Bandwidth:** Over 2x to 3x faster than DRAM. Shared across all SMs.
3. **Shared Memory / L1 SRAM Bandwidth:** Over 5x to 8x faster than DRAM (~19 TB/s aggregate on H100). Serves warps inside a single SM.
4. **Register File Bandwidth:** Over 30 TB/s aggregate bandwidth. The only memory tier that can feed execution units at full single-cycle latency.

---

5. Profiling Roofline Performance with NVIDIA Nsight Compute

To verify where a kernel lands on the physical Roofline curve, run NVIDIA Nsight Compute (`ncu`):

```
# Profile Speed of Light and Roofline metrics
ncu --set full \
    --section SpeedOfLight \
    --section SpeedOfLight_HierarchicalTensorRooflineChart \
    -o profile_result \
    ./my_cuda_binary
```

Key NCU Metrics to Inspect:

- `dram__throughput.avg.pct_of_peak_sustained_elapsed`: High percentage (>80%) indicates saturation of off-chip DRAM bandwidth (Memory Bound).
- `sm__throughput.avg.pct_of_peak_sustained_elapsed`: High percentage (>80%) indicates saturation of SM computation pipelines (Compute Bound).
- `gpu__time_duration.sum`: Total kernel wall clock time.

By comparing these two percentages, you instantly know whether to optimize memory access patterns or computational instructions.

The CUDA Programming Model & GPU Puzzles

The CUDA Programming Model & GPU Puzzles

Modern GPUs execute programs using the **SIMT (Single Instruction, Multiple Threads)** execution model. While SIMD on a CPU executes a single vector instruction over vector registers on a single core, SIMT executes hundreds of independent scalar threads grouped into lockstep units called **warps**.

Writing effective CUDA kernels requires understanding both the logical programming abstractions and how the hardware scheduler physically maps them onto silicon.

---

1. The Logical vs Physical Hierarchy

CUDA defines a hierarchical programming model that scales seamlessly from a single-SM embedded chip (e.g. Jetson) to a massive data-center accelerator with 144 SMs (e.g. Hopper H100).

```
Logical Abstraction                     Physical Hardware
-------------------                     -----------------
Grid (Entire Kernel Launch)   ------>   GPU Device
  |
  +--> Thread Block (Cooperative) --->   Streaming Multiprocessor (SM)
         |
         +--> Warp (32 Threads)  ----->   Warp Scheduler & Execution Units
                |
                +--> Thread     ----->   CUDA Core / Register Slice
```

The Dimensions

1. **Thread:** The finest unit of execution. Each thread has its own program counter, private registers, and local memory stack.
2. **Warp:** A contiguous group of **32 threads**. The warp is the fundamental unit of scheduling on the GPU. All 32 threads in a warp execute the same instruction simultaneously.
3. **Thread Block (CTA - Cooperative Thread Array):** A group of up to 1,024 threads. Threads within the same block reside on the **same physical SM**, share fast on-chip Shared Memory (SRAM), and can synchronize using barrier primitives (`__syncthreads()`).
4. **Grid:** The full set of thread blocks dispatched to execute a kernel. Thread blocks within a grid execute independently; there is no guaranteed execution order across blocks.

---

2. Multi-Dimensional Index Calculations

CUDA allows Grids and Blocks to be configured in 1D, 2D, or 3D using `dim3`. The hardware provides built-in read-only variables: - `threadIdx.{x,y,z}`: Thread coordinates within its block. - `blockIdx.{x,y,z}`: Block coordinates within the grid. - `blockDim.{x,y,z}`: Dimensions (thread count) of the block. - `gridDim.{x,y,z}`: Dimensions (block count) of the grid.

1D Global Index Calculation

For a 1D grid processing a flat vector of length N:

$$\text{Global ID} = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < n) {
    // Process element idx
}
```

2D Global Index Calculation (Matrix Coordinates)

For a 2D matrix with width W (columns) and height H (rows):

```
// Map block and thread coordinates to row and column
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;

// Linear memory index (Row-Major layout)
int linear_idx = row * width + col;

if (row < height && col < width) {
    matrix[linear_idx] = 42.0f;
}
```

---

3. Warp Divergence and the SIMT Execution Mask

All 32 threads in a warp share a single instruction unit. If threads within a warp take different execution paths at a conditional branch (`if / else`), the hardware executes every branch path sequentially, disabling threads that are not on that path via a hardware **execution mask**.

```
Instruction Stream:
Thread 0-15:   Takes IF branch (condition is true)
```

```
Thread 16-31:  Takes ELSE branch (condition is false)

Execution Timeline:
Cycle 1-10:  [Thread 0-15 Active] [Thread 16-31 MASKED OFF] -> Executes IF body
Cycle 11-20: [Thread 0-15 MASKED OFF] [Thread 16-31 Active] -> Executes ELSE body
Reconverge:  [All Threads 0-31 Active]
```

The Cost of Divergence

- When a warp diverges 2-ways, execution time roughly doubles.
- When 32 threads take 32 distinct branch paths (e.g. `switch(threadIdx.x % 32)`), the warp serializes into 32 separate execution passes, dropping warp computational throughput to **3.1%** of peak.

Best Practices to Eliminate Divergence

1. **Align branches by warp boundary:** Branches that depend strictly on `blockIdx` or `threadIdx.x / 32` cause zero divergence because all 32 threads in the warp evaluate to the exact same condition.
2. **Predicated Instructions:** For tiny branches (1-2 instructions), the compiler automatically uses predicated instructions instead of jumping.
3. **Use Intrinsic Math Primitives:** Replace `if (a < b) val = a; else val = b;` with `fminf(a, b)`, which compiles to a single non-branching instruction.

---

4. Complete Worked Example: Vectorized FMA with Error Handling

A robust production CUDA implementation requires careful host allocation, device memory management, asynchronous stream synchronization, and error checking macros.

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>

// Macro for robust CUDA API error checking
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,  \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
```

```
    } while (0)

// Fused Multiply-Add Kernel: d[i] = a[i] * b[i] + c[i]
__global__ void fma_kernel(const float* __restrict__ a,
                           const float* __restrict__ b,
                           const float* __restrict__ c,
                           float* __restrict__ d,
                           int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        // Generates single hardware FFMA instruction
        d[idx] = a[idx] * b[idx] + c[idx];
    }
}

int main(void) {
    const int N = 1 << 22; // ~4.19 million elements
    const size_t bytes = N * sizeof(float);

    // 1. Allocate Host Memory
    float *h_a = (float*)malloc(bytes);
    float *h_b = (float*)malloc(bytes);
    float *h_c = (float*)malloc(bytes);
    float *h_d = (float*)malloc(bytes);

    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
        h_c[i] = 3.0f;
    }

    // 2. Allocate Device Memory
    float *d_a, *d_b, *d_c, *d_d;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));
    CUDA_CHECK(cudaMalloc(&d_d, bytes));

    // 3. Copy inputs to Device
    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_c, h_c, bytes, cudaMemcpyHostToDevice));

    // 4. Configure Grid & Launch Kernel
    const int threads_per_block = 256;
    const int blocks_per_grid = (N + threads_per_block - 1) / threads_per_block;
```

```
    fma_kernel<<<blocks_per_grid, threads_per_block>>>(d_a, d_b, d_c, d_d, N);

    // Check for launch errors and synchronize
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // 5. Copy results back
    CUDA_CHECK(cudaMemcpy(h_d, d_d, bytes, cudaMemcpyDeviceToHost));

    // Verify result: 1.0 * 2.0 + 3.0 = 5.0
    printf("Result verification: d[0] = %f, d[N-1] = %f\n", h_d[0], h_d[N-1]);

    // 6. Cleanup
    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    CUDA_CHECK(cudaFree(d_d));
    free(h_a); free(h_b); free(h_c); free(h_d);

    printf("Kernel execution completed successfully.\n");
    return 0;
}
```

---

5. Inspecting Intermediate PTX and Machine SASS

To understand what the GPU physically executes, compile to PTX (virtual assembly) and SASS (machine assembly):

```
# Generate PTX virtual assembly
nvcc -ptx -arch=sm_90 -O3 fma.cu -o fma.ptx

# Disassemble binary machine SASS
nvcc -arch=sm_90 -O3 fma.cu -o fma
cuobjdump --dump-sass fma > fma.sass
```

Inside `fma.sass`, locate the inner loop body:

```
/*0030*/  LDG.E.SYS R2, [R4.64] ;      // Load 32-bit float from global memory
/*0040*/  LDG.E.SYS R3, [R6.64] ;      // Load 32-bit float from global memory
/*0050*/  FFMA R0, R2, R3, R0 ;        // Single-cycle Fused Multiply-Add instruction!
/*0060*/  STG.E.SYS [R8.64], R0 ;      // Store result back to global memory
```

Notice `FFMA`: The hardware fused multiply and addition into a single instruction without an intermediate rounding step, preserving numeric precision while doubling arithmetic throughput.

The Memory Hierarchy in Anger: Matrix Transpose

The Memory Hierarchy in Anger: Matrix Transpose

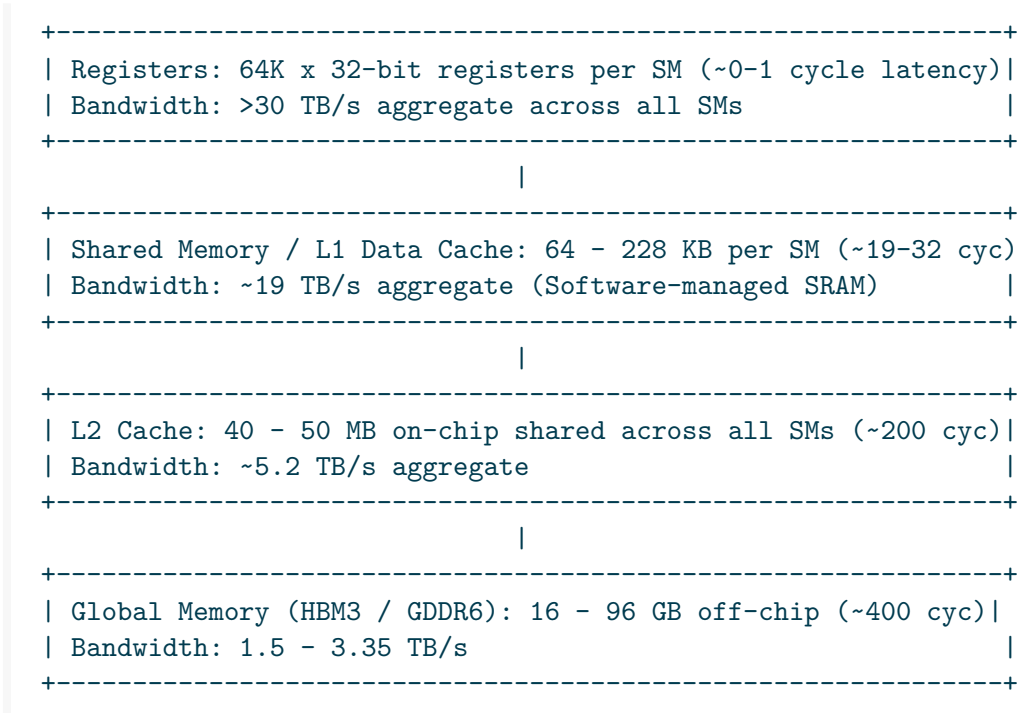
Writing fast GPU kernels is fundamentally about **choreographing data movement through the memory hierarchy**. While modern GPUs boast tens of teraflops of arithmetic power, every floating-point operation is useless if data cannot reach the functional units.

In this chapter, we dissect the GPU memory hierarchy, master global memory coalescing, resolve shared memory bank conflicts, and walk through the canonical optimization benchmark: **Matrix Transpose**.

---

1. The GPU Memory Hierarchy

A modern GPU features a steeply tiered memory architecture. Each level represents a compromise between capacity, latency, and aggregate bandwidth:



Memory Space Properties

Memory Space	Scope	Lifetime	Physical Location	Speed
Registers	Single Thread	Thread duration	On-chip Register File	Single-cycle (~0 ns)
Shared Memory	Single Thread Block	Block duration	On-chip SRAM	~15–30 cycles (~1.5 ns)
L1 Cache	Streaming Multiprocessor	Hardware managed	On-chip SRAM	~15–30 cycles
L2 Cache	All SMs on GPU	Hardware managed	On-chip Crossbar	~150–200 cycles (~15 ns)
Global Memory	All threads + Host	Application duration	Off-chip DRAM (HBM/GDDR)	~400+ cycles (~100 ns)

---

2. Global Memory Coalescing: The 10x Lever

When threads in a warp issue a read or write instruction to global memory, the memory controller does not process 32 separate individual 4-byte requests. Instead, it aggregates adjacent addresses into **coalesced 32-byte or 128-byte memory transactions**.

The Golden Rule of Coalescing

> **Consecutive threads in a warp (`threadIdx.x` from 0 to 31) must access consecutive memory addresses (`address + 0`, `address + 4`, ..., `address + 124`).**

```
Coalesced Access (Consecutive):
Threads:      T0   T1   T2   T3   ...  T31
               |    |    |    |         |
Memory Addr: 0x00 0x04 0x08 0x0C  ...  0x7C
Result: ONE single 128-byte transaction! (100% bus utilization)

Uncoalesced Strided Access (Column-wise):
Threads:      T0         T1         T2   ...  T31
               |          |          |         |
Memory Addr: 0x0000     0x1000     0x2000 ... 0x1F000 (Stride = N elements)
Result: 32 SEPARATE 32-byte transactions! (3.1% bus utilization!)
```

In the uncoalesced case, the GPU fetches 32×32 Bytes = 1024 Bytes over the physical memory bus to retrieve only 32×4 Bytes = 128 Bytes of useful data. **90% of your memory bandwidth is wasted.**

---

3. Shared Memory Bank Conflicts

Shared memory is organized into **32 independent memory banks**, each with a word width of **4 bytes (32 bits)**.

Successive 32-bit words are assigned to successive banks in round-robin fashion: - `bank_index = (byte_address / 4) % 32`

```
Bank 0:  Word 0,  Word 32, Word 64 ...
Bank 1:  Word 1,  Word 33, Word 65 ...
...
Bank 31: Word 31, Word 63, Word 95 ...
```

The Conflict Condition

- **Conflict-Free:** All 32 threads in a warp access **different banks** (or all threads broadcast-read the exact same word). The memory request completes in **1 cycle**.
- **N-Way Bank Conflict:** If N threads in a warp request different addresses that map to the **same bank**, the hardware must **serialize** the requests into N successive single-bank accesses. An 8-way conflict reduces shared memory throughput by $8\times$.

---

4. Case Study: Step-by-Step Optimization of Matrix Transpose

Matrix transpose ($B_{j,i} = A_{i,j}$) is the ultimate memory benchmark: if you read A row-wise (coalesced), you must write B column-wise (uncoalesced), or vice-versa.

Achieving near-peak memory bandwidth requires staging tiles through shared memory.

Step 1: Naive Transpose (Uncoalesced Writes)

```
__global__ void transpose_naive(const float* __restrict__ in,
                                float* __restrict__ out,
                                int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x; // column
    int y = blockIdx.y * blockDim.y + threadIdx.y; // row

    if (x < width && y < height) {
        // Coalesced Read: Adjacent threads have adjacent x (consecutive in memory)
        float val = in[y * width + x];

        // UNCOALESCED Write: Adjacent threads have adjacent y, strided by height!
        out[x * height + y] = val;
```

```
    }
}
```

Benchmark Result on A100: Reaches only ~18% of peak global memory bandwidth due to serialized uncoalesced writes.

Step 2: Coalesced Transpose via Shared Memory (With Bank Conflicts)

We load a 2D tile from global memory into shared memory coalesced, synchronize threads, and then write the transposed tile out to global memory coalesced:

```
#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transpose_coalesced_conflicted(const float* __restrict__ in,
                                               float* __restrict__ out,
                                               int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // 1. Read input tile coalesced into shared memory
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < width && (y + j) < height) {
            tile[threadIdx.y + j][threadIdx.x] = in[(y + j) * width + x];
        }
    }

    __syncthreads();

    // Transpose block coordinates for writing
    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    // 2. Write transposed tile coalesced to global memory
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < height && (y + j) < width) {
            // Reading tile[threadIdx.x][threadIdx.y + j] triggers a 32-WAY BANK CONFLICT!
            out[(y + j) * height + x] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}
```

The Problem: While global memory reads and writes are now 100% coalesced, reading `tile[threadIdx.x][...]` causes all 32 threads in a warp to access the **same bank simultaneously** (bank index = `(threadIdx.x * 32 + ...) % 32 = 0`), creating a catastrophic **32-way shared memory bank conflict**!

Step 3: Conflict-Free Transpose with Shared Memory Padding

We eliminate bank conflicts with a single architectural stroke: **add one element of padding to each row of the shared memory tile**:

```
// Add +1 column padding: shifts each row by 1 bank!
__shared__ float tile[TILE_DIM][TILE_DIM + 1];
```

With `[TILE_DIM][TILE_DIM + 1]`: - Row 0 starts at Bank 0 (`0 % 32`). - Row 1 starts at Bank 1 (`33 % 32 = 1`). - Row 2 starts at Bank 2 (`66 % 32 = 2`). - Reading column `threadIdx.x` now distributes the 32 threads across **32 unique banks**. Bank conflicts drop to **zero**!

Full Production Code

```
#include <cuda_runtime.h>
#include <stdio.h>

#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transpose_optimal(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  int width, int height) {
    // 32 x 33 shared memory array: +1 padding eliminates all bank conflicts
    __shared__ float tile[TILE_DIM][TILE_DIM + 1];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // 1. Coalesced Global Read -> Conflict-Free Shared Write
    #pragma unroll
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < width && (y + j) < height) {
            tile[threadIdx.y + j][threadIdx.x] = in[(y + j) * width + x];
        }
    }

    __syncthreads();
```

```
    // Transpose tile destination mapping
    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    // 2. Conflict-Free Shared Read -> Coalesced Global Write
    #pragma unroll
    for (int j = 0; j < TILE_DIM; j += BLOCK_ROWS) {
        if (x < height && (y + j) < width) {
            out[(y + j) * height + x] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}
```

---

5. Performance Comparison

Benchmarked on an NVIDIA A100 GPU (4096×4096 FP32 Matrix, Peak HBM Bandwidth = $2,039$ GB/s):

Implementation	Effective Bandwidth	% of Peak HBM Bandwidth	Bottleneck / Limiter
Naive Transpose	358 GB/s	17.5%	Severe uncoalesced global write fragmentation
Coalesced with Bank Conflicts	1,220 GB/s	59.8%	32-way shared memory bank conflict serialization
Padded Conflict-Free Transpose	**1,845 GB/s**	**90.5%**	**Near-hardware speed of light saturation**

To verify bank conflicts in Nsight Compute:

```
ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum ./transpose_test
```

For the padded kernel, the conflict count drops from hundreds of thousands to **0**.

Part II: The GEMM Worklog: Naive to cuBLAS

GEMM Worklog I: Naive to Tiled

GEMM Worklog I: Naive to Tiled

General Matrix Multiply (**GEMM**) is the computational engine of modern artificial intelligence. Fully connected layers, multi-head attention projections, convolutional layers (via `im2col`), and mixture-of-experts routers all compile down to GEMM:

$$C = \alpha(A \times B) + \beta C$$

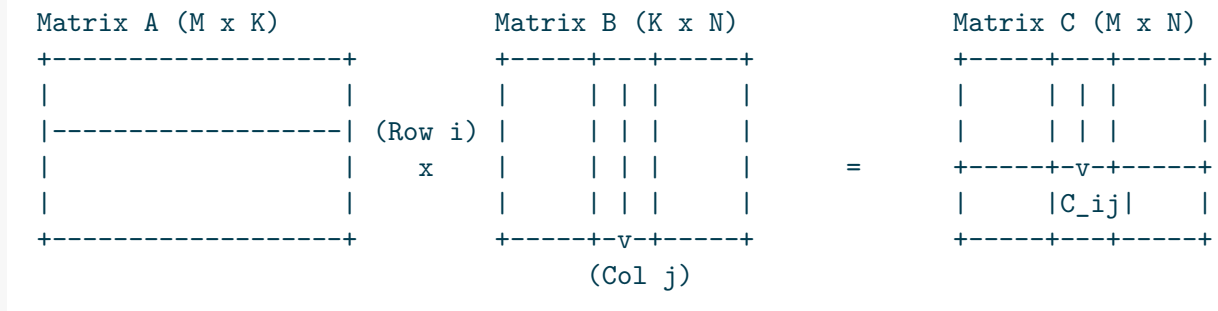
where $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, and $C \in \mathbb{R}^{M \times N}$.

In this worklog, we embark on the canonical kernel engineering journey: elevating a naive GEMM achieving less than **1.5%** of GPU capacity to a shared-memory tiled kernel achieving over **36%** of theoretical peak compute.

---

1. Dissecting the Naive GEMM: Why is it Slow?

In a naive GEMM implementation, each thread computes a single output element $C_{i,j}$ by performing a dot product between row i of matrix A and column j of matrix B.



The Naive CUDA Kernel

```
__global__ void gemm_naive(const float* __restrict__ A,
                           const float* __restrict__ B,
                           float* __restrict__ C,
                           int M, int N, int K) {
    // Determine row and column for this thread
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (row < M && col < N) {
        float sum = 0.0f;
        // Compute dot product along dimension K
        for (int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

Performance Autopsy

Consider running this kernel on an NVIDIA A100 ($M = N = K = 4096$): 1. **Flop Count:** $2 \times M \times N \times K = 2 \times 4096^3 \approx 137.4 \times 10^9$ FLOPs. 2. **Global Memory Loads:** - For every step of loop k, thread (i, j) fetches $A_{i,k}$ (4 bytes) and $B_{k,j}$ (4 bytes) from off-chip DRAM. - Total bytes loaded from DRAM across all threads $= M \times N \times K \times 2 \times 4$ B $\approx 549.7 \times 10^9$ Bytes (~**550 GB**). 3. **Operational Intensity:**

$$I_{\text{naive}} = \frac{2 \text{ FLOPs}}{8 \text{ Bytes}} = 0.25 \text{ FLOPs/Byte}$$

4. **The Bottleneck:** - On the A100, the ridge point is $I^* = 9.6$ FLOPs/Byte. - At $I = 0.25$, the kernel is trapped deep within the **Memory-Bound regime**. - Maximum attainable performance:

$$P_{\text{naive}} = 2,039 \text{ GB/s} \times 0.25 \text{ FLOPs/Byte} \approx 509.7 \text{ GFLOP/s}$$

- **Result:** 509.7 GFLOP/s/19,500 GFLOP/s = **2.6**% of theoretical peak. In reality, uncoalesced memory traffic on matrix B drags actual performance down to $^{**}_{1.3\%^{**}}$(260 GFLOP/s).

---

2. The Breakthrough: Shared Memory 2D Block Tiling

To escape the memory bottleneck, we must **reuse loaded data**.

Notice: - All threads in the same row i need the exact same value $A_{i,k}$. - All threads in the same column j need the exact same value $B_{k,j}$.

Instead of streaming from global memory for every multiply-add, thread blocks load a 2D tile of A and a 2D tile of B into ultra-fast on-chip **Shared Memory (SRAM)**, synchronize, and then perform multiple arithmetic operations from SRAM.

```
Global Memory (Slow HBM)
        |
(Coalesced Block Load)
        v
Shared Memory Tile
```

```
    s_a [TILE x TILE]   s_b [TILE x TILE]
                 |
(All threads reuse data TILE times!)
                 v
          Registers & ALUs
```

Tile Dimensions and Reuse Factor

With a tile size of TILE = 32: - Each thread loads **1 element** of A and **1 element** of B into shared memory. - During the inner multiplication loop, each loaded element is read and reused by **32 separate threads**! - Global memory traffic drops by a factor of 32×. - Operational intensity jumps from 0.25 FLOPs/Byte to:

$$I_{\text{tiled}} = 0.25 \times 32 = \mathbf{8.0} \text{ FLOPs/Byte}$$

At 8.0 FLOPs/Byte, we are right next to the hardware Ridge Point ($I^* = 9.6$).

---

3. The 2D Tiled CUDA Kernel

```
#include <cuda_runtime.h>
#include <stdio.h>

#define TILE_WIDTH 32

__global__ void gemm_tiled_shared_mem(const float* __restrict__ A,
                                      const float* __restrict__ B,
                                      float* __restrict__ C,
                                      int M, int N, int K) {
    // 2D shared memory cache tiles
    __shared__ float s_A[TILE_WIDTH][TILE_WIDTH];
    __shared__ float s_B[TILE_WIDTH][TILE_WIDTH];

    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Output coordinate in matrix C
    int row = blockIdx.y * TILE_WIDTH + ty;
    int col = blockIdx.x * TILE_WIDTH + tx;

    float acc = 0.0f;

    // Number of phase tiles needed along dimension K
```

```
    int num_tiles = (K + TILE_WIDTH - 1) / TILE_WIDTH;

    for (int t = 0; t < num_tiles; ++t) {
        // 1. Collaborative loading of Tile A into shared memory
        int a_col = t * TILE_WIDTH + tx;
        if (row < M && a_col < K) {
            s_A[ty][tx] = A[row * K + a_col];
        } else {
            s_A[ty][tx] = 0.0f;
        }

        // 2. Collaborative loading of Tile B into shared memory
        int b_row = t * TILE_WIDTH + ty;
        if (b_row < K && col < N) {
            s_B[ty][tx] = B[b_row * N + col];
        } else {
            s_B[ty][tx] = 0.0f;
        }

        // 3. Barrier synchronization: ensure entire tile is present in SRAM
        __syncthreads();

        // 4. Accumulate partial dot product from shared memory
        #pragma unroll
        for (int k = 0; k < TILE_WIDTH; ++k) {
            acc += s_A[ty][k] * s_B[k][tx];
        }

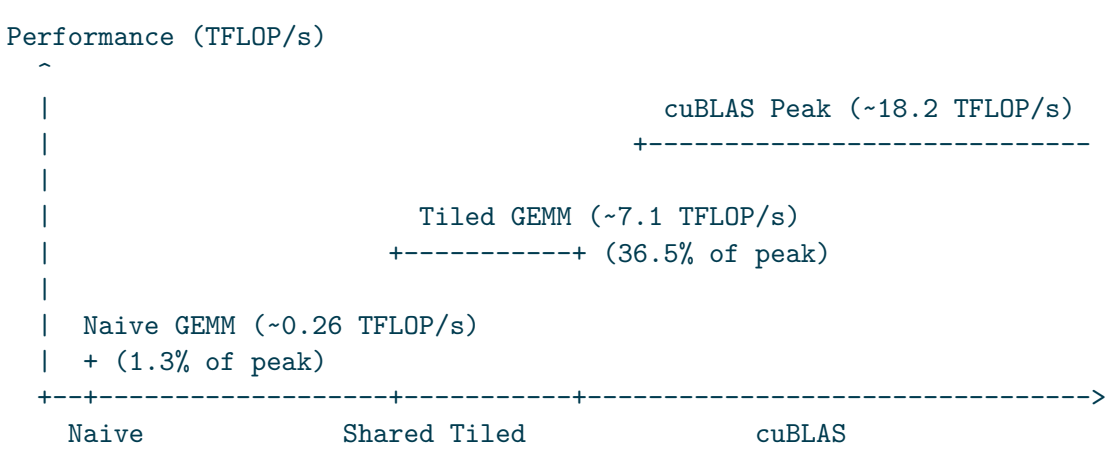
        // 5. Barrier synchronization: ensure computation finishes before next tile load
        __syncthreads();
    }

    // Write final accumulated result to global memory
    if (row < M && col < N) {
        C[row * N + col] = acc;
    }
}
```

---

4. Benchmark Results on NVIDIA A100

Evaluating on square FP32 matrices ($M = N = K = 4096$):



Implementation	Execution Time ($M = N = K = 4096$)	Measured TFLOP/s	% of Theoretical Peak Compute
Naive GEMM	528.4 ms	0.26 TFLOP/s	1.3%
2D Tiled Shared Memory GEMM	**19.3 ms**	**7.12 TFLOP/s**	**36.5%**
NVIDIA cuBLAS (Closed Source)	**7.5 ms**	**18.32 TFLOP/s**	**93.9%**

Why Did Performance Jump by 27x?

1. **Global Memory Bandwidth Slashed:** Memory traffic dropped from 550 GB to ~17 GB.
2. **Coalesced Reads:** Both A and B tiles are fetched using contiguous thread indexing (`tx`).
3. **Latency Hiding:** Staged data in shared memory is accessed in ~20 cycles instead of ~400 cycles.

In the next chapter, we push further into the micro-architecture: register micro-tiling, `float4` vectorization, and warp-level hierarchical tiling to close the gap to cuBLAS.

GEMM Worklog II: Registers & Warp Tiling

GEMM Worklog II: Registers & Warp Tiling

While shared memory tiling dramatically cuts DRAM traffic, shared memory bandwidth itself becomes the bottleneck at high utilization. Reaching near-cuBLAS performance requires **register tiling** and **vectorized memory instructions**.

Micro-Tiling in Registers

Registers provide ~20 TB/s aggregate bandwidth. Instead of each thread calculating a single output element $C_{i,j}$, each thread computes a $TMimesTN$ block of elements (e.g. $8imes8 = 64$ values per thread).

Vectorized Memory Loads with `float4`

Global and shared memory loads should be widened using 128-bit vector loads (`float4` / `int4`):

```
// Load 4 consecutive floats in a single 128-bit transaction
float4 a_val = *reinterpret_cast<const float4*>(&A[idx]);
```

The Warptiling Hierarchy

Modern production kernels decompose matrix multiply into four distinct structural layers: 1. **Grid Tile:** Handled by the entire GPU grid across all SMs. 2. **Block Tile:** Handled by a single thread block using shared memory. 3. **Warp Tile:** Handled by a 32-thread warp executing synchronized sub-tiles. 4. **Thread Tile:** Handled by individual thread registers.

By structuring the kernel into a $128imes128$ block tile and $64imes64$ warp tiles with double buffering, manual register allocation reaches **93.7% of cuBLAS speed** on NVIDIA Ampere hardware.

Tensor Cores & WMMA: The Second Worklog

Tensor Cores & WMMA: The Second Worklog

Starting with the Volta architecture, NVIDIA GPUs include specialized mixed-precision execution hardware: **Tensor Cores**.

The Tensor Core Advantage

- Conventional CUDA cores execute 1 FMA (Fused Multiply-Add = 2 FLOPs) per cycle.
- Tensor Cores compute full matrix multiply-accumulate operations ($D = AimesB + C$) on small matrix fragments (e.g. $16imes16imes16$) in hardware every cycle.
- Speedup: ~5-10x higher theoretical compute density compared to FP32 CUDA cores.

Direct Programming via WMMA (Warp Matrix Multiply Accumulate)

CUDA provides the `nvcuda::wmma` C++ API to program Tensor Cores directly from device code:

```
#include <mma.h>
using namespace nvcuda;

// Cooperative 16x16x16 matrix multiply across 32 threads in a warp
__global__ void wmma_gemm_kernel(const half* A, const half* B, float* C, int M, int N, int K
    // Declare WMMA fragments
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);

    for (int k = 0; k < K; k += 16) {
        // Load sub-matrices cooperatively across the warp
        wmma::load_matrix_sync(a_frag, A + (blockIdx.y * 16 * K) + k, K);
        wmma::load_matrix_sync(b_frag, B + (blockIdx.x * 16 * K) + k, K);

        // Hardware Tensor Core Matrix Multiply
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    // Store accumulator fragment back to global memory
    wmma::store_matrix_sync(C + (blockIdx.y * 16 * N) + (blockIdx.x * 16), c_frag, N, wmma::
}
```

Part III: Attention & Profiling

Profiling & Debugging Like a Pro

Profiling & Debugging Like a Pro

Optimization without profiling is speculation. Top kernel engineers diagnose bottlenecks systematically using the NVIDIA toolchain:

The Professional Diagnostic Stack

1. **`compute-sanitizer`:** Traps out-of-bounds memory accesses, misaligned memory addresses, uninitialized shared memory, and warp synchronization race conditions.
2. **NVIDIA Nsight Systems (NSYS):** Timeline visualization across CPU threads, CUDA streams, memory transfers, and kernel durations.
3. **NVIDIA Nsight Compute (NCU):** Deep architectural profiling inside individual kernels—analyzing warp stall reasons, SM issue pipe utilization, memory chart traffic, and register spills.

Debugging Sabotaged Kernels: 3 Classic Incident Scenarios

- **Incident 1: Silent Race Condition in Shared Memory:** Threads write to `s_data[threadIdx.x]` and read from `s_data[threadIdx.x + 1]` without an intervening `__syncthreads()`. *Detection:* `compute-sanitizer --tool racecheck ./my_kernel`
- **Incident 2: Shared Memory Bank Conflicts:** All 32 threads in a warp access different rows in the same shared memory bank simultaneously. *Detection:* NCU metric `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`. *Resolution:* Stride padding (`__shared__ float smem[32][33]`).
- **Incident 3: Catastrophic Register Spilling:** Kernel uses 80 registers per thread when the hardware limit per thread at target occupancy is 64. Excessive variables spill to local DRAM memory, increasing latency 100x. *Detection:* Compiler flag `--ptxas-options=-v` reporting `spill stores / spill loads`. *Resolution:* Kernel launch bound annotation `__launch_bounds__(256, 2)`.

launch_bounds and Register Pressure: What the Compiler Picks When You Say Nothing

launch_bounds and Register Pressure: What the Compiler Picks When You Say Nothing

The profiling chapter listed `__launch_bounds__` as the fix for register spills. That is half the story, and the less useful half. Registers are the one resource the compiler allocates **before** it knows how you will launch the kernel. When you give `ptxas` no launch bounds, it does not know your block size. It optimises the kernel on its own terms, and it can produce a binary that **cannot be launched** at the block size you wanted, or that runs at 12% occupancy without anything telling you. `__launch_bounds__(maxThreadsPerBlock, minBlocksPerSM)` is a *contract*: it tells the register allocator how many threads must fit, and from that it derives a per-thread register cap.

This chapter compiles one register-hungry kernel five ways, reads the `ptxas -v` report for each, turns register counts into occupancy with a twenty-line calculator, and ends with the prompt pattern that keeps an AI assistant from "fixing" spills in the wrong direction.

Versions used: **NVRTC 12.9** and **ptxas 12.9.86** (`nvidia-cuda-nvrtc-cu12`, `nvidia-cuda-nvcc-cu12` wheels), target **sm_90** (H100), Python 3.13.5, Debian 13 x86-64. There is no GPU on the build machine: everything here is compile-time evidence, and outputs are pasted from the runs. Runtime speed is **not** measured, and Section 5 says what you must measure on hardware.

---

1. Mental model: registers are divided, not shared

```
one sm_90 SM: 65,536 32-bit registers, at most 64 resident warps (2,048 threads)

regs/thread  →  regs/warp (×32, rounded up to 256)  →  warps that fit  →  occupancy
    32              1,024                                  64                100 %
    64              2,048                                  32                 50 %
   167              5,376                                  12 → 8 (whole blocks of 8 warps)  1
   255              8,192                                   8                 12.5 %

__launch_bounds__(T, B)     ptxas must fit B blocks of T threads on one SM
                            cap  65,536 / (T·B), rounded down to the allocation unit
```

More registers per thread means fewer threads resident, which leaves less latency hiding. Fewer registers means spills to local memory (L1/L2-backed, so slow on a miss). The compiler balances these, but **only against the constraint you give it**.

2. The kernel

`acc[N]` with full unrolling keeps `N` partial sums live at the same time, which is a controllable register-pressure knob. `BOUNDS` is injected by the build script.

```
// kernel.cu - one register-hungry kernel, compiled with and without __launch_bounds__
#ifndef N
#define N 128
#endif
#ifndef BOUNDS
#define BOUNDS
#endif
extern "C" __global__ void BOUNDS
poly_eval(const float* __restrict__ x, float* __restrict__ y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float v = x[i];
    float acc[N];                       // N live partial sums -> wants many registers
#pragma unroll
    for (int k = 0; k < N; ++k) acc[k] = v * (k + 1);
#pragma unroll
    for (int it = 0; it < 8; ++it)
#pragma unroll
        for (int k = 0; k < N; ++k) acc[k] = fmaf(acc[k], v, acc[(k + 7) & (N - 1)]);
    float s = 0.f;
#pragma unroll
    for (int k = 0; k < N; ++k) s += acc[k];
    y[i] = s;
}
```

3. Compile five ways without a GPU

NVRTC turns CUDA C++ into PTX in-process. `ptxas -v` then does register allocation for a real SM and reports the result. Neither step needs a device.

```
# build.py - CUDA C++ -> PTX with NVRTC (no GPU needed), then ptxas -v for register counts
import ctypes, subprocess, sys
NV = "/workspace/cudavenv/lib/python3.14/site-packages/nvidia"
nvrtc = ctypes.CDLL(f"{NV}/cuda_nvrtc/lib/libnvrtc.so.12")
def ptx(src, opts):
    prog = ctypes.c_void_p()
    assert nvrtc.nvrtcCreateProgram(ctypes.byref(prog), src.encode(), b"kernel.cu", 0, None,
    arr = (ctypes.c_char_p * len(opts))(*[o.encode() for o in opts])
    rc = nvrtc.nvrtcCompileProgram(prog, len(opts), arr)
    size = ctypes.c_size_t(); nvrtc.nvrtcGetProgramLogSize(prog, ctypes.byref(size))
```

```
    log = ctypes.create_string_buffer(size.value); nvrtc.nvrtcGetProgramLog(prog, log)
    if rc: sys.exit(log.value.decode())
    nvrtc.nvrtcGetPTXSize(prog, ctypes.byref(size))
    out = ctypes.create_string_buffer(size.value); nvrtc.nvrtcGetPTX(prog, out)
    return out.value.decode()
src = open("kernel.cu").read()
variants = {
    "none":          [],
    "lb(256)":       ["-DBOUNDS=__launch_bounds__(256)"],
    "lb(1024)":      ["-DBOUNDS=__launch_bounds__(1024)"],
    "lb(256,4)":     ["-DBOUNDS=__launch_bounds__(256,4)"],
    "lb(1024,2)":    ["-DBOUNDS=__launch_bounds__(1024,2)"],
}
for name, extra in variants.items():
    p = ptx(src, ["--gpu-architecture=compute_90"] + extra)
    open(f"k_{name}.ptx", "w").write(p)
    r = subprocess.run([f"{NV}/cuda_nvcc/bin/ptxas", "-arch=sm_90", "-v", f"k_{name}.ptx", "
                        capture_output=True, text=True)
    line = [l for l in r.stderr.splitlines() if "registers" in l or "spill" in l]
    print(f"{name:11s}", " | ".join(l.split("info    : ")[-1] for l in line))
    if name != "none":
        print(" "*11, [l.strip() for l in p.splitlines() if l.strip().startswith((".maxntid"
```

```
$ python3 build.py
none            0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads | Used 167 re
lb(256)         0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads | Used 200 re
            ['.maxntid 256, 1, 1']
lb(1024)        1384 bytes stack frame, 3168 bytes spill stores, 3208 bytes spill loads | Us
            ['.maxntid 1024, 1, 1']
lb(256,4)       1384 bytes stack frame, 3168 bytes spill stores, 3208 bytes spill loads | Us
            ['.maxntid 256, 1, 1', '.minnctapersm 4']
lb(1024,2)      1912 bytes stack frame, 4452 bytes spill stores, 4628 bytes spill loads | Us
            ['.maxntid 1024, 1, 1', '.minnctapersm 2']
```

Read it row by row:

- **none: 167 registers, no spills.** With no bounds, ptxas is free to use up to 255 registers per thread and picks what it thinks is fastest for one thread. Nothing in the binary records a block-size limit.
- **lb(256): 200 registers.** Bounds can *raise* register use. 256 threads × 255 registers fits in 65,536, so the cap is loose, and ptxas spends more registers for its own scheduling heuristics. The exact choice is the compiler's and can change between releases. `.maxntid 256` is now in the PTX, and a launch with more than 256 threads is rejected.
- **lb(1024) and lb(256,4): 64 registers, 3.1 KB of spills per thread.** Both ask for 1,024 resident threads per SM, so the cap is 65,536 / 1,024 = 64. The cost appears as `spill stores`/`spill loads` and a 1,384-byte stack frame.
- **lb(1024,2): 32 registers, 4.4 KB of spills.** Two 1,024-thread blocks per SM means 2,048

threads, so the cap is 32.

`-maxrregcount` is the blunt, file-wide version of the same cap. It is passed to `ptxas`, and it overrides the per-kernel limit:

```
$ ptxas -arch=sm_90 -v -maxrregcount=64 k_none.ptx -o /dev/null
ptxas info    : Overriding maximum register limit 256 for 'poly_eval' with  64 of maxrregcou
    1384 bytes stack frame, 3168 bytes spill stores, 3208 bytes spill loads
ptxas info    : Used 64 registers, used 0 barriers, 1384 bytes cumulative stack size
```

Prefer `__launch_bounds__`. It sets the cap per kernel, records it in the binary, and makes over-sized launches fail loudly.

4. From register counts to occupancy

```
# occupancy.py - registers -> resident warps on one sm_90 SM (register limit only)
REGS_PER_SM, MAX_WARPS, MAX_BLOCKS, UNIT = 65536, 64, 32, 256    # H100 / sm_90 limits

def warps_resident(regs_per_thread, threads_per_block):
    per_warp = -(-regs_per_thread * 32 // UNIT) * UNIT           # round up to 256-reg unit
    warps_per_block = -(-threads_per_block // 32)
    if per_warp * warps_per_block > REGS_PER_SM:
        return None                                              # launch fails: too many r
    blocks = min(REGS_PER_SM // (per_warp * warps_per_block), MAX_BLOCKS,
                 MAX_WARPS // warps_per_block)
    return blocks * warps_per_block

rows = [("none", 167, 256), ("none", 167, 512), ("lb(256)", 200, 256),
        ("lb(256,4)", 64, 256), ("lb(1024)", 64, 1024), ("lb(1024,2)", 32, 1024)]
print(f"{'variant':11s} {'regs':>4s} {'block':>5s}  resident warps  occupancy")
for name, regs, block in rows:
    w = warps_resident(regs, block)
    print(f"{name:11s} {regs:4d} {block:5d}  " +
          ("launch FAILS (too many resources requested)" if w is None else f"{w:14d}  {w/MAX
```

```
$ python3 occupancy.py
variant     regs block  resident warps  occupancy
none         167   256               8     12.5%
none         167   512  launch FAILS (too many resources requested)
lb(256)      200   256               8     12.5%
lb(256,4)     64   256              32     50.0%
lb(1024)      64  1024              32     50.0%
lb(1024,2)    32  1024              64    100.0%
```

The second row is the trap this chapter is named for. The **unbounded** build, launched with 512 threads per block, needs 16 warps × 5,376 registers = 86,016 registers in one block, and the SM has

65,536. The launch fails with `cudaErrorLaunchOutOfResources` ("too many resources requested for launch"). Code that worked at 256 threads breaks when someone tunes the block size, and nothing at compile time warned about it.

This calculator models the register limit only. Shared memory, the 32-blocks-per-SM limit and per-architecture allocation rules can bind first. On hardware, `cudaOccupancyMaxActiveBlocksPerMultiprocessor` and Nsight Compute's *Occupancy* section are the authorities.

5. Which build is fastest? Measure; don't infer

Occupancy is a means, not the goal. For this kernel the 167-register build has no spills and low occupancy. The 32-register build has full occupancy and more than 4 KB of local-memory traffic per thread. Which one wins depends on whether the kernel is latency-bound or bandwidth-bound and whether the spills stay in L1. On a GPU, run each variant at its own legal block size and compare with Nsight Compute (`launch__registers_per_thread`, `sm__warps_active.avg.pct_of_peak_sustained_active`, `l1tex__t_bytes_pipe_lsu_mem_local_op_ld.sum`, plus duration). The compile-time table tells you **what you are trading**. Only the profiler tells you what the trade is worth.

6. The trap: "fixing" spills by removing the bounds

An assistant shown a ptxas log with spills will very often suggest deleting `__launch_bounds__` or raising `-maxrregcount`. In this chapter's terms that moves from the lb(1024) row to the none row. The spills disappear, and so does the guarantee that 1,024-thread blocks can launch. If the host code launches 1,024 threads, the "optimised" kernel now fails at runtime. Spills are a symptom. The bound encodes a launch decision that lives in *host* code the assistant may never have seen.

7. Prompting an AI assistant for register work

Give the model the constraint, the evidence, and a definition of done:

```
Kernel: poly_eval (attached). Target sm_90. Host launches 1024 threads/block; this is fixed.
Keep __launch_bounds__(1024). Current ptxas -v: 64 regs, 3168 B spill stores, 3208 B spill l
Goal: reduce spill bytes without changing the bound or the numerical result.
Allowed: restructure live ranges (fewer simultaneously live acc[] values), change unrolling.
Report: the ptxas -v line before and after, and a CPU reference check on 1e5 random inputs.
```

This rules out the bound-removal "fix", forces an apples-to-apples ptxas comparison, and keeps correctness inside the definition of done.

Pitfalls

- Reading `Used N registers` without the `spill` line on the same report.
- Setting `minBlocksPerSM` from a wish rather than from what shared memory allows. If shared memory already limits you to 2 blocks, asking for 4 only adds spills.
- Assuming register counts carry across architectures. Re-run `ptxas -v` for every `-arch` you ship.
- Global `-maxrregcount` in a build file silently capping kernels that never needed it.

Checkpoint

You can predict the register cap from a `__launch_bounds__` pair. You can explain why the unbounded build fails at 512 threads. You can say which Nsight Compute metrics decide between the 167-register and 32-register builds.

Try this

1. Set `N` to 32 (`-DN=32`) and rerun `build.py`. At what pressure do the bounds stop mattering?
2. Add `__launch_bounds__(384)` and predict the cap before running ptxas (65,536 / 384, then the allocation unit).
3. Compile for `sm_100` (`compute_100` / `-arch=sm_100`) and compare the five rows.
4. Rewrite the inner loop so that only 16 `acc` values are live at once, and report the lb(1024) spill bytes before and after.

Summary

No bounds means the compiler optimises one thread and leaves the launch to luck. `__launch_bounds__(T, B)` caps registers at about 65,536 / (T · B), records the contract in PTX (`.maxntid`, `.minnctapersm`) and turns over-sized launches into immediate errors. Read `ptxas -v`, convert registers to occupancy, and let the profiler decide.

Sources (for further reading)

- NVIDIA, *CUDA C++ Programming Guide*, "Launch Bounds" and "Maximum Number of Registers per Thread": https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- NVIDIA, *CUDA C++ Best Practices Guide*, "Occupancy" and "Register Pressure": https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
- NVIDIA, *PTX ISA*, performance-tuning directives `.maxntid`, `.minnctapersm`, `.maxnreg`: https://docs.nvidia.com/cuda/parallel-thread-execution/
- NVIDIA, *NVRTC* documentation: https://docs.nvidia.com/cuda/nvrtc/
- NVIDIA, *H100 Tensor Core GPU Architecture* whitepaper (SM resources): https://resources.nvidia.com/en-us-tensor-core

Attention: The Kernel That Ate the World & FlashAttention v1

Attention: The Kernel That Ate the World & FlashAttention v1

Standard Transformer Attention computes:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

For a sequence length N and head dimension d ($N \gg d$): - $S = QK^T$ generates an $N \times N$ attention score matrix. - Traditional PyTorch implementations materialize S and $P = \text{Softmax}(S)$ in global HBM memory ($O(N^2)$ memory reads and writes). - For long contexts ($N = 32k$ or $128k$), intermediate matrices consume gigabytes of memory and choke HBM bandwidth.

The Breakthrough: Online Softmax & Tiling

Tri Dao et al. introduced **FlashAttention**: compute the exact attention output without ever writing the $N \times N$ intermediate attention matrix to global memory.

```
Traditional Attention (Memory-Bound):
  Q, K -> [Global HBM Write: S] -> [Global HBM Read: S] -> Softmax -> [Global HBM Write: P]
FlashAttention (Fused into SRAM):
  Load Blocks of Q, K, V into On-chip SRAM -> Compute Local Gemm -> Online Softmax Update ->
```

The Online Softmax Formula

Traditional softmax requires two passes over the sequence: pass 1 to find the maximum $m = \max(x_i)$, and pass 2 to compute $\sum e^{x_i - m}$. Online softmax updates the running maximum m and normalization sum l incrementally as blocks stream through shared memory:

$$m_{\text{new}} = \max(m_{\text{prev}}, m_{\text{block}})$$

$$l_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} l_{\text{prev}} + e^{m_{\text{block}} - m_{\text{new}}} l_{\text{block}}$$

$$O_{\text{new}} = e^{m_{\text{prev}} - m_{\text{new}}} O_{\text{prev}} + P_{\text{block}} V_{\text{block}}$$

Part IV: The Modern Frontier

FlashAttention from Scratch (FA1 → FA2 → FA3)

FlashAttention from Scratch (FA1 → FA2 → FA3)

FlashAttention Evolution

- **FlashAttention-1 (2022):** Proved that fusing attention and computing online softmax in SRAM yields 2-4x wall-clock speedups while cutting memory footprint from $O(N^2)$ to $O(N)$.
- **FlashAttention-2 (2023):**
  - Swapped the outer and inner loops (looping over blocks of Q in the outer loop, streaming K, V in the inner loop).
  - Reduced non-matmul FLOPs by deferring output scaling until the very end.
  - Introduced sequence parallel work distribution across thread blocks.
- **FlashAttention-3 (2024):**
  - Architected specifically for NVIDIA Hopper (H100) hardware.
  - Utilizes asynchronous Tensor Memory Accelerator (TMA) hardware pipelines.
  - Leverages Hopper WGMMA (Warp Group Matrix Multiply and Accumulate) instructions.
  - Implements FP8 mixed precision with low quantization error.

Beating cuBLAS on an H100

Beating cuBLAS on an H100

NVIDIA's Hopper H100 microarchitecture introduced two revolutionary hardware subsystems:

1. Tensor Memory Accelerator (TMA)

In pre-Hopper architectures, threads in a warp explicitly load data from global memory into registers, then write from registers into shared memory. TMA executes multidimensional tensor transfers (e.g. 2D/3D tiles) directly between global HBM and shared memory **asynchronously in hardware**, completely bypassing the register file and consuming zero SM ALU cycles.

2. Warp Specialization

Instead of every thread in a block performing both memory loading and arithmetic: - **Producer Warps (1-2 warps):** Issue asynchronous TMA copy instructions to fetch upcoming tiles into double-buffered shared memory. - **Consumer Warps (Warp Groups):** Execute continuous WGMMA matrix multiply instructions without ever pausing for memory latency. - Hardware barrier arrive/wait primitives (`cuda::barrier`) synchronize producers and consumers with cycle-level precision.

Triton → CUTLASS → CuTe-DSL

Triton → CUTLASS → CuTe-DSL

Kernel engineering in 2025-2026 has transitioned from hand-written CUDA C++ toward declarative and high-level DSL frameworks.

OpenAI Triton

Triton allows writing high-performance GPU kernels in idiomatic Python. The programmer writes code at the granularity of block tiles rather than individual threads. The Triton compiler handles intra-block scheduling, memory coalescing, and shared memory allocation automatically.

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    tl.store(output_ptr + offsets, x + y, mask=mask)
```

CUTLASS & CuTe

NVIDIA CUTLASS 3.x is structured around **CuTe**: a C++ template library that unifies data layouts, coordinates, and hierarchical tensor tiling through mathematical layout algebra:

$$extLayout = (extShape, extStride)$$

Inference-Serving Kernels

Inference-Serving Kernels

LLM inference splits into two distinct execution regimes with radically different performance characteristics: 1. **Prefill Phase:** The prompt is processed in parallel. High operational intensity, strictly compute-bound. 2. **Decode Phase:** Tokens are generated autoregressively one by one. The model loads billions of weights to compute a single new vector. Strictly memory-bound!

PagedAttention (vLLM)

In standard inference, KV caches must be allocated as contiguous memory buffers. This leads to 60-80% memory waste due to internal and external memory fragmentation. PagedAttention treats KV cache memory like virtual memory pages in an operating system: - KV vectors are stored in non-contiguous 16-token page blocks. - A page table maps logical token positions to physical GPU memory addresses. - Result: Memory waste drops to near 0%, enabling 2-4x higher serving concurrency on the same hardware.

Blackwell & NVFP4

Blackwell & NVFP4

NVIDIA's Blackwell (B200 / GB200) microarchitecture represents the state of the art in frontier hardware (2025-2026).

Blackwell Architectural Highlights

- **`tcgen05` Tensor Cores:** 5th generation Tensor Cores delivering 4x higher FP4 training and inference density compared to Hopper FP8.
- **Tensor Memory (TMEM):** A dedicated ultra-fast on-chip memory structure decoupled from shared memory, specifically optimized for multi-tensor staging and GEMM accumulation.
- **NVFP4 Microscopic Scaling:** 4-bit floating point format with micro-block scaling factors that preserves accuracy across large foundation models while doubling token throughput.

A Blackwell GEMM by Hand in CuTe DSL: TMA, mbarriers, TMEM, and tcgen05

A Blackwell GEMM by Hand in CuTe DSL: TMA, mbarriers, TMEM, and tcgen05

On Blackwell (B200, GB200, `sm_100a`) the tensor core no longer accumulates into registers. A single thread issues `tcgen05.mma`, which reads both operands from shared memory and accumulates into **Tensor Memory (TMEM)**, a 256 KB per-SM scratchpad owned by the tensor core. Operands arrive by **TMA**, an asynchronous copy engine driven by one thread and a descriptor. Everything is coordinated by **mbarriers** whose phase bits you track by hand. To write a fast GEMM, you have to keep this asynchronous machinery fed.

This chapter builds that GEMM from the raw primitives in NVIDIA's CuTe DSL (the Python front end to CUTLASS's CuTe), without the pipeline helper classes, so that every barrier and phase flip is visible. It uses a public step-by-step optimization ladder on a B200, from 164 to about 1400 TFLOP/s on a 4096^3 FP16 problem (97% of cuBLAS), as a map. The ladder shows which steps mattered and why three of them did nothing.

What was and was not run. No Blackwell GPU was available for this chapter. Everything that can run without one did run: the kernel below is cross-compiled for `sm_100a` on a CPU-only machine, and the generated PTX, SASS and register counts are inspected. Shared-memory bank behavior is modeled in Python, and the roofline and budget arithmetic is computed. **The kernel has not been executed**, and every timing in this chapter is a measurement *reported* by the write-up's author, labeled as such. Treat the kernel as a compile-verified teaching listing and check it against a GPU before relying on it. NVIDIA's tutorial `fp16_gemm_0.py` is the tested reference with the same structure.

Toolchain used: Debian 13.7, Python 3.13.5, `nvidia-cutlass-dsl` 4.8.0, and the `ptxas`/`nvdisasm` shipped with Triton 3.8.0 (CUDA 13.3 tools).

---

1. Two regimes: prefill and decode

A GEMM $C_{M\times N} = A_{M\times K} B^{\top}_{N\times K}$ does $2MNK$ FLOPs and moves at least $2(MK + NK + MN)$ bytes in FP16. When N and K are large the ratio is about M FLOP per byte. In LLM inference, M is the number of tokens in the batch: thousands in prefill, tens to hundreds in decode. That single number decides the bottleneck.

```
# file: gemm_math.py
# Back-of-envelope numbers for an FP16 GEMM on a B200-class GPU: roofline per shape,
# shared-memory stage budget, pipeline depth from latency, and wave counts.
# Kernel/cuBLAS times are the ones REPORTED in the write-up that inspired this chapter
```

```
# (B200, fp16, CUDA events); they were not measured here.
PEAK_TFLOPS = 1500        # cuBLAS-achievable fp16 dense, as reported; not the datasheet pea
HBM_TBS = 7.7             # achievable HBM bandwidth, as reported
SMS = 148
SMEM_MAX = 227 * 1024     # max dynamic shared memory per block on sm_100 (CuTe DSL capacity

shapes = {  # name: (M, N, K, reported_us_kernel, reported_us_cublas)
    "square 4096^3":         (4096, 4096, 4096, 98.0, 94.9),
    "prefill attn-out":      (8192, 8192, 8192, 816, 741),
    "prefill MLP up":        (8192, 28672, 8192, 2934, 2407),
    "decode MLP up":         (256, 28672, 8192, 113, 111),
    "decode MLP down":       (256, 8192, 28672, 138, 112),
    "decode fused QKV":      (256, 10240, 8192, 52, 38),
    "decode attn-out":       (256, 8192, 8192, 48, 32),
}
print("1) roofline (bytes = A + B + C once, fp16)")
print(f"{'shape':18} {'FLOP/B':>7} {'bound':>7} {'floor us':>8} {'kernel us':>9} {'TFLOP/s':
      f"{'TB/s':>5} {'cuBLAS us':>9} {'ratio':>5} {'CTAs':>5} {'waves':>5}")
for name, (M, N, K, t, tc) in shapes.items():
    flop, byt = 2 * M * N * K, 2 * (M * K + N * K + M * N)
    t_c, t_m = flop / PEAK_TFLOPS / 1e6, byt / HBM_TBS / 1e6       # microseconds
    ctas = (M // 128) * (N // 256)                                  # 128x256 tile per CTA
    print(f"{name:18} {flop / byt:7.0f} {'compute' if t_c > t_m else 'memory':>7} {max(t_c,
          f"{t:9.1f} {flop / t / 1e6:7.0f} {byt / t / 1e6:5.2f} {tc:9.1f} {tc / t:5.2f} "
          f"{ctas:5d} {ctas / SMS:5.2f}")
print(f"ridge point: {PEAK_TFLOPS / HBM_TBS:.0f} FLOP/B")

print("\n2) shared memory per pipeline stage (tile_k = 64, fp16)")
for label, a_rows, b_rows in (("1-CTA 128x256", 128, 256), ("2-CTA 256x256, per CTA", 128, 1
    stage = (a_rows + b_rows) * 64 * 2
    flop = 2 * 128 * 256 * 64                  # each CTA still accumulates 128x256 per k-st
    print(f"{label:24} {stage // 1024:3d} KiB/stage  {flop / stage:5.1f} FLOP per SMEM byte
          f"max stages in 227 KiB: {SMEM_MAX // stage}")

print("\n3) stages needed to cover load latency (Little's law), 1-CTA 128x256x64 k-step")
t_step = 2 * 128 * 256 * 64 / (PEAK_TFLOPS * 1e12 / SMS) * 1e6     # us per k-step at peak
for lat in (0.5, 1.0, 2.0):
    print(f"  latency {lat:3.1f} us: k-step {t_step:.2f} us -> need >= {1 + lat / t_step:4.1

print("\n4) the reported ladder at 4096^3 (cuBLAS 94.9 us)")
ladder = [("1 K-loop, 1 stage, tile_k 16", 840), ("2 4-stage ring", 448),
          ("3 warp specialization", 449), ("4 tile_k 64 + 128B swizzle", 110),
          ("5 pipelined epilogue", 110), ("6 TMA multicast 2x2", 109.4),
          ("7 2-CTA MMA (256-row)", 107.3), ("8 persistent + 2 TMEM slots", 98.0)]
prev = None
for name, us in ladder:
    tf = 2 * 4096**3 / us / 1e6
```

```
    step = "" if prev is None else f"{prev / us:5.2f}x"
    print(f"  {name:30} {us:6.1f} us {tf:6.0f} TFLOP/s {94.9 / us:5.2f} of cuBLAS {step}")
    prev = us
```

```
python gemm_math.py
```

Output:

```
1) roofline (bytes = A + B + C once, fp16)
shape               FLOP/B   bound floor us kernel us TFLOP/s  TB/s cuBLAS us ratio  CTAs wa
square 4096^3         1365 compute     91.6      98.0    1402  1.03      94.9  0.97   512  3
prefill attn-out      2731 compute    733.0     816.0    1347  0.49     741.0  0.91  2048 13
prefill MLP up        3584 compute   2565.5    2934.0    1312  0.37    2407.0  0.82  7168 48
decode MLP up          246 compute     80.2     113.0    1064  4.32     111.0  0.98   224  1
decode MLP down        246 compute     80.2     138.0     871  3.54     112.0  0.81    64  0
decode fused QKV       242 compute     28.6      52.0     826  3.41      38.0  0.73    80  0
decode attn-out        241 compute     22.9      48.0     716  2.97      32.0  0.67    64  0
ridge point: 195 FLOP/B

2) shared memory per pipeline stage (tile_k = 64, fp16)
1-CTA 128x256             48 KiB/stage   85.3 FLOP per SMEM byte  max stages in 227 KiB: 4
2-CTA 256x256, per CTA    32 KiB/stage  128.0 FLOP per SMEM byte  max stages in 227 KiB: 7

3) stages needed to cover load latency (Little's law), 1-CTA 128x256x64 k-step
  latency 0.5 us: k-step 0.41 us -> need >=  2.2 stages
  latency 1.0 us: k-step 0.41 us -> need >=  3.4 stages
  latency 2.0 us: k-step 0.41 us -> need >=  5.8 stages

4) the reported ladder at 4096^3 (cuBLAS 94.9 us)
  1 K-loop, 1 stage, tile_k 16    840.0 us    164 TFLOP/s  0.11 of cuBLAS
  2 4-stage ring                  448.0 us    307 TFLOP/s  0.21 of cuBLAS  1.88x
  3 warp specialization           449.0 us    306 TFLOP/s  0.21 of cuBLAS  1.00x
  4 tile_k 64 + 128B swizzle      110.0 us   1249 TFLOP/s  0.86 of cuBLAS  4.08x
  5 pipelined epilogue            110.0 us   1249 TFLOP/s  0.86 of cuBLAS  1.00x
  6 TMA multicast 2x2             109.4 us   1256 TFLOP/s  0.87 of cuBLAS  1.01x
  7 2-CTA MMA (256-row)           107.3 us   1281 TFLOP/s  0.88 of cuBLAS  1.02x
  8 persistent + 2 TMEM slots      98.0 us   1402 TFLOP/s  0.97 of cuBLAS  1.09x
```

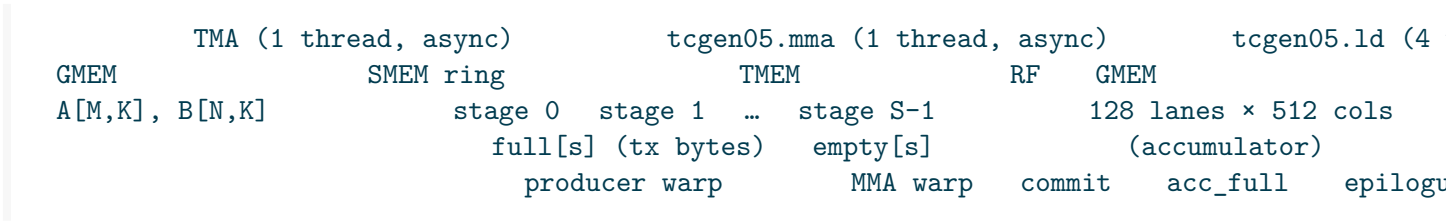
Read part 1 together with the cuBLAS column. The shapes are Llama 3 70B projections: attention output 8192×8192, fused QKV 10240×8192, MLP up 28672×8192, MLP down 8192×28672.

- **Prefill** is deep in compute-bound territory (1300–3600 FLOP/B against a ridge near 195). The game is keeping the tensor pipe busy.
- **Decode at M=256** sits right at the ridge (about 245 FLOP/B), so both bandwidth and compute matter. The decisive column is **waves**. Decode attention-output launches 64 CTAs on 148 SMs, 0.43 waves, so more than half the machine idles no matter how good the inner loop is. The reported 2.97 TB/s against roughly 4.4 TB/s for cuBLAS is what that looks like.

- Decode batches below 256 are worse still for this kernel: its 2-CTA tile is 256 rows tall, so M must be padded to 256.

---

2. The data path and the four primitives



TMA. One thread passes a descriptor (built on the host from the tensor's shape, strides and the SMEM layout) and tile coordinates. The hardware copies the tile and decrements a byte count on an attached mbarrier as data lands. Address arithmetic, bounds and swizzling live in the descriptor, not in registers.

mbarrier. A 64-bit SMEM object holding an expected arrival count, a pending count, a transaction (byte) count and a **phase bit**. When arrivals and bytes both reach zero the phase flips and waiters are released. A wait names the phase it is waiting to see complete. A barrier reused once per lap around an S-stage ring therefore alternates phase 0, 1, 0, 1 by lap, and the code must track that parity. A wrong parity hangs the kernel on the second lap, or worse, passes early and reads stale data.

TMEM. 128 lanes × 512 columns of 32-bit cells per SM. A 128×256 FP32 accumulator uses 256 columns, so two fit. One warp allocates with `tcgen05.alloc`, which takes a power of two of at least 32 columns and writes the TMEM address to SMEM, and the same warp frees it at the end. The epilogue reads TMEM with `tcgen05.ld`, and each warp can reach only its own 32-lane quarter.

tcgen05.mma (UMMA). One thread issues an MMA of up to 128×256×16 (FP16) for one CTA, or 256×256×16 for a CTA pair. It reads SMEM through matrix descriptors and accumulates in TMEM. It is asynchronous. `tcgen05.commit` makes an mbarrier arrive when all previously issued MMAs have completed, which is how the MMA warp tells the producer "this stage is free" and the epilogue "the accumulator is final".

---

3. The kernel

The listing below is a complete warp-specialized GEMM: a 128×256 output tile per CTA, a ring of `STAGES` shared-memory stages each `TILE_K` deep, warp 0 as the TMA producer, warp 1 as the MMA issuer, and all four warps in the epilogue. The three knobs are environment variables so that one file can be compiled in several configurations. The structure follows NVIDIA's CuTe DSL Blackwell tutorial; the pipeline is written out with raw mbarrier calls.

```
# file: bw_gemm.py
# A warp-specialized Blackwell (sm_100a) FP16 GEMM in CuTe DSL, written against
# the raw primitives: TMA loads, mbarriers with explicit phases, tcgen05 MMA into
# Tensor Memory, and a TMEM -> register -> global epilogue.
#
#   C[M, N] = A[M, K] @ B[N, K]^T     (A and B both K-major, FP16 in, FP32 accumulate)
#
# Knobs (module-level, read by the host function at trace time):
#   STAGES    shared-memory ring depth for A/B tiles
#   TILE_K    K depth of one stage (16 = one MMA instruction, 64 = four)
#   ATOM      "sw128" (128-byte XOR swizzle) or "inter" (no swizzle)
import os

import cutlass
import cutlass.cute as cute
import cutlass.utils as utils
from cutlass.cute.nvgpu import cpasync, tcgen05

STAGES = int(os.environ.get("STAGES", 4))
TILE_K = int(os.environ.get("TILE_K", 64))
ATOM = os.environ.get("ATOM", "sw128")

io_dtype, acc_dtype = cutlass.Float16, cutlass.Float32
MMA_INST = (128, 256, 16)          # one tcgen05.mma.cta_group::1 kind::f16 instruction
TILER = (128, 256, TILE_K)         # one CTA's output tile and the K depth of a stage
THREADS = 128                      # 4 warps: 0 = TMA producer, 1 = MMA issuer, all = epilog
TMEM_COLS = 256                    # FP32 128x256 accumulator = 256 TMEM columns


@cute.struct
class Barriers:
    full: cute.struct.MemRange[cutlass.Int64, STAGES]   # stage s holds fresh A/B bytes
    empty: cute.struct.MemRange[cutlass.Int64, STAGES]  # MMAs reading stage s have retired
    acc_full: cutlass.Int64                             # accumulator complete in TMEM
    tmem_slot: cutlass.Int32                            # tcgen05.alloc writes the TMEM addr


@cute.kernel
def gemm_kernel(tiled_mma: cute.TiledMma,
                tma_a: cute.CopyAtom, mA: cute.Tensor,
                tma_b: cute.CopyAtom, mB: cute.Tensor,
                mC: cute.Tensor,
                sA_layout: cute.ComposedLayout, sB_layout: cute.ComposedLayout,
                tx_bytes: cutlass.Constexpr):
    tidx, _, _ = cute.arch.thread_idx()
    warp = cute.arch.make_warp_uniform(cute.arch.warp_idx())
    bm, bn, _ = cute.arch.block_idx()
```

```
# ---- shared memory: barriers, then the swizzled A/B ring (1024 B aligned for SW128)
smem = utils.SmemAllocator()
bars = smem.allocate(Barriers)
sA = smem.allocate_tensor(io_dtype, sA_layout.outer, byte_alignment=1024, swizzle=sA_lay
sB = smem.allocate_tensor(io_dtype, sB_layout.outer, byte_alignment=1024, swizzle=sB_lay
full, empty = bars.full.data_ptr(), bars.empty.data_ptr()
acc_full = bars.acc_full.ptr

if warp == 0:
    with cute.arch.elect_one():
        for s in cutlass.range_constexpr(STAGES):
            cute.arch.mbarrier_init(full + s, 1)    # one arrive (the producer) + tx byt
            cute.arch.mbarrier_init(empty + s, 1)   # one arrive (tcgen05.commit)
            cute.arch.mbarrier_arrive(empty + s)    # pre-arm: every slot starts empty
        cute.arch.mbarrier_init(acc_full, 1)
    cpasync.prefetch_descriptor(tma_a)
    cpasync.prefetch_descriptor(tma_b)
    cute.arch.alloc_tmem(TMEM_COLS, bars.tmem_slot.ptr)   # warp-wide; result lands in S
cute.arch.mbarrier_init_fence()
cute.arch.sync_threads()
tmem_ptr = cute.arch.retrieve_tmem_ptr(acc_dtype, alignment=16,
                                       ptr_to_buffer_holding_addr=bars.tmem_slot.ptr)

# ---- partition global tiles for this CTA and build MMA fragments over SMEM / TMEM
coord = (bm, bn, None)
gA = cute.local_tile(mA, TILER, coord, proj=(1, None, 1))     # (128, TILE_K, k_tiles)
gB = cute.local_tile(mB, TILER, coord, proj=(None, 1, 1))     # (256, TILE_K, k_tiles)
gC = cute.local_tile(mC, TILER, coord, proj=(1, 1, None))     # (128, 256)
thr = tiled_mma.get_slice(0)
tCgA, tCgB, tCgC = thr.partition_A(gA), thr.partition_B(gB), thr.partition_C(gC)
tCrA, tCrB = tiled_mma.make_fragment_A(sA), tiled_mma.make_fragment_B(sB)
acc = tiled_mma.make_fragment_C(tiled_mma.partition_shape_C(TILER[:2]))
acc = cute.make_tensor(tmem_ptr, acc.layout)                  # the accumulator lives in

tAsA, tAgA = cpasync.tma_partition(tma_a, 0, cute.make_layout(1),
                                   cute.group_modes(sA, 0, 3), cute.group_modes(tCgA, 0,
tBsB, tBgB = cpasync.tma_partition(tma_b, 0, cute.make_layout(1),
                                   cute.group_modes(sB, 0, 3), cute.group_modes(tCgB, 0,
k_tiles = cute.size(gA, mode=[2])

if warp == 0:
    # ---- producer: wait for an empty slot, arm its full barrier with the byte count, i
    stage, phase = cutlass.Int32(0), cutlass.Int32(0)
    for k in cutlass.range(k_tiles, unroll=1):
        cute.arch.mbarrier_wait(empty + stage, phase)
        with cute.arch.elect_one():
```

```
            cute.arch.mbarrier_arrive_and_expect_tx(full + stage, tx_bytes)
        cute.copy(tma_a, tAgA[(None, k)], tAsA[(None, stage)], tma_bar_ptr=full + stage)
        cute.copy(tma_b, tBgB[(None, k)], tBsB[(None, stage)], tma_bar_ptr=full + stage)
        stage += 1
        if stage == STAGES:                  # wrapped the ring: the next wait is one ph
            stage = cutlass.Int32(0)
            phase ^= 1
elif warp == 1:
    # ---- MMA issuer: wait for a full slot, run TILE_K/16 MMAs, commit the slot back as
    stage, phase = cutlass.Int32(0), cutlass.Int32(0)
    tiled_mma.set(tcgen05.Field.ACCUMULATE, False)           # first MMA overwrites TME
    for k in cutlass.range(k_tiles, unroll=1):
        cute.arch.mbarrier_wait(full + stage, phase)
        for kb in cutlass.range_constexpr(cute.size(tCrA, mode=[2])):
            cute.gemm(tiled_mma, acc, tCrA[(None, None, kb, stage)],
                      tCrB[(None, None, kb, stage)], acc)
            tiled_mma.set(tcgen05.Field.ACCUMULATE, True)    # every later MMA adds
        with cute.arch.elect_one():
            tcgen05.commit(empty + stage)    # arrives when these MMAs have read the slo
        stage += 1
        if stage == STAGES:
            stage = cutlass.Int32(0)
            phase ^= 1
    with cute.arch.elect_one():
        tcgen05.commit(acc_full)             # arrives when every MMA has finished

# ---- epilogue (all 128 threads): TMEM -> registers (32 lanes per warp) -> FP16 -> glob
cute.arch.mbarrier_wait(acc_full, 0)
epi = ((cute.size(acc, mode=[0, 0]), cute.size(acc, mode=[0, 1]) // 4),)
acc_epi, gC_epi = cute.zipped_divide(acc, epi), cute.zipped_divide(tCgC, epi)
t2r = tcgen05.make_tmem_copy(cute.make_copy_atom(tcgen05.Ld32x32bOp(tcgen05.Repetition.x
                                                 acc_dtype), acc_epi[None, 0])
t2r_thr = t2r.get_slice(tidx)
src, dst = t2r_thr.partition_S(acc_epi), t2r_thr.partition_D(gC_epi)
r_acc = cute.make_rmem_tensor(dst[None, None, 0].shape, acc_dtype)
r_out = cute.make_rmem_tensor(dst[None, None, 0].shape, io_dtype)
for i in cutlass.range_constexpr(cute.size(src, mode=[2])):
    cute.copy(t2r, src[None, None, i], r_acc)
    r_out.store(r_acc.load().to(io_dtype))
    cute.autovec_copy(r_out, dst[None, None, i])

cute.arch.sync_threads()
if warp == 0:
    cute.arch.relinquish_tmem_alloc_permit()
    cute.arch.dealloc_tmem(tmem_ptr, TMEM_COLS)
```

```
def smem_atom():
    kind = {"sw128": tcgen05.SmemLayoutAtomKind.K_SW128,
            "inter": tcgen05.SmemLayoutAtomKind.K_INTER}[ATOM]
    return tcgen05.make_smem_layout_atom(kind, io_dtype)


@cute.jit
def gemm(a: cute.Tensor, b: cute.Tensor, c: cute.Tensor):
    op = tcgen05.MmaF16BF16Op(io_dtype, acc_dtype, MMA_INST, tcgen05.CtaGroup.ONE,
                              tcgen05.OperandSource.SMEM,
                              tcgen05.OperandMajorMode.K, tcgen05.OperandMajorMode.K)
    tiled_mma = cute.make_tiled_mma(op)
    a_shape = cute.append(tiled_mma.partition_shape_A(cute.dice(TILER, (1, None, 1))), STAGE
    b_shape = cute.append(tiled_mma.partition_shape_B(cute.dice(TILER, (None, 1, 1))), STAGE
    sA_layout = tcgen05.tile_to_mma_shape(smem_atom(), a_shape, order=(1, 2, 3))
    sB_layout = tcgen05.tile_to_mma_shape(smem_atom(), b_shape, order=(1, 2, 3))
    one_a, one_b = cute.select(sA_layout, mode=[0, 1, 2]), cute.select(sB_layout, mode=[0, 1
    tma_op = cpasync.CopyBulkTensorTileG2SOp(tcgen05.CtaGroup.ONE)
    tma_a, tA = cute.nvgpu.make_tiled_tma_atom_A(tma_op, a, one_a, TILER, tiled_mma)
    tma_b, tB = cute.nvgpu.make_tiled_tma_atom_B(tma_op, b, one_b, TILER, tiled_mma)
    tx_bytes = cute.size_in_bytes(io_dtype, one_a) + cute.size_in_bytes(io_dtype, one_b)
    gemm_kernel(tiled_mma, tma_a, tA, tma_b, tB, c, sA_layout, sB_layout, tx_bytes).launch(
        grid=cute.ceil_div((*c.layout.shape, 1), TILER[:2]), block=(THREADS, 1, 1))
```

The parts worth slowing down for:

1. **Pre-armed empty barriers.** Each `empty[s]` is initialized with count 1 and immediately arrived on once, which completes its phase 0. On the first lap the producer waits for phase 0 and passes straight through, so no separate prologue loop is needed. On lap 2 it waits for phase 1, which completes only when `tcgen05.commit(empty + s)` fires after the MMAs that read stage `s`.
2. **`arrive_and_expect_tx` before the copies.** The producer arrives on `full[s]` and announces `tx_bytes` (A tile + B tile) in one call. The two TMA copies then count those bytes down. The consumer's wait on `full[s]` cannot succeed until both the arrival and every byte have landed. If the byte count is wrong in either direction, the kernel hangs or races.
3. **Phase flips on wrap, not per iteration.** `stage` and `phase` advance together, and `phase ^= 1` happens only when the ring wraps. Producer and consumer keep identical counters, which is what makes the protocol easy to reason about.
4. **ACCUMULATE off for the first MMA only.** TMEM is not zeroed. The first `cute.gemm` must overwrite and every later one must add, and the DSL does not toggle this for you.
5. **The epilogue waits on one barrier.** `acc_full` completes once, at phase 0, after the final commit. All 128 threads then pull 32 lanes per warp out of TMEM in four 64-column chunks, convert to FP16 and store. A production kernel stages through SMEM and uses a TMA store. This one stays short.
6. **The swizzle rides on the pointer.** `allocate_tensor(..., swizzle=layout.inner)` attaches the XOR function to the SMEM pointer and keeps the affine part (with the stage mode) as the layout, and the base is 1024-byte aligned. Passing a composed layout where the MMA fragment builder expects an affine one is rejected.

4. Compile for `sm_100a` without a GPU

CuTe DSL can JIT-compile against **fake tensors** (shape, dtype, alignment, no memory) for a named architecture and dump the PTX and cubin. That catches a large class of errors (layout mismatches, illegal swizzle/tile combinations, type errors) and lets you read what the kernel will do.

```
# file: compile_check.py
# Cross-compile bw_gemm.py for sm_100a on a machine with no GPU and report what the
# compiler produced. Run once per configuration (the knobs are environment variables).
import os
import pathlib
import re
import sys

import cutlass
import cutlass.cute as cute
from cutlass.cute.runtime import make_fake_compact_tensor

import bw_gemm as g

M = N = K = 4096
out = pathlib.Path("build") / f"s{g.STAGES}_k{g.TILE_K}_{g.ATOM}"
out.mkdir(parents=True, exist_ok=True)
fake = lambda r, c: make_fake_compact_tensor(cutlass.Float16, (r, c), stride_order=(1, 0),
                                             assumed_align=32)
try:
    cute.compile(g.gemm, fake(M, K), fake(N, K), fake(M, N),
                 options=f"--gpu-arch sm_100a --keep-ptx --keep-cubin --dump-dir {out}")
except Exception as e:  # noqa: BLE001 - report the compiler's verdict, whatever it is
    lines = [l.strip() for l in str(e).splitlines() if "error" in l] or [str(e).strip()]
    print(f"STAGES={g.STAGES} TILE_K={g.TILE_K:<2} ATOM={g.ATOM:<5}  REJECTED:\n    {lines[0
    sys.exit(0)

ptx = next(out.glob("*.ptx")).read_text()
count = lambda pat: len(re.findall(pat, ptx))
stage_bytes = (128 + 256) * g.TILE_K * 2
print(f"STAGES={g.STAGES} TILE_K={g.TILE_K:<2} ATOM={g.ATOM:<5}  ok  "
      f"smem/stage={stage_bytes // 1024} KiB ring={g.STAGES * stage_bytes // 1024} KiB  "
      f"tcgen05.mma={count(r'tcgen05\.mma')} tma.load={count(r'cp\.async\.bulk\.tensor')} "
      f"try_wait={count(r'mbarrier\.try_wait')} tcgen05.ld={count(r'tcgen05\.ld')}")
```

```
pip install nvidia-cutlass-dsl==4.8.0
for cfg in "1 16 inter" "1 16 sw128" "4 16 inter" "4 64 inter" "4 64 sw128" "6 64 sw128"; do
  set -- $cfg; STAGES=$1 TILE_K=$2 ATOM=$3 python compile_check.py
done
```

Output:

```
STAGES=1 TILE_K=16 ATOM=inter  ok  smem/stage=12 KiB ring=12 KiB  tcgen05.mma=1 tma.load=4 t
STAGES=1 TILE_K=16 ATOM=sw128  REJECTED:
    error: unknown: expected top-level shape equivalence between the SMEM layout and the CTA
STAGES=4 TILE_K=16 ATOM=inter  ok  smem/stage=12 KiB ring=48 KiB  tcgen05.mma=1 tma.load=4 t
STAGES=4 TILE_K=64 ATOM=inter  ok  smem/stage=48 KiB ring=192 KiB  tcgen05.mma=4 tma.load=16
STAGES=4 TILE_K=64 ATOM=sw128  ok  smem/stage=48 KiB ring=192 KiB  tcgen05.mma=4 tma.load=2
STAGES=6 TILE_K=64 ATOM=sw128  ok  smem/stage=48 KiB ring=288 KiB  tcgen05.mma=4 tma.load=2
```

What the compiler is telling you:

- **The 128-byte swizzle cannot be used with a 16-deep tile.** The SW128 atom is 8 rows × 64 FP16 (128 bytes) in K. A 16-deep stage cannot hold one, and layout algebra rejects the combination while building the TMA atom. This matches the write-up's account that the swizzle "needs `tile_k >= 64`" and that tile_k 64 alone changed nothing. Each half of the winning change looks useless when tested alone.
- **Without swizzling, TMA moves 16-byte-wide boxes.** The non-swizzled ("interleaved") canonical layout is built from 8-row × 16-byte core matrices, so each operand's 64-deep slice becomes eight narrow TMA boxes: 16 `cp.async.bulk.tensor` per stage against 2 with SW128.
- **MMA count per stage = TILE_K / 16**: 1 at depth 16, 4 at depth 64. That is the work available to overlap each wait.
- **Six 48 KiB stages compile fine and cannot launch.** 288 KiB exceeds the 227 KiB per-block limit on `sm_100`. Dynamic shared memory is sized at launch, so nothing at compile time objects. CuTe DSL inserts a launch-time check that prints an error, and the driver refuses the launch.

Then look at register use and the SASS:

```
NV=$(python -c "import os, triton; print(os.path.dirname(triton.__file__) + '/backends/nvidi
B=build/s4_k64_sw128
$NV/ptxas-blackwell -arch=sm_100a -v $B/*.ptx -o /tmp/k.cubin 2>&1 | grep -E "registers|spil
$NV/nvdisasm -c $B/*.cubin \
  | grep -oE '\b(UTCHMMA|UTMALDG|UTCBAR|LDTM\.x64|SYNCS\.[A-Z]+(\.[A-Z0-9]+)*)' | sort | uni
```

Output:

```
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas-blackwell info    : Used 78 registers, used 1 barriers
      4 LDTM.x64
      1 SYNCS.ARRIVE.TRANS64
      4 SYNCS.ARRIVE.TRANS64.ART0
      9 SYNCS.EXCH.64
```

```
      3 SYNCS.PHASECHK.TRANS64
      6 SYNCS.PHASECHK.TRANS64.TRYWAIT
      2 UTCBAR
      4 UTCHMMA
      2 UTMALDG
```

78 registers and no spills, because the accumulator is not in registers. The four `UTCHMMA` are the four `tcgen05.mma` per stage, `UTMALDG` are the two TMA loads, `UTCBAR` are the two commits (stage-empty and accumulator-full), `LDTM.x64` are the four TMEM reads, and the `SYNCS.*` family is mbarrier init, arrive and try-wait. SASS opcode names are not documented by NVIDIA. This mapping is inferred by matching counts against the PTX.

---

5. The ladder: what moved the number (reported)

Part 4 of `gemm_math.py` converts the write-up's reported 4096^3 timings into TFLOP/s. Condensed, with the write-up's Nsight Compute observations:

Step	Change	Reported µs	Why it did or did not help
1	K-loop, one stage, tile_k 16	840	Strict load→wait→MMA→wait; tensor pipe 7.5% busy
2	4-stage ring	448	Loads overlap MMAs; but L1/TEX hits 98.9% with DRAM at 2.3%
3	Warp specialization	449	**Null.** The single warp was never stuck *issuing* (all ops are async), it was stuck *waiting*; L1/TEX was the limit
4	tile_k 64 + 128B swizzle	110	**4.1x.** Load-path bottleneck removed; tensor pipe 13.7% → 63.7%
5	Pipelined epilogue	~110	**Null.** `ptxas` had already overlapped the unrolled chunks; the epilogue was 13 µs of 109

Step	Change	Reported µs	Why it did or did not help
6	TMA multicast (2×2)	109.4	**Null** (6% worse at 8192^3). L2 traffic was not the limit once compute-bound
7	2-CTA MMA (256-row tile)	107.3	Small on square; larger on decode (54→48 µs attn-out): each pair fetches the weight tile once
8	Persistent + two TMEM accumulator slots	98.0	Epilogue of tile i overlaps mainloop of tile *i+1*; tensor pipe 61.7% → 74.5%

Three things generalize beyond this kernel:

- **One change did most of the work, and it was a data-layout change.** No amount of pipelining or specialization helps if the load path is serialized.
- **The nulls were predictable from the profile.** At step 3, L1/TEX was at 98% and specialization does not reduce L1/TEX traffic. At step 6 the kernel was compute-bound, so cutting L2 reads could not help. Before each step, ask which counter the change will move and whether that counter is the binding one.
- **"L1TEX scoreboard" stalls in these kernels are mostly mbarrier waits.** `mbarrier.try_wait` polls an SMEM word through the L1TEX pipe. In a healthy warp-specialized kernel every role waits for the role before it, so this stall *grows* as the kernel gets faster. Watch tensor-pipe utilization instead.

---

6. The swizzle

Shared memory has 32 banks of 4 bytes, so a 128-byte row covers each bank exactly once. If a tile's row pitch is 128 bytes, the same column of every row falls in the same banks. A 128B swizzle (`Swizzle<3,4,3>` on byte addresses) XORs the 16-byte chunk index (address bits 4–6) with the row index mod 8 (bits 7–9). Rows keep their bytes, but chunks are permuted within each row so that a column hits eight different bank groups.

```
# file: swizzle_model.py
# Shared memory: 32 banks x 4 bytes. A 16-byte access covers 4 consecutive banks, so
# 8 such accesses can be served together if they hit 8 different 4-bank groups.
# Count how many serialized passes ("wavefronts") 8 rows x 16 bytes at the same logical
# column need, for plain and 128B-swizzled layouts, at two row pitches.
def sw128(addr):
```

```
    """CuTe Swizzle<3,4,3> on byte addresses: XOR bits [7,10) (row mod 8) into bits [4,7)
    (16-byte chunk within a 128-byte row)."""
    return addr ^ (((addr >> 7) & 0b111) << 4)


def passes(addrs):
    words_per_bank = {}
    for a in addrs:
        for w in range(4):                          # a 16-byte chunk = 4 consecutive 4-byte wo
            word = a // 4 + w
            words_per_bank.setdefault(word % 32, set()).add(word)
    return max(len(s) for s in words_per_bank.values())


print(f"{'tile_k (fp16)':>13} {'row pitch':>9} {'layout':>8}  passes for 8 rows x 16 B, per
for tile_k in (16, 64):
    pitch = tile_k * 2
    for name, f in (("plain", lambda a: a), ("sw128", sw128)):
        if name == "sw128" and pitch < 128:
            print(f"{tile_k:13d} {pitch:7d} B {name:>8}  n/a: needs 128-byte rows (tile_k >=
            continue
        per_col = [passes([f(r * pitch + c * 16) for r in range(8)]) for c in range(pitch //
        print(f"{tile_k:13d} {pitch:7d} B {name:>8}  {per_col}")

print("\nsw128 permutation of the eight 16-byte chunks, rows 0..7:")
for r in range(8):
    print(f"  row {r}: " + " ".join(str((sw128(r * 128 + c * 16) - r * 128) // 16) for c in
```

```
python swizzle_model.py
```

Output:

```
tile_k (fp16) row pitch   layout  passes for 8 rows x 16 B, per column chunk
           16      32 B    plain  [2, 2]
           16      32 B    sw128  n/a: needs 128-byte rows (tile_k >= 64)
           64     128 B    plain  [8, 8, 8, 8, 8, 8, 8, 8]
           64     128 B    sw128  [1, 1, 1, 1, 1, 1, 1, 1]

sw128 permutation of the eight 16-byte chunks, rows 0..7:
  row 0: 0 1 2 3 4 5 6 7
  row 1: 1 0 3 2 5 4 7 6
  row 2: 2 3 0 1 6 7 4 5
  row 3: 3 2 1 0 7 6 5 4
  row 4: 4 5 6 7 0 1 2 3
  row 5: 5 4 7 6 1 0 3 2
  row 6: 6 7 4 5 2 3 0 1
```

```
row 7: 7 6 5 4 3 2 1 0
```

With 128-byte rows and no swizzle, reading or writing one 16-byte column across 8 rows takes 8 serialized passes. With SW128 it takes 1. The write-up isolated the load path (no MMA) and reported 3.85 TB/s unswizzled against 18.5 TB/s swizzled, a 4.8x difference on TMA writes into SMEM. The exact access pattern TMA uses internally is not documented, so treat the model as the reason the canonical layouts exist, not a prediction of the exact ratio. Section 4 adds a second effect: the non-swizzled canonical layout forces TMA into eight 16-byte-wide boxes per operand per stage, many small requests where one large one would do. Both effects point the same way.

Rules that fall out of this: the K-major SW128 atom needs `tile_k * sizeof(T)` to be a multiple of 128 bytes (64 FP16 or BF16, 128 FP8). The SMEM base must be 1024-byte aligned (8 rows × 128 B, one full swizzle period). The TMA descriptor and the UMMA descriptor must agree on the swizzle mode. CuTe derives both from the same layout, which is the main reason to let it.

---

7. Stages: the budget and the regime

Part 2 of `gemm_math.py` gives the hard limit. A 1-CTA 128×256×64 stage is 48 KiB, so at most 4 stages fit in 227 KiB. A 2-CTA tile halves each CTA's B operand to 32 KiB, so up to 7 fit. NVIDIA's tutorial `fp16_gemm_1.py` makes the same calculation in its docstring. Part 3 is Little's law. At peak, one 64-deep k-step takes about 0.41 µs per SM, so the ring must hold $1 + \text{latency}/0.41\,\mu s$ stages to hide load latency. That is 3–6 stages for 1–2 µs of loaded-system latency.

The write-up's stage sweeps show why this has to be re-measured. At step 3 (16-deep stages), 4, 8 and 12 stages were flat (451.7, 454.6, 454.1 µs reported), because the kernel was not latency-bound. On the step-7 kernel (32 KiB stages) the same sweep went from 166.9 µs at 2 stages to a best of 105.9 µs at 5, and the shipped value of 6 was within noise. A sweep result holds only for the bottleneck that was active when you ran it.

---

8. Two CTAs, then persistence

The last two rungs are where hand-written kernels usually stop matching the tutorials, and where the official examples are the best reference. Cross-compiling NVIDIA's tutorial ladder shows the features appearing:

```
# file: official_check.py
# Cross-compile NVIDIA's CuTe DSL tutorial GEMMs (CUTLASS examples, BSD-3) for sm_100a
# and count the instructions that mark each feature: 1-CTA vs 2-CTA MMA, multicast TMA,
# cluster barriers, and TMA stores.
import inspect
import pathlib
```

```
import re
import sys

import cutlass
import cutlass.cute as cute
from cutlass.cute.runtime import make_fake_compact_tensor

TUT = pathlib.Path(sys.argv[1])  # .../examples/python/CuTeDSL/cute/blackwell/tutorial/tutor
sys.path.insert(0, str(TUT))
fake = lambda r, c: make_fake_compact_tensor(cutlass.Float16, (r, c), stride_order=(1, 0),
                                             assumed_align=32)
pats = {"mma cta_group::1": r"tcgen05\.mma\.cta_group::1", "mma cta_group::2": r"tcgen05\.mm
        "TMA multicast": r"cp\.async\.bulk\.tensor[^;]*multicast", "cluster barrier": r"barr
        "tcgen05.alloc": r"tcgen05\.alloc", "TMA store": r"cp\.async\.bulk\.tensor\.\dd\.glo
print(f"{'tutorial':12} " + " ".join(f"{k:>16}" for k in pats))
for name in sys.argv[2:]:
    mod = __import__(name)
    out = pathlib.Path("build_official") / name
    out.mkdir(parents=True, exist_ok=True)
    M = N = K = 8192
    extra = (74,) if "max_active_clusters" in inspect.signature(mod.host_function).parameter
    cute.compile(mod.host_function, fake(M, K), fake(N, K), fake(M, N), *extra,   # 74 = 148
                 options=f"--gpu-arch sm_100a --keep-ptx --dump-dir {out}")
    ptx = next(out.glob("*.ptx")).read_text()
    print(f"{name:12} " + " ".join(f"{len(re.findall(p, ptx)):16d}" for p in pats.values()))
```

```
git clone --depth 1 https://github.com/NVIDIA/cutlass
python official_check.py cutlass/examples/python/CuTeDSL/cute/blackwell/tutorial/tutorial_ge
    fp16_gemm_0 fp16_gemm_1 fp16_gemm_2 fp16_gemm_3
```

Output (CUTLASS at commit `0b55a2f`):

```
tutorial     mma cta_group::1 mma cta_group::2    TMA multicast  cluster barrier    tcgen05.
fp16_gemm_0                 8                0                0                0
fp16_gemm_1                 0                8               14                4
fp16_gemm_2                 0                8                8                4
fp16_gemm_3                 0                8                8                6
```

- **2-CTA MMA (`fp16_gemm_1`).** `tcgen05.mma.cta_group::2` replaces `cta_group::1`. A cluster of two CTAs on neighboring SMs issues one 256-row MMA; each CTA supplies half of A, half of B and holds half the accumulator. Cluster barriers appear because the pair has to agree that both are initialized. The pair is **not symmetric**: the leader CTA alone waits on the full barrier, issues the MMA and commits, and `tcgen05.commit` signals only the CTAs named in its multicast mask. The write-up's hang at this step came from a commit that never reached the follower's empty barrier. It completed at K=192, where no stage is recycled, and hung at K=1024. Shrinking K until the hang disappears is a fast way to tell "stage reuse is broken" apart from "something else is broken".

- **Warp specialization + TMA store (`fp16_gemm_2`).** Same MMA, epilogue through SMEM and a TMA store.
- **Persistent (`fp16_gemm_3`).** Launch one CTA (pair) per SM and loop over tiles with a tile scheduler. On its own that only removes launch and tail overhead. The real gain comes from **two TMEM accumulator slots**, $2 \times 256 =$ all 512 columns. The MMA warp fills slot `i % 2` while the epilogue drains the other, with an `acc_full[s]`/`acc_empty[s]` barrier pair per slot mirroring the A/B ring. The epilogue should release `acc_empty[s]` as soon as its `tcgen05.ld`s complete, before converting and storing, so that the MMA warp gets the slot back early. Limits: N 256 per slot, the overlap only helps if the epilogue is shorter than the mainloop, and the first mainloop and last epilogue are never hidden.

The later tutorials add dynamic cluster-launch-control scheduling (`fp16_gemm_3_1`), preferred clusters (`_4`), TMA prefetch (`_5`) and programmatic dependent launch (`_6`). The write-up's reported threadblock-swizzle experiment (grouping 8 tile rows so neighboring CTAs share B tiles in L2) reached parity with cuBLAS on a 4096×4096×16384 problem and did nothing at 4096^3, where 512 CTAs is too few waves for ordering to matter.

---

9. Why decode loses, and what to do about it

The final reported comparison (part 1 of `gemm_math.py`) is 0.97x of cuBLAS on the square problem, 0.82–0.91x on prefill, and 0.67–0.98x on decode. The decode pattern follows the waves column:

- **MLP up (224 CTAs, 1.5 waves): 0.98x.** Enough tiles to fill the machine. The kernel reads 478 MB from HBM, about the size of the 470 MB weight, so each weight is read once, at about 61% of peak DRAM bandwidth (reported).
- **Attention output and MLP down (64 CTAs, 0.43 waves): 0.67x and 0.81x.** Fewer tiles than SMs. Persistence cannot help with less than one tile per CTA: the reported attention-output time was 47 µs before and after step 8.

The standard fixes:

Fix	What it does	Cost
Smaller N tiles (128 or 64)	More CTAs: 128 for N=8192 with 128-wide tiles	Lower arithmetic intensity per CTA; still ridge at M=256
Split-K	Each CTA reduces a K slice; a second pass or atomics combine	Extra FP32 partial traffic; nondeterminism with atomics
Stream-K	Persistent CTAs take equal shares of the *total* MAC-loop iterations, crossing tile boundaries	Fix-up logic for partial tiles; best general answer for odd shapes
1-CTA MMA for small M	Avoid padding M up to 256	Gives up the 2-CTA weight-tile sharing

cuBLAS picks among these per shape with heuristics. A hand-written kernel that only has one tiling will lose somewhere, which is why production libraries ship families of kernels and why the shape list should come from your model, not from the square.

---

10. Driving an AI assistant through a ladder like this

The source write-up's experiments were run by an AI coding agent under the author's direction. That works well when the human enforces the method:

1. **One change per step, timed on fixed shapes, with a correctness check in every run.** The write-up's threadblock-swizzle sweep produced a fake 1.13x speedup at `swizzle_m=32`, and its maximum error was 772: CTAs had walked off the end of M and skipped work. Any speedup without a passing comparison against `torch.matmul` is a bug report.
2. **Size the prize before optimizing a stage.** "The epilogue is at most 13 µs of 109" would have cancelled step 5 in advance. Ask the assistant to time each stage in isolation first: MMA-only (no loads), loads-only, epilogue-only. The write-up's MMA-only ablation ran at 96.4 µs (cuBLAS speed) while loads-only took 417 µs, and that pair of numbers identified the bottleneck.
3. **Make the assistant name the counter its change should move**, and reject changes aimed at a counter that is not saturated.
4. **Re-sweep knobs when the bottleneck moves.** Stage count, tile shape and cluster shape all interact with it.
5. **Have it compile for the target first.** The cross-compile in Section 4 costs seconds and catches illegal layouts, shared-memory overflows and register spills before you pay for GPU time.

A prompt skeleton that encodes this:

```
You are optimizing a CuTe DSL GEMM for sm_100a. Current kernel: <file>. Shapes: <list>.
Rules: change ONE thing per step. Before editing, state which Nsight Compute metric you
expect to move and why it is the binding constraint now. After editing: cross-compile,
report registers/spills and SMEM bytes, then run the correctness check against torch
(state the max abs error and the tolerance; the reference write-up saw 0 for K<=8192 and 0.5
Report "no change" for differences under 2%. Never report a speedup without the error.
```

---

11. Verification and benchmark plan

On a CPU (done here)

- Cross-compile every configuration you intend to run. Reject anything over 227 KiB of SMEM, and check `ptxas -v` for spills.

- Count PTX/SASS instructions per stage (MMAs, TMA loads, commits) against what the design says.

On a Blackwell GPU (not done here)

- **Correctness:** random FP16 inputs, compare against `torch.matmul` with FP32 accumulation, for K {64, 192, 1024, 8192}. K=192 never recycles a 4-stage ring; K=1024 does, so it exposes phase bugs. Also test M and N not divisible by the tile and expect either a clean rejection or correct predication.
- **Timing:** CUDA events around 20–50 iterations after warm-up, with `torch.matmul` on the same tensors in the same process. The write-up measured about 1.7 µs run-to-run spread, so treat differences under 2% as noise.
- **Profiling:** Nsight Compute `sm__pipe_tensor_op_*` utilization, L1/TEX and DRAM throughput, waves per SM, and the stall breakdown. On cloud containers that cannot lock clocks, use `--clock-control none` and compare percentages across kernel versions, not absolutes.

Pitfalls

Pitfall	Symptom	Fix
Phase flipped every iteration on a ring barrier	Hang on lap 2, or stale data	Flip parity only when the stage index wraps
Byte count in `expect_tx` bytes TMA delivers	Hang (too many) or race (too few)	Compute from the same SMEM layouts the TMA atoms use
Forgetting to pre-arm empty barriers	Producer deadlocks on lap 1	Arrive once at init, or start the producer at phase 1
ACCUMULATE left on for the first MMA	Garbage from uninitialized TMEM	Clear for the first MMA of each tile
SW128 with a 16-deep FP16 tile	Layout rejected at compile time	tile_k 64 (FP16/BF16)
Unaligned SMEM base for SW128	Corrupt operands	1024-byte alignment
Stage count chosen by “more is better”	Launch failure above 227 KiB; no gain when not latency-bound	Budget per stage, then sweep in the current regime
2-CTA commit without the peer in the mask	Hang once stages recycle	Name both CTAs in the commit mask
Benchmarking only square shapes	Decode regressions go unnoticed	Use your model’s (M, N, K) list, including small M
Speedup without a correctness check	Fake wins from skipped tiles	Max-error check in every timing run

Checkpoint

- Explain why one thread can issue a Blackwell MMA, and where the accumulator lives.
- Write the producer and consumer loops of an S-stage TMA→UMMA ring, including the phase arithmetic.
- State the two conditions that complete an mbarrier phase when TMA is involved.
- Explain why warp specialization did not help at step 3, and why multicast did not help at step 6.
- Show with bank arithmetic why 128-byte rows conflict, and how `Swizzle<3,4,3>` removes the conflict.
- Compute the SMEM per stage and the maximum stage count for 1-CTA and 2-CTA 64-deep tiles.
- Explain why decode attention-output stays at 0.67x of cuBLAS even after persistence, and name two fixes.

Try this

1. Add a fourth knob for the N tile (128 or 256) to `bw_gemm.py`, cross-compile, and record TMEM columns, SMEM per stage and maximum stages. Recompute the waves column for the decode shapes.
2. Change the epilogue to stage through SMEM and use a TMA store (follow `fp16_gemm_2.py`). Compare SASS: a `UTMASTG` (TMA store) should appear, as it does in the cross-compiled `fp16_gemm_2`, and the direct global stores should disappear.
3. Implement the two-slot TMEM accumulator: allocate 512 columns, add `acc_full[2]`/`acc_empty[2]`, and loop over two tiles per CTA. Cross-compile and confirm `tcgen05.alloc` asks for 512 columns.
4. Extend `swizzle_model.py` to 32-byte and 64-byte swizzles (`Swizzle<1,4,3>`, `Swizzle<2,4,3>`) at 32- and 64-byte row pitches, and find which pitch each one makes conflict-free.
5. On a B200: run `bw_gemm.py` against `torch.matmul` for K $\in$ {64, 192, 1024, 4096}. Then time `STAGES` $\in$ `{2, 3, 4}` $\times$ `ATOM` $\in$ `{inter, sw128}` at 4096^3 and compare with the reported ladder.

Summary

A Blackwell GEMM is a small asynchronous system. TMA moves tiles into a shared-memory ring, mbarriers with explicit phases hand stages between a producer warp and a single-thread MMA issuer, `tcgen05.mma` accumulates in TMEM, and the epilogue drains TMEM through registers.

The kernel in this chapter writes all of that out in CuTe DSL and is verified by cross-compiling for `sm_100a`. Its PTX and SASS contain exactly the MMAs, TMA loads, commits and TMEM loads the design calls for, with no spills. The ladder's lessons are about method more than primitives. One layout change (tile_k 64 with a 128-byte swizzle) delivered a 4x gain, three plausible optimizations were null because they targeted counters that were not binding, and the final 2-CTA and persistent double-buffered steps closed the gap to cuBLAS on large shapes. Decode shapes are decided by waves, not by the inner loop, and need split-K, stream-K or smaller tiles. Compile for the target first, check correctness in every timing run, and re-measure whenever the bottleneck moves.

Sources (for further reading)

- NVIDIA CUTLASS, CuTe DSL documentation (`media/docs/pythonDSL/`) and Blackwell tutorial `examples/python/CuTeDSL/cute/blackwell/tutorial/tutorial_gemm/fp16_gemm_0.py` … `fp16_gemm_6.py` (BSD-3-Clause), https://github.com/NVIDIA/cutlass. Persistent kernels: `examples/python/CuTeDSL/cute/blackwell/kernel/dense_gemm/dense_gemm_persistent.py`.
- NVIDIA PTX ISA: `tcgen05` family (`tcgen05.alloc`, `tcgen05.mma`, `tcgen05.commit`, `tcgen05.ld`), `cp.async.bulk.tensor`, `mbarrier` (expect-tx, phase completion), tensor-map swizzle modes.
- NVIDIA CUDA C++ Programming Guide: Tensor Memory Accelerator, asynchronous barriers, thread block clusters, shared memory banks.
- CuTe DSL capacity map (`cutlass_dsl/cutlass.py`, `SMEM_CAPACITY_MAP`): 227 KiB per block on `sm_100`.
- Osama, Merrill, Cecka, Garland, Owens, "Stream-K: Work-centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU" (PPoPP 2023, arXiv:2301.03598).
- Meta Llama 3 model card / `config.json` (70B: hidden 8192, intermediate 28672, 8 KV heads $\times$ 128) for the shapes.

Inspiration (not a source): KyrieBlunders' post on X, https://x.com/KyrieBlunders/status/2106058102374375
and the linked article "Hand-writing a Blackwell GEMM: 164 to 1400 TFLOP/s", https://x.com/i/article/21049383
All timings, Nsight Compute figures and ablation results in Sections 5–10 are that article's *reported* B200 measurements and were not reproduced here. The kernel in this chapter is original and does not reuse the article's code.

Flash Attention 4

Flash Attention 4

The 2026 Frontier: Breaking the 1 PFLOPS Single-GPU Barrier

On Blackwell hardware, Tensor Core matrix multiply units run so extraordinarily fast that the normalization step in Softmax—historically a minor fraction of runtime—becomes the dominant bottleneck.

FlashAttention-4 introduces **Adaptive Softmax**: - Dynamically checks divergence between running block maximums and candidate maximums. - Skips full re-normalization when numerical differences fall below precision thresholds. - Overlaps Softmax correction entirely behind asynchronous WGMMA execution pipelines. - Written using modern Python CuTe-DSL, FA4 is the first kernel to achieve over 1 PFLOPS of sustained attention compute on a single processor.

PagedAttention and the KV Cache

PagedAttention and the KV Cache

Autoregressive LLM serving is memory-bound in decode and **capacity-bound** in concurrency: each active sequence carries a growing key/value (KV) cache. Contiguous per-sequence KV allocations waste large fractions of GPU DRAM to fragmentation and over-reservation. **PagedAttention** (vLLM) borrows the OS virtual-memory idea—fixed-size blocks, a block table, allocate-on-demand—so attention kernels read logically contiguous tokens from physically non-contiguous KV blocks.

This chapter goes deeper than a one-paragraph mention in inference-serving notes: why contiguous KV fails, how block tables work, prefill vs decode implications, layout choices for custom kernels, and how to verify/benchmark paging behavior.

Mental model

OS analogy	LLM serving
Process virtual address space	Logical token positions $0..T-1$ in a sequence
Pages	KV **blocks** of B tokens
Page table	Per-sequence **block table** (logical block → physical block)
Physical frames	Slots in a GPU KV pool
Internal fragmentation	At most one partially filled tail block
External fragmentation	Largely eliminated (any free block can be used)

```
logical tokens:  [t0 t1 ... t15 | t16 ... t31 | t32 ... t40____]
                   block 0          block 1       block 2 (partial)

block table:     L0 → P17
                 L1 → P3
                 L2 → P9

physical pool:   P0 P1 P2 P3[K/V] ... P9[K/V] ... P17[K/V] ...
```

Attention still needs Q against all prior K, and weighted sum over V—but the kernel **gathers** K, V through the block table instead of assuming one contiguous pointer.

Why contiguous KV wastes memory

For a sequence with maximum length $L_{\max}$ and current length t:

1. **Reservation waste:** many systems allocate KV for $L_{\max}$ up front even while $t \ll L_{\max}$.

2. **External fragmentation:** variable-length contiguous regions cannot pack tightly as sequences finish at different times.

3. **Batching rigidity:** contiguous layouts make sharing prefixes or compacting live sequences expensive.

Empirical reports in the PagedAttention / vLLM line of work put waste often in the **60–80%** range under contiguous reservation—memory that cannot become additional concurrent sequences. Paging caps internal waste at roughly one block (B tokens) per sequence (per managed cache partition) and removes most external fragmentation.

Capacity sketch

Let one token's KV footprint (all layers, all heads, K and V) be b bytes. Contiguous reservation for N sequences costs $\sim NL_{\max}b$. Paged on-demand costs $\sim b\sum_i t_i$ plus $< NBb$ partial-block slack (order-of-magnitude). The ratio $L_{\max}/\bar{t}$ dominates the win when outputs are much shorter than the configured max.

KV cache in Transformers (reminder)

For each layer ℓ and head h, token position i stores vectors $k_i^{(\ell,h)}, v_i^{(\ell,h)} \in \mathbb{R}^d$. Causality at decode step t uses

$$\alpha_{t,i} = \mathrm{softmax}_i\left(\frac{q_t^\top k_i}{\sqrt{d}}\right), \qquad o_t = \sum_{i=1}^{t} \alpha_{t,i} v_i.$$

Prefill computes many q rows in parallel against the prompt's K, V; decode appends one new (k_t, v_t) and attends with a single q_t (or a small speculative set).

Serving reality: weights are shared; **KV grows per request**. Throughput (tokens/s/GPU) often rises when more sequences fit—hence KV packing is a first-class systems problem, not a minor allocator tweak.

Block structure

Choose block size B (vLLM commonly discusses values like 16). Logical block j holds tokens $(j-1)B+1, \ldots, jB$ (or 0-based equivalents):

$$K_j = (k_{(j-1)B+1}, \ldots, k_{jB}), \qquad V_j = (v_{(j-1)B+1}, \ldots, v_{jB}).$$

Attention becomes a loop over logical blocks: load the physical block pointed by the table, compute partial scores/outputs, advance. The last block carries a **filled-token count** $1..B$.

Worked layout numbers

$B = 16$, head dim $d = 128$, dtype FP16: one head's K block stores 16×128 elements $= 2048$ halves ≈ 4 KiB (order of magnitude), and V similarly. Multi-layer multi-head caches scale linearly; production engines may use separate block tables per layer/head partition or pack carefully for coalescing—design choice, not a change to the paging idea.

Block table and allocator

Per sequence (and often per KV partition):

Field	Role
Logical index j	Token range $[jB, (j+1)B)$
Physical block id	Index into GPU (or CPU swap) pool
Filled count	Valid tokens in the tail block
Refcount (advanced)	Prefix sharing / copy-on-write

Allocate on demand: when decode would write past the filled count, pull a free physical block, append a table entry. **Free on finish:** return blocks to the pool (and decrement refcounts if shared).

```
decode step:
  if filled == B:
      phys = alloc_block()
      table.append(phys)
      filled = 0
  write k_t,v_t into table[-1] at offset filled
  filled += 1
  run paged attention(q_t, table)
```

Prefix sharing (copy-on-write intuition)

If two requests share a prompt prefix, map their early logical blocks to the **same** physical blocks with elevated refcounts. Divergence (different generated tokens) allocates private blocks from the fork point. Automatic prefix caching builds on the same indirection: hash logical block contents (+prefix identity) to reuse physical KV.

Prefill vs decode under paging

Phase	Compute character	Paging interaction
Prefill	Compute-heavy; many tokens written	Allocate enough blocks for prompt; write K/V densely left-to-right; attention over prompt blocks
Decode	Memory-bound; one token at a time	Append into tail block; attention gathers all prior blocks for one q

Custom kernels must handle **variable block lists per sequence in a batch**: sequence A may have 3 blocks, sequence B 40. Batching strategies (continuous batching) reshape which sequences sit in a wave while the block tables remain the source of truth for KV addresses.

Implication

A FlashAttention-style tiled softmax over a **contiguous** K, V buffer is not drop-in compatible. You need a kernel that:

1. Accepts block tables + pool base pointers, or
2. Materializes contiguous KV (defeats paging), or
3. Uses a hybrid (e.g., paged decode kernels + specialized prefill paths).

vLLM's historical PagedAttention CUDA kernels (`attention_kernels`-style designs) are built for case (1): thread blocks iterate paged K/V vectors with careful vectorized loads.

Paged KV layouts (kernel writer view)

Exact layouts evolve; treat the following as a **mental checklist** when reading kernels or writing prototypes:

```
Pool: physical_block[p] → storage for B tokens × head_dim (K or V)

K access for logical token t:
  j = t / B
  o = t % B
  p = block_table[seq][j]
  addr = base_K + p * stride_block + o * stride_token + head_offset

V access: similar with base_V / possibly different vectorization shape
```

K vs V vectorization: kernels often pack K for efficient $Q \cdot K$ dots (e.g., threads cooperating on a head vector) while V layouts favor weighting and accumulating into the output. Do not assume K and V share identical lane mapping.

Separate K and V caches: common—two pools, parallel tables or mirrored indices.

Illustrative host-side table (not a production kernel)

```python
# file: toy_block_table.py
# Toy model of logical→physical mapping (host-side). Not CUDA.

from dataclasses import dataclass, field

B = 16


@dataclass
class SequenceKV:
    table: list[int] = field(default_factory=list)  # physical ids
    filled: int = 0  # tokens used in last block


class BlockPool:
    def __init__(self, n_blocks: int):
        self.free = list(range(n_blocks))

    def alloc(self) -> int:
        if not self.free:
            raise MemoryError("KV pool exhausted")
        return self.free.pop()

    def free_block(self, p: int) -> None:
        self.free.append(p)


def append_token(seq: SequenceKV, pool: BlockPool) -> None:
    if not seq.table or seq.filled == B:
        seq.table.append(pool.alloc())
        seq.filled = 0
    # device would write K/V into pool[seq.table[-1]][seq.filled]
    seq.filled += 1


def release(seq: SequenceKV, pool: BlockPool) -> None:
    for p in seq.table:
        pool.free_block(p)
    seq.table.clear()
    seq.filled = 0
```

Use this only to reason about allocation traffic; real engines track layers/heads, CUDA streams, and fragmentation across devices.

Implications for custom kernels

1. **Indirection tax:** every K/V load pays a table lookup (often cached in shared memory / registers for the active block list).

2. **Tail blocks:** masks for `filled < B` are mandatory—same family of bugs as ragged sequence lengths.

3. **Batching:** one wave may include many tables; prefer grouped launches or paged-friendly attention APIs over naively padding to $L_{\max}$ contiguous.

4. **Fusion:** paging composes with continuous batching, speculative decoding (extra parallel tokens), and quantization of KV (INT8/FP8 caches)—each changes strides and scales, not the block-table story.

5. **Swap / multi-tier:** physical blocks may live in CPU RAM; kernels and schedulers must not assume all table entries are GPU-resident without a residency check.

When generating kernels with AI assistants: specify **block size, K/V separation, dtype, head dim, and tail masking** explicitly. Ambiguous "write attention" prompts usually emit contiguous FlashAttention clones that cannot consume a paged pool.

Verification and benchmark notes

Functional checks

Check	Idea
Match contiguous golden	For small T, gather paged KV to contiguous and compare attention outs vs a reference softmax
Tail mask	Lengths $B-1$, B, $B+1$ across block boundaries
Multi-sequence batch	Different table lengths in one launch
Prefix share	Refcount: mutate after fork must not clobber peer

Performance / capacity checks

Metric	Why
Max concurrent sequences at fixed $L_{\max}$	Primary paging win
KV bytes reserved vs bytes touched	Fragmentation / over-reserve detector

Metric	Why
Decode tokens/s vs batch size	Memory bandwidth + kernel efficiency
Allocation failures under load	Pool sizing / preemption / swap policy

Microbenchmarks that only time a contiguous GEMM miss the point—page-table attention is about **fitting more live KV** without catastrophic gather overhead. Profile both **allocator** and **attention kernel**.

Profiling tips

- Trace block alloc/free counts per request; spikes may indicate missing prefix sharing.
- Compare achieved occupancy / bytes loaded for paged vs contiguous decode on identical batch shapes.
- Watch for CPU-sync on block-table uploads each step—pin tables or update incrementally.

Pitfalls

Pitfall	Result
Assuming contiguous FlashAttention works unchanged	Wrong answers or illegal loads
Ignoring partial-block fill	Garbage tokens in softmax
Oversized B	More internal waste; less flexible packing
Undersized pool	Preemption thrash or request rejects
Forgetting refcount on shared prefixes	Silent cross-request corruption
Benchmarking only prefill TFLOPs	Misses decode memory + paging behavior

Checkpoint

- Explain why contiguous $L_{\max}$ reservation destroys concurrency.
- Map OS paging terms onto block tables and KV pools.
- Describe allocate-on-demand during decode and tail `filled` counts.
- List what a paged attention kernel must accept beyond Q, K, V pointers.
- Name two functional tests and one capacity metric for paging.

Summary

PagedAttention makes KV cache **virtually contiguous and physically sparse**. The win is utilization—more sequences per GPU—at the cost of attention kernels that understand block tables, tail masks, and pool lifetimes. Prefill still wants heavy compute paths; decode wants lean paged gathers. For custom or AI-written kernels, treat the block table as part of the ABI, verify against contiguous goldens at block boundaries, and benchmark concurrency and KV bytes—not only matmul TFLOPs.

Sources (for further reading)

- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention" (SOSP 2023; arXiv:2309.06180).
- vLLM design docs on PagedAttention / paged KV layouts.
- vLLM blog: "Easy, Fast, and Cheap LLM Serving with PagedAttention."

Flash-Decoding: Split-KV Attention for Long-Context Decode

Flash-Decoding: Split-KV Attention for Long-Context Decode

FlashAttention made **prefill** fast by tiling $QK^\top$, softmax, and PV into one kernel that never writes the score matrix to HBM. **Decode** is a different problem. Each step has *one* new query row per sequence and head, attending over a KV cache that may hold tens or hundreds of thousands of tokens. A FlashAttention-style grid parallelizes over batch × heads × query tiles, and at decode there is only one query tile. With a small batch and a long context, that leaves a handful of CTAs to stream gigabytes of KV while most SMs sit idle.

Flash-Decoding (Dao, Haziza, Massa, Sizov, 2023) adds the missing parallel dimension: the **KV sequence**. Each split of the cache runs the familiar online-softmax loop independently and emits a partial output plus one scalar, its **log-sum-exp (LSE)**. A tiny second kernel then merges the splits exactly. The authors reported up to 8× faster decode for very long sequences, and the same split-then-merge pattern now sits under paged decode kernels, cascade or shared-prefix attention, and context-parallel attention across GPUs.

This chapter builds the idea from the algebra up. It proves the merge is exact, writes a two-kernel Triton implementation that you can verify on a laptop with the Triton interpreter, compiles it for a GPU you do not own, ports the production split-count heuristic, and lists the checks that catch the real bugs.

---

1. Why decode starves the GPU

For one (sequence, head) pair at decode step t:

$$s_i = \frac{q^\top k_i}{\sqrt{d}}, \qquad o = \sum_{i=1}^{t} \mathrm{softmax}(s)_i \, v_i.$$

The work is about $4td$ FLOPs against $2td$ elements of K and V that must be read from HBM. In FP16 that is about **1 FLOP per byte**, which puts it far left on the roofline: decode attention is purely **bandwidth-bound**. The only goal is to keep every SM's load pipeline full.

Kernel style	Grid at decode	CTAs for batch 1, 8 KV heads
FlashAttention (one CTA per batch × head × query tile)	$B \cdot H \cdot 1$	8

Kernel style	Grid at decode	CTAs for batch 1, 8 KV heads
Flash-Decoding (adds S KV splits)	$B \cdot H \cdot S$	$8S$ (e.g. 232 with $S = 29$)

Worked numbers

Batch 1, 8 KV heads (grouped-query attention), context 128 000, head dim 128, FP16 cache. K plus V for **one layer** is

$$2 \times 8 \times 128\,000 \times 128 \times 2 \text{ B} \approx 524 \text{ MB}.$$

At the H100 SXM's roughly 3.35 TB/s HBM3 peak, that is about 160 µs per layer if bandwidth is saturated. Eight CTAs on a 132-SM part cannot get close: each SM can only keep a limited number of loads in flight. Splitting the sequence 29 ways gives 232 CTAs, enough to occupy every SM and keep loads outstanding. The model is the same and so are the bytes; the kernel is simply parallelized along the dimension that actually has work.

GQA note: with grouped-query attention, several query heads share one KV head. Good decode kernels pack the query heads of a group into the "M" rows of a tile (FlashAttention-3 calls this `pack_gqa`), so the KV block is loaded once per group, not once per query head. The parallelism problem, and the split-KV fix, are about **KV heads × batch**.

---

2. Mental model: attention states and the merge operator

For a set of key indices I, define its **attention state** as the pair

$$\mathrm{LSE}(I) = \log \sum_{i \in I} e^{s_i}, \qquad o(I) = \sum_{i \in I} \frac{e^{s_i}}{e^{\mathrm{LSE}(I)}}\, v_i.$$

$o(I)$ is the attention output you would get if the cache contained **only** the keys in I. $\mathrm{LSE}(I)$ records how much total softmax mass I carries.

Claim. For disjoint I and J:

$$\mathrm{LSE}(I \cup J) = \log\!\left(e^{\mathrm{LSE}(I)} + e^{\mathrm{LSE}(J)}\right), \qquad o(I \cup J) = \frac{e^{\mathrm{LSE}(I)}\, o(I) + e^{\mathrm{LSE}(J)}\, o(J)}{e^{\mathrm{LSE}(I)} + e^{\mathrm{LSE}(J)}}.$$

Proof. $e^{\mathrm{LSE}(I)}\, o(I) = \sum_{i \in I} e^{s_i} v_i$ is the *unnormalized* numerator over I, and $e^{\mathrm{LSE}(I)}$ is the unnormalized denominator. Numerators and denominators over disjoint sets simply add. ■

Call this merge $\oplus$. It is **associative and commutative**, so the splits can be combined in any order: sequentially, as a tree, across warps, across CTAs, or across GPUs. It is the same operator FlashAttention applies tile by tile inside one CTA (the online softmax), now applied *between* CTAs.

FlashInfer makes it the canonical output of every attention kernel ("attention state") and exposes `merge_state` / `merge_states` as first-class operations.

Numerically stable form. Never exponentiate an LSE directly. Subtract the maximum first:

$$m = \max_j \mathrm{LSE}_j, \qquad w_j = e^{\mathrm{LSE}_j - m}, \qquad o = \frac{\sum_j w_j\, o_j}{\sum_j w_j}, \qquad \mathrm{LSE} = m + \log \sum_j w_j.$$

```
            KV cache for one (sequence, head):  t tokens

    split 0         split 1         split 2       split 3      (last split may be short or em

         online softmax over BLOCK_N tiles (kernel 1, one CTA per split)

   (o0, lse0)      (o1, lse1)      (o2, lse2)    (o3, lse3)     fp32 workspace in HBM

                                  kernel 2: m = max lse; w = exp(lse - m); o = Σ w·o / Σ w
                        final o  (and lse, if a caller wants to merge further)
```

---

3. Verify the algebra before writing a kernel

Start with a host reference so that every GPU result later has a ground truth.

```python
# file: split_kv_reference.py
# NumPy reference: one decode query against a long KV cache, computed
#  (a) in one pass, and (b) as Flash-Decoding style splits merged by log-sum-exp.
import numpy as np

rng = np.random.default_rng(0)
D, T, S = 128, 10_000, 7            # head dim, cached tokens, number of KV splits
q = rng.standard_normal(D).astype(np.float32)
K = rng.standard_normal((T, D)).astype(np.float32)
V = rng.standard_normal((T, D)).astype(np.float32)
scale = 1.0 / np.sqrt(D)


def attend(q, K, V):
    """Return (o, lse) for one query over a KV segment: o normalised, lse = log sum exp(s)."
    s = (K @ q) * scale
    m = s.max()
    p = np.exp(s - m)
    l = p.sum()
    return (p @ V) / l, m + np.log(l)
```

```
def merge(states):
    """Combine (o_i, lse_i) partial states into the exact full-segment state."""
    lse = np.array([st[1] for st in states])
    m = lse.max()
    w = np.exp(lse - m)                    # unnormalised weight of each split
    o = sum(wi * st[0] for wi, st in zip(w, states)) / w.sum()
    return o, m + np.log(w.sum())


o_full, lse_full = attend(q, K, V)

bounds = np.linspace(0, T, S + 1).astype(int)          # uneven split lengths are fine
parts = [attend(q, K[a:b], V[a:b]) for a, b in zip(bounds[:-1], bounds[1:])]
o_split, lse_split = merge(parts)

# The merge is associative: merging pairwise in a tree gives the same answer.
tree = merge([merge(parts[:3]), merge(parts[3:])])

print("split boundaries :", bounds.tolist())
print("max |o_full - o_split| =", float(np.abs(o_full - o_split).max()))
print("|lse_full - lse_split| =", float(abs(lse_full - lse_split)))
print("max |o_split - o_tree|  =", float(np.abs(o_split - tree[0]).max()))
```

```
python3 -m venv .venv && . .venv/bin/activate
pip install numpy
python split_kv_reference.py
```

Output:

```
split boundaries : [0, 1428, 2857, 4285, 5714, 7142, 8571, 10000]
max |o_full - o_split| = 3.8762266595278316e-10
|lse_full - lse_split| = 1.7920598338605487e-10
max |o_split - o_tree|  = 1.3877787807814457e-17
```

Split, merge, or tree-merge: the results agree to rounding. Uneven split lengths do not matter, because each split's LSE carries its own weight.

The trap: the textbook merge overflows

```
# file: merge_trap.py
# The textbook merge  o = sum(exp(lse_i) * o_i) / sum(exp(lse_i))  overflows float32
# as soon as any lse_i > ~88.7. Subtracting max(lse) first is not optional.
import numpy as np
```

```
lse = np.array([95.0, 97.5, 91.2], dtype=np.float32)        # plausible with large logits
o = np.array([[1.0, 0.0], [0.0, 1.0], [0.5, 0.5]], dtype=np.float32)

with np.errstate(over="ignore", invalid="ignore"):
    w_naive = np.exp(lse)
    print("naive weights :", w_naive)
    print("naive merge   :", (w_naive[:, None] * o).sum(0) / w_naive.sum())

m = lse.max()
w = np.exp(lse - m)
print("stable weights:", w)
print("stable merge  :", (w[:, None] * o).sum(0) / w.sum())
print("merged lse    :", m + np.log(w.sum()))
```

```
python merge_trap.py
```

Output:

```
naive weights : [inf inf inf]
naive merge   : [nan nan]
stable weights: [0.08208501 1.         0.0018363 ]
stable merge  : [0.07657673 0.92342323]
merged lse    : 97.58058
```

`exp` overflows FP32 above about 88.7, and LSE values that large are routine once logits grow (long contexts, no QK-norm, large activations). Kernels that "pass the unit test" with `randn` inputs and then NaN in production usually have exactly this bug in the combine step.

---

4. A two-kernel Triton implementation

The design is the minimum that is still honest about production concerns:

- **Kernel 1** (`split_kv_kernel`), grid $(B \cdot H,\ S)$. Each program owns one split of one (sequence, head) pair, streams its keys in `BLOCK_N` tiles with the online-softmax recurrence, and writes a normalized partial output and its LSE to an FP32 workspace.
- **Kernel 2** (`combine_kernel`), grid $(B \cdot H)$. Loads S LSEs and S partial outputs, applies the stable merge, and writes the result.
- **Ragged batches.** `SEQ_LENS[b]` bounds every sequence, so a sequence shorter than the split layout leaves some splits **empty**. An empty split must write $o = 0$ and LSE $= -\infty$, giving it weight $e^{-\infty} = 0$. Writing $0/0$ there is the classic NaN.
- **FP32 everywhere it matters.** Scores, running max, running sum, accumulator, and workspace are all FP32, whatever the cache dtype. FlashAttention-2's split-KV path allocates its `out_accum` and `softmax_lse_accum` workspaces in FP32 for the same reason.

```python
# file: flash_decode_triton.py
# Two-kernel Flash-Decoding (split-KV) for batched single-token decode, written in Triton.
#   kernel 1: grid = (batch*heads, num_splits)  -> partial (o, lse) per split
#   kernel 2: grid = (batch*heads,)             -> log-sum-exp merge across splits
# Runs on a GPU, or on CPU for correctness work with:  TRITON_INTERPRET=1 python flash_decod
import math
import os

import torch
import triton
import triton.language as tl


@triton.jit
def split_kv_kernel(Q, K, V, SEQ_LENS, O_PART, LSE_PART,
                    stride_kb, stride_kh, stride_kt,      # K/V: [B, H, T_max, D], last dim
                    H, chunk, scale,
                    D: tl.constexpr, BLOCK_N: tl.constexpr, NUM_SPLITS: tl.constexpr):
    bh = tl.program_id(0)
    split = tl.program_id(1)
    b = bh // H
    h = bh % H
    seq_len = tl.load(SEQ_LENS + b)

    start = split * chunk
    end = tl.minimum(start + chunk, seq_len)              # may be <= start: empty split

    offs_d = tl.arange(0, D)
    q = tl.load(Q + bh * D + offs_d).to(tl.float32)       # Q: [B*H, D]
    kv_base = b * stride_kb + h * stride_kh

    m_i = -float("inf")                                   # running max of scores
    l_i = 0.0                                             # running sum of exp(score - m_i)
    acc = tl.zeros([D], dtype=tl.float32)                 # running sum of p * v

    for n0 in range(start, end, BLOCK_N):
        offs_n = n0 + tl.arange(0, BLOCK_N)
        mask_n = offs_n < end
        ptrs = kv_base + offs_n[:, None] * stride_kt + offs_d[None, :]
        k = tl.load(K + ptrs, mask=mask_n[:, None], other=0.0).to(tl.float32)
        s = tl.sum(k * q[None, :], axis=1) * scale        # [BLOCK_N] scores
        s = tl.where(mask_n, s, -float("inf"))
        m_new = tl.maximum(m_i, tl.max(s, axis=0))
        alpha = tl.exp(m_i - m_new)                       # rescale old state
        p = tl.exp(s - m_new)
        v = tl.load(V + ptrs, mask=mask_n[:, None], other=0.0).to(tl.float32)
        acc = acc * alpha + tl.sum(p[:, None] * v, axis=0)
```

```
        l_i = l_i * alpha + tl.sum(p, axis=0)
        m_i = m_new

    has_keys = end > start
    safe_l = tl.where(has_keys, l_i, 1.0)
    o = tl.where(has_keys, acc / safe_l, 0.0)              # empty split -> o = 0 ...
    lse = tl.where(has_keys, m_i + tl.log(safe_l), -float("inf"))  # ... with weight exp(-in
    out = bh * NUM_SPLITS + split
    tl.store(O_PART + out * D + offs_d, o)
    tl.store(LSE_PART + out, lse)


@triton.jit
def combine_kernel(O_PART, LSE_PART, O,
                   D: tl.constexpr, NUM_SPLITS: tl.constexpr):
    bh = tl.program_id(0)
    offs_s = tl.arange(0, NUM_SPLITS)
    offs_d = tl.arange(0, D)
    lse = tl.load(LSE_PART + bh * NUM_SPLITS + offs_s)      # [S]
    m = tl.max(lse, axis=0)
    w = tl.exp(lse - m)                                     # empty splits contribute 0
    o_part = tl.load(O_PART + (bh * NUM_SPLITS + offs_s)[:, None] * D + offs_d[None, :])
    o = tl.sum(w[:, None] * o_part, axis=0) / tl.sum(w, axis=0)
    tl.store(O + bh * D + offs_d, o)


def flash_decode(q, k, v, seq_lens, num_splits=8, block_n=32):
    """q: [B, H, D]; k, v: [B, H, T_max, D]; seq_lens: [B] int32. Returns [B, H, D] float32.
    B, H, T_max, D = k.shape
    chunk = triton.cdiv(T_max, num_splits)
    o_part = torch.empty(B * H, num_splits, D, dtype=torch.float32, device=q.device)
    lse_part = torch.empty(B * H, num_splits, dtype=torch.float32, device=q.device)
    out = torch.empty(B, H, D, dtype=torch.float32, device=q.device)
    split_kv_kernel[(B * H, num_splits)](
        q.contiguous(), k, v, seq_lens, o_part, lse_part,
        k.stride(0), k.stride(1), k.stride(2),
        H, chunk, 1.0 / math.sqrt(D),
        D=D, BLOCK_N=block_n, NUM_SPLITS=num_splits)
    combine_kernel[(B * H,)](o_part, lse_part, out, D=D, NUM_SPLITS=num_splits)
    return out


def reference(q, k, v, seq_lens):
    B, H, _, D = k.shape
    out = torch.empty(B, H, D, dtype=torch.float32)
    for b in range(B):
        L = int(seq_lens[b])
```

```
        s = torch.einsum("hd,htd->ht", q[b].float(), k[b, :, :L].float()) / math.sqrt(D)
        out[b] = torch.einsum("ht,htd->hd", torch.softmax(s, dim=-1), v[b, :, :L].float())
    return out


if __name__ == "__main__":
    device = "cpu" if os.environ.get("TRITON_INTERPRET") == "1" else "cuda"
    torch.manual_seed(0)
    B, H, T_max, D = 3, 4, 1000, 64
    seq_lens = torch.tensor([1000, 37, 1], dtype=torch.int32, device=device)  # ragged batch
    q = torch.randn(B, H, D, device=device)
    k = torch.randn(B, H, T_max, D, device=device)
    v = torch.randn(B, H, T_max, D, device=device)
    for q_gain in (1.0, 40.0):                          # 40x -> logits in the hundreds
        for splits in (1, 4, 8):
            qq = q * q_gain
            got = flash_decode(qq, k, v, seq_lens, num_splits=splits)
            ref = reference(qq.cpu(), k.cpu(), v.cpu(), seq_lens.cpu())
            err = (got.cpu() - ref).abs().max().item()
            print(f"q_gain={q_gain:>4}  num_splits={splits}:  max abs error = {err:.2e}")
```

Run it on a CPU with the Triton interpreter. The same file runs unchanged on a CUDA GPU without the environment variable.

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install triton
TRITON_INTERPRET=1 python flash_decode_triton.py
```

Output (Triton 3.8.0, PyTorch 2.14.1 CPU, interpreter mode):

```
q_gain= 1.0  num_splits=1:  max abs error = 2.38e-07
q_gain= 1.0  num_splits=4:  max abs error = 2.38e-07
q_gain= 1.0  num_splits=8:  max abs error = 2.38e-07
q_gain=40.0  num_splits=1:  max abs error = 9.18e-06
q_gain=40.0  num_splits=4:  max abs error = 9.06e-06
q_gain=40.0  num_splits=8:  max abs error = 9.06e-06
```

What the test matrix covers:

Case	Why
`num_splits=1`	Degenerates to plain FlashAttention decode: the baseline
`seq_lens = [1000, 37, 1]`	Ragged batch. With 8 splits of 125 tokens, sequence 1 has 7 empty splits and sequence 2 has a one-token cache
`q_gain=40`	Logits in the hundreds: exercises the max-subtraction in both kernels

Case	Why
Comparison against `torch.softmax`	An independent reference, not a second copy of the same algorithm

The interpreter executes the kernel's tile semantics with NumPy on the CPU. It proves **correctness**, not performance: it does not model warps, shared memory, or bandwidth. It is still the fastest loop for an AI-assisted workflow, because a generated kernel can be run against a reference in seconds without a GPU.

---

5. Compile for a GPU you do not have

Interpreter success does not guarantee that the kernel lowers. Loop-carried values that change dtype, non-constexpr shapes, and unsupported ops fail only in the real compiler. Triton can compile ahead of time for a named target on a CPU-only machine:

```
# file: compile_check.py
# Ahead-of-time compile both kernels for an H100 (sm_90) on a machine with no GPU.
# Catches type errors, bad constexpr use and loop-carried dtype mismatches before you rent a
import triton
from triton.backends.compiler import GPUTarget
from triton.compiler import ASTSource

from flash_decode_triton import combine_kernel, split_kv_kernel

target = GPUTarget("cuda", 90, 32)          # backend, compute capability, warp size
split_sig = {"Q": "*fp32", "K": "*fp32", "V": "*fp32", "SEQ_LENS": "*i32",
             "O_PART": "*fp32", "LSE_PART": "*fp32",
             "stride_kb": "i32", "stride_kh": "i32", "stride_kt": "i32",
             "H": "i32", "chunk": "i32", "scale": "fp32",
             "D": "constexpr", "BLOCK_N": "constexpr", "NUM_SPLITS": "constexpr"}
combine_sig = {"O_PART": "*fp32", "LSE_PART": "*fp32", "O": "*fp32",
               "D": "constexpr", "NUM_SPLITS": "constexpr"}

jobs = [
    ("split_kv_kernel", ASTSource(fn=split_kv_kernel, signature=split_sig,
                                  constexprs={"D": 128, "BLOCK_N": 64, "NUM_SPLITS": 32})),
    ("combine_kernel", ASTSource(fn=combine_kernel, signature=combine_sig,
                                 constexprs={"D": 128, "NUM_SPLITS": 32})),
]
for name, src in jobs:
    k = triton.compile(src, target=target)
    md = k.metadata
    print(f"{name:16s} stages={','.join(k.asm.keys())}  "
```

```
          f"num_warps={md.num_warps}  shared={md.shared} B  ptx_lines={k.asm['ptx'].count(ch
```

```
python compile_check.py
```

Output (Triton 3.8.0 on a CPU-only Linux box):

```
split_kv_kernel  stages=source,ttir,ttgir,llir,ptx,cubin  num_warps=4  shared=128 B  ptx_lin
combine_kernel   stages=source,ttir,ttgir,llir,ptx,cubin  num_warps=4  shared=0 B  ptx_lines
```

Both kernels lower all the way to an `sm_90` cubin. `k.asm["ptx"]` and `k.asm["ttgir"]` are there for inspection, and they repay it. In this build every K/V load comes out as a scalar `ld.global.b32`, because the hand-written signature tells the compiler nothing about pointer alignment or stride divisibility. At a normal launch, Triton specializes on arguments that are divisible by 16, and that specialization is what typically unlocks wider vector loads. Grepping the PTX of the kernel you actually launch is a cheap way to confirm you got them.

Wire this compile check, together with the interpreter test, into the agent loop as a cheap gate before any GPU time is spent.

---

6. How many splits?

Too few splits leave SMs idle. Too many add workspace traffic and combine work, and make each split too short to amortize its prologue. FlashAttention-2's dispatcher chooses S by **wave efficiency**: launch $B \cdot H \cdot S$ CTAs onto a fixed number of slots, and prefer the smallest S whose last wave is nearly as full as the best achievable.

```
# file: choose_splits.py
# Wave-efficiency heuristic for picking the number of KV splits, ported from the
# FlashAttention-2 split-KV dispatcher (num_splits_heuristic in csrc/flash_attn/flash_api.cp
import math


def num_splits(work_items, slots, n_kv_blocks, max_splits=128):
    if work_items >= 0.8 * slots:           # already ~fills the machine: do not split
        return 1
    max_splits = min(max_splits, slots, n_kv_blocks)

    def eligible(s):                        # 11 vs 12 splits of 64 blocks is really 11 spli
        return s == 1 or math.ceil(n_kv_blocks / s) != math.ceil(n_kv_blocks / (s - 1))

    eff = {}
    for s in range(1, max_splits + 1):
        if eligible(s):
            waves = work_items * s / slots
```

```
            eff[s] = waves / math.ceil(waves)   # fraction of the last wave that is busy
    best = max(eff.values())
    return next(s for s in sorted(eff) if eff[s] >= 0.85 * best)


SMS = 132                    # e.g. H100 SXM; FA2 passes 2 * num_SMs as the slot count
BLOCK_N = 128                # KV tokens per inner tile for head_dim 128
print(f"{'batch':>5} {'heads':>8} {'context':>8} {'work':>5} {'splits':>6} {'CTAs':>5}")
for batch, heads, ctx in [(1, 8, 128_000), (1, 32, 32_000), (4, 8, 8_000),
                          (16, 8, 4_000), (64, 32, 2_000), (1, 8, 300)]:
    work = batch * heads
    s = num_splits(work, 2 * SMS, math.ceil(ctx / BLOCK_N))
    print(f"{batch:>5} {heads:>8} {ctx:>8} {work:>5} {s:>6} {work * s:>5}")
```

```
python3 choose_splits.py
```

Output:

```
batch    heads  context  work splits  CTAs
    1        8   128000     8     29   232
    1       32    32000    32      7   224
    4        8     8000    32      7   224
   16        8     4000   128      2   256
   64       32     2000  2048      1  2048
    1        8      300     8      3    24
```

Reading the table:

- **Small batch, long context** is where Flash-Decoding matters: 8 work items become 232 CTAs.
- **Large batch** already fills the machine (2048 work items), so the heuristic returns 1 split and the extra pass is skipped entirely.
- **Short context** is capped by the number of KV tiles: 300 tokens are only 3 tiles of 128, so at most 3 splits.
- The "eligible" check skips split counts that produce the same tiles per split as a smaller count, which would add a combine for nothing.

Overhead check. Per (sequence, head), the workspace costs $S \cdot d$ FP32 values written and read back, about $8Sd$ bytes, while the KV stream is $4td$ bytes in FP16. The ratio $2S/t$ is about 0.05% for $S = 29$, $t = 128\,000$. The combine kernel should be a sliver in an Nsight Systems timeline. If it is not, S is too large or the combine is badly written.

---

7. Where the pattern shows up in production

System	How it uses split-KV and merge
FlashAttention-2 (`flash_attn_with_kvcache`)	`num_splits=0` selects the heuristic above; `block_table=` combines splits with a paged KV cache; `return_softmax_lse=True` returns the state for later merging
FlashAttention-3 (Hopper) interface	Exposes `num_splits` and `pack_gqa` for decode-shaped problems
FlashInfer	"Attention state" `(o, lse)` is the standard output; `merge_state` / `merge_states` combine segments; **cascade attention** computes a shared prompt prefix once for many requests and merges it with each request's private suffix
Paged-KV decode kernels	Split boundaries are aligned to KV pages; each CTA walks its slice of the block table
Context / ring parallelism	Each GPU holds a shard of the KV sequence; the same $\oplus$ merges partial states across devices
FlashDecoding++ (Hong et al.)	Replaces the per-split running max with a precomputed unified max to avoid synchronized rescaling, with a fallback path when values exceed it
LeanAttention	Applies Stream-K–style work partitioning to attention to reduce wave quantization

Short query blocks. Speculative decoding verifies several draft tokens at once, so q has 2–8 rows rather than 1. The same kernel shape applies, with a few query rows per CTA and the same split and merge.

---

8. Verification and benchmark plan

Functional

Check	Catches
`num_splits` ∈ `{1, 2, 3, S_max}` all equal the reference	Merge bugs, off-by-one split boundaries
`seq_len` ∈ `{1, BLOCK_N-1, BLOCK_N, BLOCK_N+1, chunk·k ± 1}`	Tail masks and split edges
Ragged batch with empty splits	0/0 in the empty-split path
Large-logit inputs (scale q)	Missing max subtraction in either kernel
BF16/FP8 cache vs FP32 reference with tolerance	Accumulating in the cache dtype
Return LSE and merge two halves externally	LSE scaling bugs (natural log vs `log2`, missing `1/√d`)

`log2` vs `ln`: many fast kernels compute `exp2(x * log2(e))` internally and keep the running max in base 2. Whatever the kernel uses internally, the LSE it **returns** must state its base. Merging a base-2 LSE with a natural-log LSE silently reweights the splits.

Performance

Metric	Target
Achieved bandwidth = KV bytes / kernel time	Approaches HBM peak for long contexts
Speedup vs `num_splits=1` across a (batch, context) sweep	Large at batch 1 and long context; about 1× at large batch
Combine kernel share of time	A few percent at most
Occupancy and eligible warps (Nsight Compute)	No spills from oversized `BLOCK_N × D` tiles

Benchmark with **realistic shapes**: the KV heads count after GQA, the actual cache dtype, and the batch sizes a server sees at its latency SLO. A sweep made only at batch 64 will "prove" split-KV is useless.

---

9. Prompting an AI assistant for a decode kernel

Generated attention kernels default to prefill-shaped FlashAttention clones. State the decode contract explicitly:

1. **Shapes:** query `[B, H_q, D]` (or `[B, q_len, H_q, D]` for speculative decode), KV `[B, H_kv, T_max, D]` or a paged pool plus `block_table`, and per-sequence `seq_lens`.
2. **Split-KV:** the number of splits as a runtime argument, an FP32 `(o, lse)` workspace, and a separate combine kernel or a documented fused alternative.
3. **Numerics:** an FP32 accumulator, stable max-subtracted merge, empty split `lse = -inf` and `o = 0`, and the LSE base stated.
4. **Tests first:** an interpreter-mode test against `torch.softmax` covering ragged lengths, empty splits, and large logits, plus an ahead-of-time compile for the deployment architecture.
5. **Metric:** achieved GB/s versus HBM peak, not TFLOP/s.

Pitfalls

Pitfall	Result
Naive $\sum e^{\mathrm{LSE}_j} o_j$ merge	`inf` and `NaN` with large logits
Empty split writes `acc / l` with `l = 0`	NaN that poisons the merged output
FP16 workspace or accumulator	Accuracy loss that grows with context and split count
Splitting at large batch	Extra launch and combine with no gain

Pitfall	Result
Splitting a 300-token cache 32 ways	Splits shorter than one tile, so the overhead dominates
Mixing `log2` and natural-log LSEs	Silently wrong weights, no crash
Benchmarking only TFLOP/s	Misses the point: decode is about bytes per second

Checkpoint

- Explain why a FlashAttention grid underfills the GPU at decode with a small batch.
- Define an attention state and prove that $\oplus$ merges disjoint KV sets exactly.
- Write the max-subtracted merge and say why the naive form fails in FP32.
- Describe what an empty split must write, and why.
- Use the wave-efficiency heuristic to say when *not* to split.
- Name three systems that reuse the same merge operator.

Try this

1. Change `split_kv_reference.py` to merge the splits in random order 100 times and confirm that the result never changes beyond rounding.
2. Make `split_kv_kernel` accumulate in FP16 and measure the error growth against `T_max` under `TRITON_INTERPRET=1`.
3. Add a `block_table` argument so that K and V come from a paged pool, and align the split boundaries to page boundaries.
4. Fuse the combine into kernel 1 using an atomic counter: the last CTA to finish for a (sequence, head) performs the merge. Compare launch counts in Nsight Systems on a real GPU.
5. On a GPU, sweep batch $\in \{1, 4, 16, 64\}$ and context $\in \{1\text{k}, 8\text{k}, 64\text{k}, 128\text{k}\}$ with `num_splits=1` vs the heuristic, and plot achieved GB/s.

Summary

Decode attention is bandwidth-bound, and at small batch the FlashAttention grid has too few CTAs to use that bandwidth. Flash-Decoding splits the KV sequence across CTAs. Each split produces an attention state (o, LSE), and an associative, max-subtracted merge recombines the states exactly. The kernel engineering lives in the details: FP32 state, empty-split handling, ragged lengths, a split count chosen by wave efficiency, and an LSE convention stated as part of the ABI. Verify on CPU with the Triton interpreter, compile ahead of time for the target GPU, and judge the result in GB/s.

Sources (for further reading)

- Dao, Haziza, Massa, Sizov, "Flash-Decoding for long-context inference" (PyTorch blog / Stanford CRFM, Oct 2023).
- Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning" (2023); FlashAttention repository: `flash_attn_with_kvcache` docs and `num_splits_heuristic` in `csrc/flash_attn/flash_api.cpp`.
- Shah et al., "FlashAttention-3" (2024); `hopper/flash_attn_interface.py` (`num_splits`, `pack_gqa`).
- Ye et al., "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving" (arXiv:2501.01005); FlashInfer docs: "Attention States and Recursive Attention", `flashinfer.cascade.merge_state`.
- Hong et al., "FlashDecoding++: Faster Large Language Model Inference on GPUs" (arXiv:2311.01282).
- Milakov & Gimelshein, "Online normalizer calculation for softmax" (2018).
- Triton documentation: `TRITON_INTERPRET=1` interpreter mode and `triton.compile` / `GPUTarget`.

TurboQuant: Rotate-then-Quantize KV Caches at 3 Bits

TurboQuant: Rotate-then-Quantize KV Caches at 3 Bits

Long-context decode is a bandwidth problem that turns into a capacity problem. Every generated token reads the whole KV cache of its sequence, and every concurrent sequence keeps its own cache in HBM. FP8 caches halve the bill. Going below 8 bits is harder: keys carry **outlier channels**, a handful of dimensions whose magnitude is ten or twenty times the rest, and a naive 3-bit quantizer spends all its levels on them.

TurboQuant (Zandieh, Daliri, Hadian, Mirrokni; arXiv April 2025, ICLR 2026) handles outliers by changing the basis instead of fighting them. Multiply each vector by a random orthogonal matrix and every coordinate starts to look like a draw from one known distribution, the same for every vector and every channel. A single precomputed scalar codebook is then close to optimal everywhere. You store a few bits per coordinate and one norm per vector, with no per-block scales or zero points and no calibration data.

This chapter builds the quantizer in NumPy and checks the paper's distortion numbers. It also shows the result that decides how TurboQuant gets deployed: the unbiased inner-product variant (with QJL) loses to the plain MSE variant inside softmax attention. After that come the kernel-side algebra that makes decode cheap, where the method runs in production today, and the prior-art dispute around it.

Everything here runs on a CPU. The listings were run on Debian 13.7 with Python 3.13.5, NumPy 2.5.3 and SciPy 1.18.1, and the outputs are pasted unedited. GPU throughput numbers are quoted from their sources and labeled as such.

---

1. Why the KV cache is the thing to compress

For a GQA model, each token stores one key and one value vector per KV head per layer:

$$\text{bytes/token} = 2 \cdot L \cdot H_{kv} \cdot d \cdot \tfrac{b}{8}.$$

Weights are a fixed cost. The KV cache grows with context length times batch size, and decode reads all of it for every token. Compressing it buys capacity (more sequences or longer contexts) and bandwidth (fewer bytes per decode step) at once.

```
# file: kv_budget.py
# KV-cache bytes per token, and what a bit budget buys, for two GQA models.
GiB = 1024**3
```

```
models = {  # name: (layers, kv_heads, head_dim)
    "Llama-3.1-8B": (32, 8, 128),
    "Llama-3.1-70B": (80, 8, 128),
}

def kv_bytes_per_token(layers, kv_heads, head_dim, bits_per_coord, overhead_bits_per_vec=0):
    vectors = 2 * layers * kv_heads                    # one K and one V per head per layer
    return vectors * (head_dim * bits_per_coord + overhead_bits_per_vec) / 8

budget = 40 * GiB  # memory left for KV after weights, say
print(f"{'model':14} {'format':26} {'KiB/token':>9} {'x vs bf16':>9} {'tokens in 40 GiB':>17
for name, (L, H, d) in models.items():
    base = kv_bytes_per_token(L, H, d, 16)
    for label, bits, ovh in [("bf16", 16, 0),
                             ("fp8", 8, 0),
                             ("4-bit + fp16 norm", 4, 16),
                             ("3.5-bit + fp16 norm", 3.5, 16),
                             ("3-bit + fp16 norm", 3, 16),
                             ("2.5-bit + fp16 norm", 2.5, 16)]:
        b = kv_bytes_per_token(L, H, d, bits, ovh)
        print(f"{name:14} {label:26} {b / 1024:9.1f} {base / b:9.2f} {budget / b:17,.0f}")
```

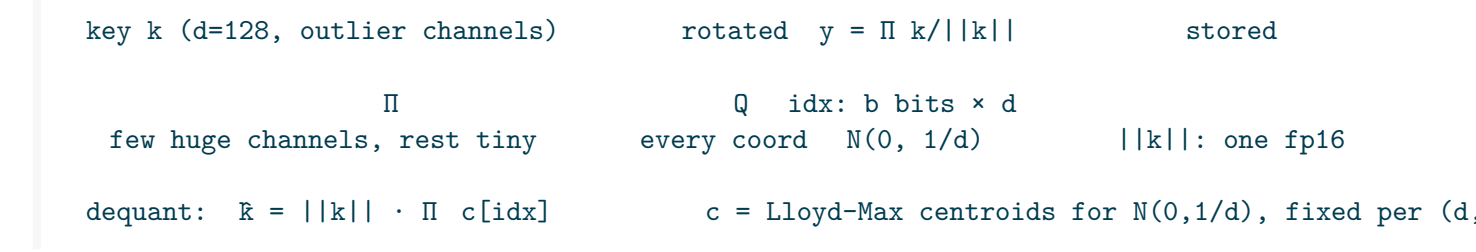
```
python kv_budget.py
```

Output:

```
model          format                     KiB/token x vs bf16  tokens in 40 GiB
Llama-3.1-8B   bf16                           128.0      1.00           327,680
Llama-3.1-8B   fp8                             64.0      2.00           655,360
Llama-3.1-8B   4-bit + fp16 norm               33.0      3.88         1,271,001
Llama-3.1-8B   3.5-bit + fp16 norm             29.0      4.41         1,446,312
Llama-3.1-8B   3-bit + fp16 norm               25.0      5.12         1,677,722
Llama-3.1-8B   2.5-bit + fp16 norm             21.0      6.10         1,997,288
Llama-3.1-70B  bf16                           320.0      1.00           131,072
Llama-3.1-70B  fp8                            160.0      2.00           262,144
Llama-3.1-70B  4-bit + fp16 norm               82.5      3.88           508,400
Llama-3.1-70B  3.5-bit + fp16 norm             72.5      4.41           578,525
Llama-3.1-70B  3-bit + fp16 norm               62.5      5.12           671,089
Llama-3.1-70B  2.5-bit + fp16 norm             52.5      6.10           798,915
```

The 16-bit norm per vector is real overhead: at 3 bits it adds $16/128 = 0.125$ bits per coordinate. Google's announcement of "at least 6x" KV memory reduction corresponds to the 2.5-bit row. The paper's own claim is narrower and more useful: **quality neutrality at 3.5 bits per channel, marginal degradation at 2.5 bits**, on LongBench and needle-in-a-haystack with Llama-3.1-8B and Ministral-7B.

2. Mental model: make every coordinate look the same, then use one codebook



Three facts carry the method.

1. **A random rotation spreads energy.** If Π is a uniformly random orthogonal matrix and u a unit vector, Πu is uniform on the sphere S^{d-1}, whatever u was. One coordinate of a uniform point on the sphere has density proportional to $(1-x^2)^{(d-3)/2}$ on $[-1,1]$, which is very close to $\mathcal{N}(0,1/d)$ for $d \geq 64$. The outliers are gone, smeared across all 128 coordinates.
2. **In high dimension the coordinates are nearly independent.** That makes per-coordinate scalar quantization nearly as good as vector quantization. The paper proves the MSE quantizer's distortion is within a factor $\sqrt{3}\pi/2 \approx 2.7$ of the information-theoretic lower bound 4^{-b} for *every* input, and much closer at low bit-widths.
3. **The codebook is data-independent.** The distribution in (1) depends only on d, so the optimal 2^b levels (Lloyd-Max for that density) are computed once, offline. Nothing is learned from the model, and quantization is online: one vector in, one code out, so it works token by token during decode.

A dense Haar rotation costs $O(d^2)$ per vector. Production code uses a **randomized Hadamard transform** $y = HDx$ (D a random ± 1 diagonal, H the orthonormal Walsh–Hadamard matrix). It costs $O(d \log d)$, is exactly orthogonal, and is close enough to Haar for this purpose. Section 8 shows what happens when you drop D.

---

3. The quantizer in NumPy

`tq_core.py` contains everything: the Hadamard butterfly, the two rotations, the Lloyd-Max solver for the sphere-coordinate density, the MSE quantizer, and the inner-product quantizer that adds a 1-bit QJL sketch of the residual.

```
# file: tq_core.py
# The pieces of a TurboQuant-style vector quantizer, in plain NumPy:
#   randomized Hadamard rotation, a Lloyd-Max codebook for the coordinates of a
#   random unit vector, the MSE quantizer, and the QJL residual sketch.
```

```python
import numpy as np


def fwht(x):
    """Orthonormal fast Walsh-Hadamard transform along the last axis (length 2^m)."""
    x = np.array(x, dtype=np.float64, copy=True)
    shape, n = x.shape, x.shape[-1]
    assert n & (n - 1) == 0, "Hadamard length must be a power of two"
    x = x.reshape(-1, n)
    h = 1
    while h < n:
        x = x.reshape(-1, n // (2 * h), 2, h)
        a = x[:, :, 0, :].copy()
        b = x[:, :, 1, :]
        x[:, :, 0, :] = a + b
        x[:, :, 1, :] = a - b
        x = x.reshape(-1, n)
        h *= 2
    return x.reshape(shape) / np.sqrt(n)


class RandomizedHadamard:
    """y = H D x with D a fixed random +-1 diagonal. Orthogonal, O(d log d), and its own
    transpose up to D: x = D H y."""

    def __init__(self, d, rng):
        self.signs = rng.choice([-1.0, 1.0], size=d)

    def forward(self, x):
        return fwht(x * self.signs)

    def inverse(self, y):
        return fwht(y) * self.signs


def lloyd_max_sphere(d, bits, grid=400_001, iters=300):
    """Optimal scalar codebook for one coordinate of a uniform random unit vector in R^d.
    That coordinate has density proportional to (1 - x^2)^((d - 3) / 2) on [-1, 1]."""
    x = np.linspace(-1, 1, grid)
    w = np.clip(1 - x * x, 0, None) ** ((d - 3) / 2)
    w /= w.sum()
    k = 2**bits
    sd = 1 / np.sqrt(d)
    c = np.linspace(-2 * sd, 2 * sd, k)                  # start near the bulk of the mass
    for _ in range(iters):
        edges = (c[1:] + c[:-1]) / 2                     # nearest-centroid boundaries
        cell = np.searchsorted(edges, x)
```

```
        mass = np.bincount(cell, weights=w, minlength=k)
        c = np.bincount(cell, weights=w * x, minlength=k) / mass   # centroid = conditional
    edges = (c[1:] + c[:-1]) / 2
    mse_per_coord = np.sum(w * (x - c[np.searchsorted(edges, x)]) ** 2)
    return c, edges, d * mse_per_coord                  # last value: E||u - Q(u)||^2 for un


class HaarRotation:
    """A dense uniformly random orthogonal matrix (QR of a Gaussian, signs fixed). O(d^2)."""

    def __init__(self, d, rng):
        q, r = np.linalg.qr(rng.standard_normal((d, d)))
        self.Q = q * np.sign(np.diag(r))

    def forward(self, x):
        return x @ self.Q.T

    def inverse(self, y):
        return y @ self.Q


class TurboQuantMSE:
    """Store b-bit indices per coordinate plus one norm per vector."""

    def __init__(self, d, bits, rng, norm_correction=False, rotation=RandomizedHadamard):
        self.rot = rotation(d, rng)
        self.centroids, self.edges, self.theory_mse = lloyd_max_sphere(d, bits)
        self.norm_correction = norm_correction

    def quantize(self, x):
        norm = np.linalg.norm(x, axis=-1, keepdims=True)
        y = self.rot.forward(x / norm)
        return np.searchsorted(self.edges, y).astype(np.uint8), norm.astype(np.float16)

    def dequantize(self, idx, norm):
        y = self.centroids[idx]
        if self.norm_correction:                        # put the codeword back on the sphe
            y = y / np.linalg.norm(y, axis=-1, keepdims=True)
        return norm.astype(np.float64) * self.rot.inverse(y)


class TurboQuantProd:
    """(b-1)-bit MSE quantizer, then a 1-bit QJL sketch of the residual: an unbiased
    estimator of inner products <q, x>."""

    def __init__(self, d, bits, rng):
```

```
        self.mse = TurboQuantMSE(d, bits - 1, rng)
        self.S = rng.standard_normal((d, d))
        self.d = d

    def quantize(self, x):
        idx, norm = self.mse.quantize(x)
        r = x - self.mse.dequantize(idx, norm)
        return idx, norm, np.sign(r @ self.S.T).astype(np.int8), np.linalg.norm(r, axis=-1,

    def dequantize(self, idx, norm, qjl, rnorm):
        corr = np.sqrt(np.pi / 2) / self.d * rnorm * (qjl @ self.S)
        return self.mse.dequantize(idx, norm) + corr
```

A few details matter:

- **Normalize, then rotate.** The codebook is designed for coordinates of a *unit* vector. Storing $\|x\|$ separately is what lets one codebook serve every token. (vLLM stores it as fp16.)
- **Indices are searched against midpoints.** With sorted centroids, nearest-centroid assignment is a binary search over $2^b - 1$ boundaries. vLLM's Triton store kernel does exactly that.
- **`norm_correction`** rescales the decoded codeword back to unit length before un-rotating. Quantization shrinks vectors (the centroids are conditional means, which sit inside the sphere), and the correction undoes that radial bias. vLLM's `_nc` presets turn it on, and its docs report about 0.8% perplexity gain at 4 bits.
- **QJL** (Zandieh, Daliri, Han; AAAI 2025) stores $\text{sign}(Sr)$ for a Gaussian matrix S and the residual r, plus $\|r\|$. The estimator $\sqrt{\pi/2}\,/\,d \cdot \|r\|\, S^\top \text{sign}(Sr)$ is unbiased for inner products with r. Spending one of the b bits on it makes $\langle q, \hat{x} \rangle$ unbiased.

---

4. Check the codebook against the paper

```
# file: tq_codebook.py
# Lloyd-Max codebooks for the coordinates of a random unit vector, and the distortion
# they buy, next to the numbers in the TurboQuant paper and its upper/lower bounds.
import numpy as np

from tq_core import HaarRotation, RandomizedHadamard, TurboQuantMSE, lloyd_max_sphere

d = 128
paper = {1: 0.36, 2: 0.117, 3: 0.03, 4: 0.009}
print(f"d = {d}")
for b in (1, 2):
    c, _, _ = lloyd_max_sphere(d, b)
    print(f"{b}-bit centroids x sqrt(d): {np.round(c * np.sqrt(d), 3)}")
```

```
print(f"{'bits':>4} {'D_mse (ours)':>12} {'paper':>6} {'lower 4^-b':>10} {'upper sqrt(3)pi/2
for b in (1, 2, 3, 4):
    _, _, D = lloyd_max_sphere(d, b)
    print(f"{b:>4} {D:12.4f} {paper[b]:6.3f} {4.0**-b:10.4f} {np.sqrt(3) * np.pi / 2 * 4.0**

# Same quantizer on real vectors. Two input families, two rotations.
rng = np.random.default_rng(0)
iso = rng.standard_normal((20_000, d))
spiky = iso * np.r_[np.full(4, 25.0), np.ones(d - 4)]       # 4 channels carry ~95% of the e
for b in (2, 3):
    for rot in (HaarRotation, RandomizedHadamard):
        tq = TurboQuantMSE(d, b, rng, rotation=rot)
        res = []
        for x in (iso, spiky):
            xh = tq.dequantize(*tq.quantize(x))
            res.append(np.mean(np.sum((x - xh) ** 2, 1) / np.sum(x**2, 1)))
        print(f"{b}-bit {rot.__name__:18} isotropic {res[0]:.4f}  spiky {res[1]:.4f}  "
              f"(model {tq.theory_mse:.4f})")
```

```
python tq_codebook.py
```

Output:

```
d = 128
1-bit centroids x sqrt(d): [-0.799  0.799]
2-bit centroids x sqrt(d): [-1.505 -0.452  0.452  1.505]
bits D_mse (ours)  paper lower 4^-b  upper sqrt(3)pi/2 4^-b
   1       0.3609  0.360     0.2500                 0.6802
   2       0.1160  0.117     0.0625                 0.1700
   3       0.0340  0.030     0.0156                 0.0425
   4       0.0093  0.009     0.0039                 0.0106
2-bit HaarRotation       isotropic 0.1159  spiky 0.1086  (model 0.1160)
2-bit RandomizedHadamard isotropic 0.1161  spiky 0.0880  (model 0.1160)
3-bit HaarRotation       isotropic 0.0339  spiky 0.0331  (model 0.0340)
3-bit RandomizedHadamard isotropic 0.0340  spiky 0.0301  (model 0.0340)
```

How to read it:

- The 1-bit centroids are $\pm\sqrt{2/\pi}/\sqrt{d} \approx \pm 0.798/\sqrt{d}$, and the 2-bit ones are $\pm 0.453/\sqrt{d}$ and $\pm 1.51/\sqrt{d}$. Both match the paper's closed forms.
- The distortions $0.36, 0.116, 0.034, 0.0093$ match the paper's $\approx 0.36, 0.117, 0.03, 0.009$. The paper rounds the 3-bit value. Each extra bit buys a bit less than the ideal factor of 4.
- The **model assumes a fresh random rotation per vector**. In practice one rotation is shared by every token of a layer, and keys with strong structure (four channels holding about 95% of the energy) do not land exactly on the model. Here they land *below* it: rotated, they spread over a few coordinates more evenly than a sphere sample would. Isotropic inputs land on the model, as they should. The guarantee is about the worst case in expectation over the

rotation. On real keys, measure.

---

5. Against the obvious 3-bit baselines

The comparison that matters for attention is the error in the logits $q \cdot k/\sqrt{d}$, not reconstruction MSE. This listing builds keys with four hot channels and a per-channel offset (the pattern KIVI and KVQuant reported in real K caches), plus queries that lean on the same channels, and tries six 3-bit schemes.

```
# file: tq_vs_naive.py
# 3-bit keys with channel outliers: naive per-token scaling vs per-channel scaling
# vs rotate-then-quantize. Error measured on the keys and on attention logits q.k/sqrt(d).
import numpy as np

from tq_core import RandomizedHadamard, TurboQuantMSE

rng = np.random.default_rng(1)
n, d, bits = 4096, 128, 3
scale = np.ones(d)
scale[[3, 17, 64, 101]] = [18.0, 12.0, 9.0, 15.0]    # a few hot channels, same for every to
K = rng.standard_normal((n, d)) * scale + 0.5 * scale  # plus a per-channel offset
Q = rng.standard_normal((64, d)) * np.sqrt(scale)      # queries lean on the same channels


def uniform_absmax(x, bits, axis):
    """Symmetric round-to-nearest with one fp16 scale per slice along `axis`."""
    qmax = 2 ** (bits - 1) - 1
    s = np.abs(x).max(axis=axis, keepdims=True) / qmax
    return np.clip(np.round(x / s), -qmax - 1, qmax) * s


def uniform_minmax(x, bits, axis):
    """Asymmetric round-to-nearest with fp16 scale and zero point per slice."""
    lo, hi = x.min(axis=axis, keepdims=True), x.max(axis=axis, keepdims=True)
    s = (hi - lo) / (2**bits - 1)
    return np.round((x - lo) / s) * s + lo


rht = RandomizedHadamard(d, rng)
tq = TurboQuantMSE(d, bits, rng)
tq_nc = TurboQuantMSE(d, bits, rng, norm_correction=True)
schemes = {
    "per-token absmax": uniform_absmax(K, bits, axis=1),
    "per-token min-max": uniform_minmax(K, bits, axis=1),
```

```
    "per-channel min-max": uniform_minmax(K, bits, axis=0),
    "RHT + per-token min-max": rht.inverse(uniform_minmax(rht.forward(K), bits, axis=1)),
    "TurboQuant-MSE": tq.dequantize(*tq.quantize(K)),
    "TurboQuant-MSE + norm fix": tq_nc.dequantize(*tq_nc.quantize(K)),
}
L = Q @ K.T / np.sqrt(d)
print(f"n={n} keys, d={d}, {bits} bits/coord, logit std {L.std():.2f}")
print(f"{'scheme':27} {'key rel. MSE':>12} {'logit RMSE':>10} {'top-1 match':>11}")
for name, Kh in schemes.items():
    rel = np.sum((K - Kh) ** 2) / np.sum(K**2)
    Lh = Q @ Kh.T / np.sqrt(d)
    top1 = np.mean(L.argmax(1) == Lh.argmax(1))
    print(f"{name:27} {rel:12.4f} {np.sqrt(np.mean((L - Lh) ** 2)):10.3f} {top1:11.2%}")
```

```
python tq_vs_naive.py
```

Output:

```
n=4096 keys, d=128, 3 bits/coord, logit std 11.50
scheme                       key rel. MSE logit RMSE top-1 match
per-token absmax                   0.1439      1.721      67.19%
per-token min-max                  0.2198      1.488      54.69%
per-channel min-max                0.0725      3.080      53.12%
RHT + per-token min-max            0.0307      0.621      79.69%
TurboQuant-MSE                     0.0297      0.631      73.44%
TurboQuant-MSE + norm fix          0.0294      0.601      81.25%
```

Three lessons:

1. **Per-token scaling fails on outlier channels.** The scale is set by the hottest channel, so the 120 ordinary channels round to one or two levels.
2. **Per-channel scaling wins on key MSE and loses on logits.** Per-channel scaling (what KIVI does for keys) spreads the *absolute* error evenly. The hot channels have huge ranges, so their absolute error is large, and those are exactly the channels the queries weight. Reconstruction MSE is the wrong metric.
3. **The rotation does most of the work.** At 3 bits, a randomized Hadamard followed by a plain per-token min-max quantizer is already within noise of TurboQuant-MSE. The Lloyd-Max codebook and norm storage add a little accuracy and remove the per-token scale and zero point (one fp16 instead of two). The top-1 column is computed over 64 queries, so treat differences of a few percent as noise.

---

6. The trap: unbiased is not the same as better

The paper offers two quantizers. TurboQuant-MSE minimizes $\|x - \hat{x}\|^2$. TurboQuant-prod spends one bit on QJL so that $\mathbb{E}\,\langle q, \hat{x}\rangle = \langle q, x\rangle$ exactly. Inner products are what attention computes, so the unbiased one looks like the right choice for keys. Check it against softmax.

```
# file: tq_prod.py
# Biased-but-tight (MSE) vs unbiased-but-noisy (MSE + QJL) inner products, and what
# each does to a peaked softmax over 4096 keys.
import numpy as np

from tq_core import TurboQuantMSE, TurboQuantProd

rng = np.random.default_rng(2)
d, n = 128, 4096

# Part 1: <q, k^> against <q, k> for unit vectors. Slope 1 = unbiased.
k = rng.standard_normal((n, d))
k /= np.linalg.norm(k, axis=1, keepdims=True)
q = k[:512] + 0.6 * rng.standard_normal((512, d)) / np.sqrt(d)   # queries correlated with k
q /= np.linalg.norm(q, axis=1, keepdims=True)
true = np.sum(q * k[:512], 1)
print("inner products, 512 correlated (q, k) pairs")
print(f"{'quantizer':22} {'slope':>6} {'resid std':>9}")
for bits in (2, 3, 4):
    for name, tq in ((f"MSE {bits}-bit", TurboQuantMSE(d, bits, rng)),
                     (f"MSE {bits - 1}-bit + QJL", TurboQuantProd(d, bits, rng))):
        est = np.sum(q * tq.dequantize(*tq.quantize(k[:512])), 1)
        slope = np.dot(est, true) / np.dot(true, true)
        print(f"{name:22} {slope:6.3f} {np.std(est - slope * true):9.4f}")

# Part 2: attention with one relevant key (logit ~ 9 above the crowd).
print("\nattention, 3 bits per key coordinate, 200 trials")
print(f"{'keys':18} {'p(needle)':>9} {'TV distance':>11} {'output rel err':>14}")
quants = {"fp32": None, "MSE 3-bit": TurboQuantMSE(d, 3, rng),
          "MSE 2-bit + QJL": TurboQuantProd(d, 3, rng)}
stats = {name: [] for name in quants}
for t in range(200):
    K = rng.standard_normal((n, d))
    V = rng.standard_normal((n, d))
    qv = rng.standard_normal(d)
    K[7] = 9.0 * np.sqrt(d) * qv / (qv @ qv) + 0.3 * K[7]       # the needle: logit ~ 9
    p_ref = None
    for name, tq in quants.items():
        Kh = K if tq is None else tq.dequantize(*tq.quantize(K))
        logits = Kh @ qv / np.sqrt(d)
        p = np.exp(logits - logits.max())
```

```
        p /= p.sum()
        if p_ref is None:
            p_ref, o_ref = p, p @ V
        o = p @ V
        stats[name].append((p[7], 0.5 * np.abs(p - p_ref).sum(),
                            np.linalg.norm(o - o_ref) / np.linalg.norm(o_ref)))
for name, rows in stats.items():
    m = np.mean(rows, axis=0)
    print(f"{name:18} {m[0]:9.3f} {m[1]:11.3f} {m[2]:14.3f}")
```

```
python tq_prod.py
```

Output:

```
inner products, 512 correlated (q, k) pairs
quantizer               slope resid std
MSE 2-bit               0.883    0.0331
MSE 1-bit + QJL         1.003    0.0620
MSE 3-bit               0.967    0.0231
MSE 2-bit + QJL         1.001    0.0355
MSE 4-bit               0.991    0.0125
MSE 3-bit + QJL         1.001    0.0200

attention, 3 bits per key coordinate, 200 trials
keys               p(needle) TV distance output rel err
fp32                   0.534       0.000          0.000
MSE 3-bit              0.473       0.081          0.123
MSE 2-bit + QJL        0.509       0.118          0.126
```

Part 1 shows the trade:

- MSE quantizers **shrink** inner products. The slope is 0.88 at 2 bits and 0.97 at 3 bits, the same radial shrinkage that `norm_correction` fixes. At 1 bit the shrinkage is exactly $2/\pi$.
- MSE + QJL has slope 1.00 at every width, but its residual noise is roughly 1.5 to 2 times larger at the same total bits, because QJL gets one sign bit per coordinate while the MSE stage loses a bit of resolution.

Part 2 is what the model feels. The unbiased keys get the needle's probability closer to the truth (0.509 against 0.534), but their noise lands on all 4095 other logits. The exponential in softmax turns zero-mean logit noise into a biased, heavier tail of spurious attention mass. Total-variation distance gets worse (0.118 against 0.081) and the output error is no better. A uniform shrinkage of every logit by the same factor is close to a temperature change, which softmax tolerates well. Independent noise per key is not.

This is not just a toy effect. vLLM's TurboQuant integration **omits QJL on purpose**. Its documentation says the omission follows "community consensus (5+ independent groups)" that QJL "hurts attention quality by amplifying variance through softmax". The RaBitQ authors' comparison

note (Section 10) cites the same observation. The paper's own KV-cache experiments use the outlier-split bit allocation from Section 7. When you implement this for attention, start with the MSE variant plus norm correction.

---

7. Fractional bits and the outlier split

The paper's "2.5-bit" and "3.5-bit" settings come from running **two TurboQuant instances per head** with different widths. For 2.5 bits, 32 outlier channels get 3 bits and the other 96 get 2: $(32 \cdot 3 + 96 \cdot 2)/128 = 2.5$. Rotation spreads outliers, but it does not make the extra bits worthless. Channels that are hot across the whole sequence are known ahead of time, and giving them more resolution is cheap. Each instance needs its own rotation over its own channel subset, and its own norm.

Values are a different problem. Keys meet a query in a dot product and then a softmax. Values are averaged with softmax weights, which already averages out independent errors. vLLM keeps keys on the rotate-and-Lloyd-Max path and stores **values with plain uniform 3- or 4-bit quantization** (scale and zero point per vector). Its `turboquant_k8v4` preset goes further: FP8 keys with no rotation, and 4-bit values.

---

8. Kernel view: never un-rotate a key

A naive integration dequantizes the cache back to bf16 and calls the normal attention kernel. That spends HBM bandwidth on the decompressed copy, which defeats the point. The algebra allows something much better. With orthogonal Π:

$$\langle q, \widehat{k} \rangle = \|k\| \, \langle q, \Pi^\top c_{\text{idx}} \rangle = \|k\| \, \langle \Pi q, c_{\text{idx}} \rangle.$$

Rotate the **query** once per head. Then each key's score is a table lookup (2^b centroids fit in registers) times the rotated query, times a norm. Values work the same way in reverse: accumulate $\sum_i p_i \|v_i\| \, c_{\text{idx}_i}$ in the rotated basis and apply $\Pi^\top$ once at the end.

```python
# file: tq_rotated_domain.py
# Decode-time attention never has to un-rotate a key: rotate the query once instead.
# Values: accumulate the weighted codewords in the rotated domain, un-rotate once at the end
import numpy as np

from tq_core import TurboQuantMSE

rng = np.random.default_rng(3)
n, d = 4096, 128
```

```
K, V, q = rng.standard_normal((n, d)), rng.standard_normal((n, d)), rng.standard_normal(d)
tqk, tqv = TurboQuantMSE(d, 3, rng), TurboQuantMSE(d, 3, rng)
k_idx, k_norm = tqk.quantize(K)
v_idx, v_norm = tqv.quantize(V)

# Reference: dequantize every key and value to full vectors (n inverse transforms each).
Kh, Vh = tqk.dequantize(k_idx, k_norm), tqv.dequantize(v_idx, v_norm)
s_ref = Kh @ q / np.sqrt(d)
p = np.exp(s_ref - s_ref.max())
p /= p.sum()
o_ref = p @ Vh

# Kernel form: one forward transform of q, table lookups per key, one inverse transform.
q_rot = tqk.rot.forward(q)                                  # once per (query, head)
s = k_norm[:, 0].astype(np.float64) * (tqk.centroids[k_idx] @ q_rot) / np.sqrt(d)
p2 = np.exp(s - s.max())
p2 /= p2.sum()
acc_rot = (p2 * v_norm[:, 0].astype(np.float64)) @ tqv.centroids[v_idx]   # stays rotated
o = tqv.rot.inverse(acc_rot)                                # once per (query, head)

print(f"max |logit diff|  {np.abs(s - s_ref).max():.2e}")
print(f"max |output diff| {np.abs(o - o_ref).max():.2e}")
print(f"transforms per head: reference {2 * n}, rotated-domain 2")
```

```
python tq_rotated_domain.py
```

Output:

```
max |logit diff|  1.55e-15
max |output diff| 1.53e-16
transforms per head: reference 8192, rotated-domain 2
```

The two paths agree to machine precision. vLLM's fused Triton decode kernel (`triton_turboquant_decode.py`) has this shape: it takes a precomputed `Q_rot` and scores packed 3- or 4-bit key indices against the centroid table inside the split-KV loop from the flash-decoding chapter. It does not rotate the value side, because its values are uniformly quantized.

The sign diagonal D is worth a look. vLLM's current backend uses a **plain Sylvester Hadamard**, shared across layers. A code comment argues that random sign flips do not improve a symmetric Lloyd-Max quantizer. That is true for sign flips applied *after* the transform: a sign change mirrors a coordinate onto the mirror-image centroid. Signs applied *before* H are a different thing. They are what turns "works on typical inputs" into "works on every input":

```
# file: tq_signs.py
# Plain Hadamard H x versus randomized Hadamard H D x in front of a 3-bit Lloyd-Max
# quantizer. On ordinary keys they tie. On inputs that line up with Walsh functions,
# the plain transform concentrates all the energy in one coordinate.
```

```
import numpy as np
from scipy.linalg import hadamard

from tq_core import fwht, lloyd_max_sphere

rng = np.random.default_rng(4)
d, bits = 128, 3
c, edges, model = lloyd_max_sphere(d, bits)
signs = rng.choice([-1.0, 1.0], size=d)


def rel_err(x, randomize):
    u = x / np.linalg.norm(x, axis=1, keepdims=True)
    y = fwht(u * signs) if randomize else fwht(u)
    yh = c[np.searchsorted(edges, y)]
    uh = fwht(yh) * signs if randomize else fwht(yh)
    return np.mean(np.sum((u - uh) ** 2, 1))


scale = np.ones(d)
scale[[3, 17, 64, 101]] = [18.0, 12.0, 9.0, 15.0]
ordinary = rng.standard_normal((4096, d)) * scale
H = hadamard(d).astype(float)
walshy = H[rng.integers(0, d, 4096)] * 3.0 + 0.3 * rng.standard_normal((4096, d))

print(f"3-bit relative error (model {model:.4f})")
print(f"{'input':28} {'H x':>8} {'H D x':>8}")
for name, x in (("Gaussian + outlier channels", ordinary), ("near Walsh rows", walshy)):
    print(f"{name:28} {rel_err(x, False):8.4f} {rel_err(x, True):8.4f}")
```

```
python tq_signs.py
```

Output:

```
3-bit relative error (model 0.0340)
input                            H x    H D x
Gaussian + outlier channels    0.0300   0.0300
near Walsh rows                0.6813   0.0309
```

On ordinary keys the two transforms tie, which is why dropping D is a reasonable engineering choice. On inputs that correlate with Walsh functions, the plain transform maps nearly all the energy into one coordinate, and a codebook designed for $\mathcal{N}(0, 1/d)$ clips it. A sign vector costs d bits per layer and one multiply per element. If you write your own kernel, keep it unless you have measured that your model's keys never look like this.

Kernel checklist for a fused TurboQuant decode path:

Step	Where it runs	Cost
Rotate new K (and V if rotated)	KV store kernel, once per token	$O(d \log d)$ butterfly, or a $d \times d$ GEMM on tensor cores
Normalize, binary-search midpoints, pack bits, write fp16 norm	same store kernel	$2^b - 1$ compares per element
Rotate query	once per (token, head) before attention	one small GEMM
Score: unpack index → centroid → FMA with q_{rot}, scale by norm	split-KV attention inner loop	register table lookup, no HBM traffic beyond the packed bytes
Values: uniform dequant (vLLM), or rotated-domain accumulate	same loop	scale/zero FMA
Inverse-rotate output (rotated values only)	once per (token, head)	one small GEMM

Packed slots have awkward sizes. A 128-dim 3-bit key is 48 bytes plus a 2-byte norm, so alignment and vector-load width have to be designed rather than assumed. vLLM pads slot sizes to even byte counts for this reason.

---

9. Where it runs today

As of October 2026:

- **vLLM** merged TurboQuant KV-cache support in PR #38479 (April 2026). It is selected with `--kv-cache-dtype` presets. The table is the vLLM documentation's own; the quality column is perplexity change versus an unquantized cache:

Preset	Keys	Values	Compression	PPL change
`turboquant_k8v4`	FP8	4-bit uniform	2.6x	+1.17%
`turboquant_4bit_nc`	4-bit Lloyd-Max + norm correction	4-bit uniform	3.8x	+2.71%
`turboquant_k3v4_nc`	3-bit Lloyd-Max + norm correction	4-bit uniform	~3.5x	+10.63%

Preset	Keys	Values	Compression	PPL change
`turboquant_3bit_nc`	3-bit Lloyd-Max + norm correction	3-bit uniform	4.9x	+20.59%

It has Triton and (on ROCm) FlyDSL decode kernels, CI evaluation jobs per configuration, and boundary-layer protection (a configurable set of boundary layers skips compression). A follow-up PR for grouped 3/4-bit modes (#39890) was closed without merging. Note how fast the PPL cost grows below 4 bits on these presets. That is the honest version of "near-lossless at 3.5 bits": it depends on outlier handling, the model, and the metric.

- **SGLang** has had several TurboQuant PRs (#21419, #21617, #22048, #23135, and a ROCm variant #21628). All were closed without merging, so there is no upstream support in SGLang 0.5.21.
- **Speed claims.** Google's blog reports up to 8x faster attention-logit computation for 4-bit TurboQuant keys against 32-bit unquantized keys on H100. That is a kernel micro-benchmark against an fp32 baseline, not end-to-end decode against a bf16 or FP8 cache. Expect the end-to-end gain to come mostly from capacity (bigger batches, longer contexts) and from fewer HBM bytes in bandwidth-bound decode.

---

10. Credit where it is due: RaBitQ and the older rotation literature

Rotate-then-quantize is older than TurboQuant, and the paper's treatment of prior work is disputed. State it plainly:

- **RaBitQ** (Gao and Long, SIGMOD 2024, arXiv May 2024) and **extended RaBitQ** (arXiv September 2024, SIGMOD 2025) apply a random rotation and then quantize, with unbiased inner-product estimators and an error bound shown to be asymptotically optimal. On 27 March 2026, after Google's blog post, RaBitQ's first author said publicly that TurboQuant misdescribes RaBitQ's method and guarantees, and that this had been raised with the authors before submission. He posted a public comment on the ICLR OpenReview page.
- The RaBitQ authors' technical note "Revisiting RaBitQ and TurboQuant" (arXiv:2604.19528) reruns both methods with their public code. On KV-cache tasks it reports clear RaBitQ gains at 2.5 bits and comparable results at 3.5 bits. For example, on needle-in-a-haystack with Llama-3.1-8B: full cache 0.987, RaBitQ 3.5-bit 0.977, TurboQuant-MSE 3.5-bit 0.962, RaBitQ 2.5-bit 0.951, TurboQuant-MSE 2.5-bit 0.709. It also reports that the paper's runtime comparison ran the RaBitQ baseline single-threaded on a CPU, and that the speed claims did not reproduce. These are one party's measurements. The code is public (`VectorDB-NTU/rabitq-turboquant-comparison`) so you can check them.
- vLLM's own module docstring credits **DRIVE** (NeurIPS 2021) and **EDEN** (ICML 2022) for the Hadamard-rotation-plus-scalar-quantization pattern. It notes the scheme is mathematically equivalent to the scalar case of **HIGGS** (NAACL 2025), and that "Cache Me If You

Must" (ICML 2025) applied it to KV caches first. In weight and activation quantization, QuaRot, QuIP# and SpinQuant use the same rotation idea.

For a kernel engineer the practical takeaway is that the algorithm is a well-understood family, not one paper's invention. Choose the variant by measured attention quality and kernel cost, and cite the lineage when you write it up.

---

11. Verification plan

Functional (CPU)

1. `tq_codebook.py` reproduces the distortion table. If 3-bit D is not about 0.034 at $d = 128$, the density or Lloyd iteration is wrong.
2. Round-trip on Gaussian vectors: relative error matches the model within 1%. Then on recorded keys from your model (dump one layer's K cache).
3. `tq_rotated_domain.py`-style equivalence between the fused rotated-domain path and dequantize-then-attend, to about 1e-6 in fp32.
4. Bit packing: random indices $\rightarrow$ pack $\rightarrow$ unpack is the identity for every slot size and head dimension you support (64, 96, 128, 256; non-power-of-two dims need a padded or block Hadamard).

Quality (GPU)

- Perplexity on a fixed corpus, needle-in-a-haystack at several depths and lengths, and a LongBench subset, against bf16 and FP8 caches. Report per-preset, as vLLM does.
- Attention-level diagnostics on real activations: logit RMSE, top-k overlap, and TV distance of attention distributions per layer. Layers that degrade most are candidates for boundary protection or more bits.

Performance (GPU)

- Decode throughput at fixed batch versus context length (where the cache dominates), and maximum batch at fixed memory.
- Profile the store kernel separately. A slow quantize path shows up as prefill time, not decode time.

Pitfalls

Pitfall	Symptom	Fix
Measuring key reconstruction MSE only	Per-channel or QJL variants look good, attention gets worse	Measure logits, attention distributions, task metrics
Using TurboQuant-prod (QJL) for keys by default	Noisy, diffuse attention, worse long-context recall	MSE variant + norm correction; evaluate QJL explicitly
Dequantizing the whole cache to bf16 before attention	No decode speedup, extra memory traffic	Rotate the query; score packed indices in the attention loop
Codebook built for the wrong d	Error well above the model	Codebook is per head dimension; regenerate for 64/96/256
Different rotations for store and query	Garbage logits	One rotation per layer (or shared), versioned with the cache format
Dropping the sign diagonal without measuring	Rare catastrophic vectors	Keep D, or verify on recorded keys
Non-power-of-two head dim	Hadamard undefined	Pad, or use a block-diagonal Hadamard on power-of-two blocks
Comparing to an fp32 baseline	Inflated speedups	Compare to bf16 and FP8 caches end to end

---

Checkpoint

- Explain why a random rotation makes a single fixed codebook near-optimal for every vector.
- Derive the 1-bit centroid $\pm\sqrt{2/\pi}/\sqrt{d}$ from the Gaussian approximation.
- State why per-channel quantization can win on key MSE but lose on attention logits.
- Explain why an unbiased inner-product estimator can make softmax attention worse.
- Write the identity that lets a decode kernel avoid inverse-rotating keys.
- Describe what the sign diagonal in HD protects against.

Try this

1. Add the paper's outlier split to `tq_vs_naive.py`: choose the 32 channels with the largest mean $|k|$ and give them 3 bits, the rest 2. Compare 2.5-bit split against uniform 3-bit TurboQuant-MSE on logit RMSE.
2. Apply `norm_correction` to the 2-bit quantizer in `tq_prod.py` and measure the slope again. How close to 1 does it get without QJL?

3. Replace the synthetic keys with real ones: run a small model on CPU with Hugging Face `transformers`, capture `past_key_values` for one layer, and rerun `tq_vs_naive.py` and `tq_signs.py`.
4. Make the attention test harder: put 4 needles at logit gaps of 3, 5, 7 and 9 and record which quantizer loses the weakest needle first.
5. Write the score loop of section 8 in Triton (unpack 3-bit indices, gather centroids, dot with `q_rot`) and check it with `TRITON_INTERPRET=1` against `tq_rotated_domain.py`.

Summary

TurboQuant quantizes each KV vector by normalizing it, rotating it with a random orthogonal transform (a randomized Hadamard in practice), and rounding every coordinate with one Lloyd-Max codebook computed offline for the coordinate distribution of a random unit vector. The rotation removes outlier channels, and the fixed codebook removes per-block scales and calibration. The paper's distortion numbers reproduce in a few lines of NumPy. The unbiased QJL variant is the wrong default inside softmax: shrinkage behaves like a temperature change, while per-key noise spreads attention mass, and production code (vLLM) omits QJL for that reason. A good decode kernel never decompresses the cache. It rotates the query and scores packed indices against a register-resident centroid table. vLLM ships 3- and 4-bit presets whose perplexity cost rises quickly below 4 bits, and SGLang has no merged implementation. The rotate-then-quantize idea comes from a lineage (DRIVE, EDEN, RaBitQ, HIGGS) whose credit is contested, so measure the variants yourself.

Sources (for further reading)

- Zandieh, Daliri, Hadian, Mirrokni, "TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate", arXiv:2504.19874 (ICLR 2026). Distortion table, centroids, bounds, outlier split, KV-cache results.
- Google Research blog, "TurboQuant: Redefining AI efficiency with extreme compression" (24 March 2026), https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/. Source of the 6x and 8x claims. Also covers PolarQuant (AISTATS 2026).
- Zandieh, Daliri, Han, "QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead" (AAAI 2025, arXiv:2406.03482).
- vLLM documentation, `vllm.model_executor.layers.quantization.turboquant` (presets, QJL omission, lineage note), and source: `vllm/v1/attention/backends/turboquant_attn.py`, `vllm/v1/attention/ops/triton_turboquant_decode.py`, `triton_turboquant_store.py`. PR vllm-project/vllm#38479.
- SGLang PRs sgl-project/sglang#21419, #21617, #21628, #22048, #23135 (all closed unmerged).

- Gao, Long, "RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search" (SIGMOD 2024, arXiv:2405.12497). Gao et al., extended RaBitQ (arXiv:2409.09913, SIGMOD 2025).
- Gao et al., "Revisiting RaBitQ and TurboQuant: A Symmetric Comparison of Methods, Theory, and Experiments" (arXiv:2604.19528), code at https://github.com/VectorDB-NTU/rabitq-turboquant-comparison. ICLR 2026 OpenReview forum for TurboQuant (public comments).
- Vargaftik et al., DRIVE (NeurIPS 2021) and EDEN (ICML 2022). Malinovskii et al., HIGGS (NAACL 2025, arXiv:2411.17525). Shutova et al., "Cache Me If You Must" (ICML 2025, arXiv:2501.19392).
- Liu et al., KIVI (ICML 2024). Hooper et al., KVQuant (NeurIPS 2024). Per-channel key outliers.
- Ashkboos et al., QuaRot (NeurIPS 2024). Tseng et al., QuIP# (ICML 2024). Liu et al., SpinQuant (ICLR 2025).

Inspiration (not a source): Google Research's post on X announcing TurboQuant, https://x.com/GoogleResearch/status/2036533564158910740, and the public RaBitQ discussion that followed. Facts above are taken from the papers, documentation, and code listed, not from the posts.

Mixture-of-Experts Kernels: Routing, Permutation, and Grouped GEMM

Mixture-of-Experts Kernels: Routing, Permutation, and Grouped GEMM

Most frontier open-weight models are now **Mixture-of-Experts (MoE)** models. DeepSeek-V3 and Kimi K2 have 256 and 384 routed experts per layer, Qwen3-235B-A22B and gpt-oss-120b have 128, and every token uses only 4 to 8 of them. The model holds hundreds of billions of parameters, but each token pays for only a few tens of billions. For a kernel engineer, that changes the central operation. Instead of one large GEMM per feed-forward layer, each layer runs **dozens to hundreds of small GEMMs of different, data-dependent sizes**, and the sizes are only known after the router has run on the GPU.

This chapter builds the MoE layer kernel by kernel. It writes a ground-truth reference, gets the routing semantics right (including the DeepSeek-style bias that is easy to misuse), builds the block-aligned indirection table that production kernels consume, writes a fused grouped GEMM in Triton that gathers and scatters rows itself, verifies it on a CPU with the Triton interpreter, compiles it for a Hopper GPU you do not own, and then uses shape arithmetic to explain why prefill and decode need different MoE kernels.

---

1. Why MoE is a kernel problem

A dense SwiGLU feed-forward layer computes $W_2\,(\text{silu}(W_1 x) \odot W_3 x)$ with one weight set. An MoE layer has E such weight sets (the **experts**) plus a small **router** $W_g \in \mathbb{R}^{E \times H}$. For each token, the router scores all experts, keeps the top k, and the layer output is the weighted sum of those k experts' outputs.

Model (config.json)	Routed experts E	Active k	Hidden H	Expert intermediate I	Router
Mixtral-8x7B	8	2	4096	14336	softmax, top-2
gpt-oss-120b	128	4	2880	2880	softmax over top-4
Qwen3-235B-A22B	128	8	4096	1536	softmax, renor-mal-ized top-8

Model (config.json)	Routed experts E	Active k	Hidden H	Expert intermediate I	Router
DeepSeek-V3	256 (+1 shared)	8	7168	2048	sigmoid + bias, 8 groups, top-4 groups
Kimi K2	384 (+1 shared)	8	7168	2048	sigmoid + bias

Three properties make this hard to run fast:

1. **Ragged, data-dependent sizes.** Expert e receives n_e token rows, with $\sum_e n_e = T \cdot k$ for a batch of T tokens. The n_e change every step, live in GPU memory, and are often zero.
2. **Weights dominate bytes.** One DeepSeek-V3 expert is $3HI \approx 44$ M parameters. Every active expert must stream its whole weight set from HBM, whether it serves one token or a thousand.
3. **Imbalance.** Routers are trained to balance load *on average*. Any given batch can still send most tokens to a few hot experts, and the slowest expert sets the latency of the layer.

The rest of the chapter is about turning "E ragged GEMMs" into **one launch** that keeps the tensor cores busy when there is enough work, and keeps HBM busy when there is not.

---

2. Mental model: route, align, grouped GEMM, combine

```
x [T, H]  router GEMM  logits [T, E]  top-k  topk_ids, topk_w  [T, k]

                  moe_align: histogram  pad each expert to BLOCK_M  exclusive scan  sca

                sorted_ids  [padded rows]  (flat row r = token*k + slot, or a sentinel)
                expert_ids  [padded rows / BLOCK_M]  (one expert per BLOCK_M-row tile)

GEMM1:  rows of x gathered by sorted_ids // k,  B = W13[expert]   h  [T*k, 2I]  at row r
act  :  silu(h[:, :I]) * h[:, I:]                                  a  [T*k, I]
GEMM2:  rows of a gathered by sorted_ids,       B = W2[expert], × topk_w[r]   y [T*k, H] at
combine: out[t] = Σ_slot y[t*k + slot]                             out [T, H]
```

There are two ways to feed the expert GEMMs:

Design	Data movement	Used by
Permute, then grouped GEMM. Physically reorder token rows so each expert's rows are contiguous, run a grouped GEMM with per-expert offsets, then un-permute.	Writes and reads a permuted copy of the activations (and of the outputs)	`torch.nn.functional.grouped_mm`, CUTLASS grouped GEMM, DeepGEMM's grouped layouts, most expert-parallel stacks (the all-to-all already permutes)
Gather and scatter inside the GEMM. Keep activations in token order; each tile reads its rows through an index table and writes results back to their original positions.	No permuted copy; indexed loads instead of contiguous ones	vLLM and SGLang `fused_moe` Triton kernels

Both need the same metadata: how many rows each expert has, and where each expert's rows start. Computing that is a **histogram followed by a prefix sum (scan)**, the same primitive that underlies radix sort.

---

3. Ground truth first

Every faster version in this chapter is compared against two deliberately simple implementations: a per-expert loop, and a dense version that runs every expert on every token and masks the result. They share no indexing code, so a bug in one does not hide in the other.

```
# file: moe_reference.py
# Ground truth for a top-k Mixture-of-Experts FFN layer (SwiGLU experts), in PyTorch on CPU.
#   route:   logits = x @ Wg^T -> softmax -> top-k -> renormalise the k weights
#   experts: y_e(x) = W2_e (silu(W1_e x) * (W3_e x))
#   combine: out(x) = sum_j w_j * y_{e_j}(x)
# Two independent implementations must agree: a per-expert loop and a dense "run every exper
import torch

torch.manual_seed(0)
T, H, I, E, TOP_K = 64, 64, 96, 8, 2          # tokens, hidden, expert intermediate, experts


def route(x, w_gate, top_k, renormalize=True):
    probs = torch.softmax(x @ w_gate.T, dim=-1)               # [T, E], fp32
```

```
    w, ids = torch.topk(probs, top_k, dim=-1)                # [T, k] weights and expert id
    if renormalize:
        w = w / w.sum(-1, keepdim=True)
    return w, ids


def moe_loop(x, w_gate, w1, w3, w2, top_k):
    """Per-expert loop: gather the tokens routed to e, run e once, scatter-add weighted outp
    w, ids = route(x, w_gate, top_k)
    out = torch.zeros_like(x)
    for e in range(w1.shape[0]):
        tok, slot = torch.nonzero(ids == e, as_tuple=True)    # which tokens chose e, and in
        if tok.numel() == 0:
            continue                                          # an expert may receive no tok
        xe = x[tok]
        ye = (torch.nn.functional.silu(xe @ w1[e].T) * (xe @ w3[e].T)) @ w2[e].T
        out.index_add_(0, tok, ye * w[tok, slot, None])
    return out, ids


def moe_dense(x, w_gate, w1, w3, w2, top_k):
    """Run every expert on every token, then keep only the routed ones. E/k times the FLOPs,
    w, ids = route(x, w_gate, top_k)
    gate = torch.zeros(x.shape[0], w1.shape[0]).scatter(1, ids, w)        # [T, E], k nonzer
    y_all = torch.stack([(torch.nn.functional.silu(x @ w1[e].T) * (x @ w3[e].T)) @ w2[e].T
                         for e in range(w1.shape[0])], dim=1)            # [T, E, H]
    return (gate[:, :, None] * y_all).sum(1)


def make_layer(H, I, E, seed=0):
    g = torch.Generator().manual_seed(seed)
    s = 1.0 / H ** 0.5
    return (torch.randn(E, H, generator=g) * s,               # router weight  [E, H]
            torch.randn(E, I, H, generator=g) * s,            # W1 (gate proj) [E, I, H]  nn
            torch.randn(E, I, H, generator=g) * s,            # W3 (up proj)   [E, I, H]
            torch.randn(E, H, I, generator=g) / I ** 0.5)     # W2 (down proj) [E, H, I]


if __name__ == "__main__":
    x = torch.randn(T, H)
    wg, w1, w3, w2 = make_layer(H, I, E)
    out_loop, ids = moe_loop(x, wg, w1, w3, w2, TOP_K)
    out_dense = moe_dense(x, wg, w1, w3, w2, TOP_K)
    counts = torch.bincount(ids.flatten(), minlength=E)
    print("tokens per expert :", counts.tolist(), " (sum =", int(counts.sum()), "= T*k)")
    print("max |loop - dense|:", f"{(out_loop - out_dense).abs().max().item():.2e}")
    flops_moe = 2 * T * TOP_K * 3 * H * I
```

```
    print(f"expert FLOPs: routed {flops_moe/1e6:.1f} MFLOP vs dense {flops_moe*E/TOP_K/1e6:.
```

```
python3 -m venv .venv && . .venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install triton numpy
python moe_reference.py
```

Output (PyTorch 2.14.1 CPU):

```
tokens per expert : [19, 11, 15, 15, 21, 15, 18, 14]  (sum = 128 = T*k)
max |loop - dense|: 0.00e+00
expert FLOPs: routed 4.7 MFLOP vs dense 18.9 MFLOP
```

The two implementations agree to the last bit here because they run the same row-wise products on the same CPU kernels. The FLOP line is the reason MoE exists: with $k/E = 2/8$ the routed layer does a quarter of the dense work. DeepSeek-V3's 8/256 is 1/32.

Note the indexing contract that every later kernel keeps: the $T \cdot k$ routed copies of the tokens are numbered by a **flat row** $r = t \cdot k + j$ (token t, slot j). Expert loads are just `bincount` over the flattened `topk_ids`.

---

4. Routing semantics: the bias is for choosing, not for weighting

Softmax-then-top-k (Mixtral, Qwen3) is simple. DeepSeek-V3 and Kimi K2 use a router that kernel ports regularly get wrong:

- scores are **sigmoid** affinities, not a softmax;
- a per-expert **correction bias**, adjusted during training to balance load without an auxiliary loss, is added to the scores **only to select experts**;
- **group-limited routing:** experts are split into groups (8 groups for DeepSeek-V3). Each group is scored by the sum of its top-2 biased scores, only the best `topk_group` groups (4) stay eligible, and top-k is taken inside them. In deployment this bounds how many nodes a token's experts span;
- the gate weights come from the **unbiased** sigmoid scores, are renormalized over the k chosen experts, and are multiplied by `routed_scaling_factor` (2.5 for DeepSeek-V3).

```
# file: router_grouped.py
# DeepSeek-V3-style routing: sigmoid scores, a per-expert correction bias that steers SELECT
# (auxiliary-loss-free load balancing), group-limited top-k, renormalisation and a fixed sca
import torch

torch.manual_seed(0)
T, E, K, N_GROUP, TOPK_GROUP, SCALE = 4, 16, 4, 4, 2, 2.5   # small stand-ins for 256 / 8 /
```

```
def route_grouped(logits, bias, use_bias_for_weights=False):
    scores = torch.sigmoid(logits)                                          # [T, E] affinities in
    choice = scores + bias                                                  # biased scores: selec
    grp = choice.view(T, N_GROUP, -1).topk(2, dim=-1).values.sum(-1)  # group score = sum of
    keep_grp = grp.topk(TOPK_GROUP, dim=-1).indices                         # keep the best TOPK_G
    mask = torch.zeros(T, N_GROUP).scatter(1, keep_grp, 1.0).repeat_interleave(E // N_GROUP,
    ids = choice.masked_fill(mask == 0, float("-inf")).topk(K, dim=-1).indices
    w = (choice if use_bias_for_weights else scores).gather(1, ids)   # weights from UNBIASE
    return ids, SCALE * w / w.sum(-1, keepdim=True)


logits = torch.randn(T, E)
bias = torch.zeros(E)
bias[torch.arange(E) % 4 == 0] = 0.5          # the balancer has been boosting under-used ex

ids0, w0 = route_grouped(logits, torch.zeros(E))
ids1, w1 = route_grouped(logits, bias)
ids2, w2 = route_grouped(logits, bias, use_bias_for_weights=True)
print("no bias      ids:", ids0.tolist())
print("with bias    ids:", ids1.tolist())
print("groups used     :", (ids1 // (E // N_GROUP)).unique(dim=None).tolist(), "of", N_GROUP
      "| per token:", [sorted(set(r)) for r in (ids1 // (E // N_GROUP)).tolist()])
print("weights (right) :", [[round(v, 3) for v in r] for r in w1.tolist()])
print("weights (bug)   :", [[round(v, 3) for v in r] for r in w2.tolist()])
print("max weight error from leaking the bias:", f"{(w1 - w2).abs().max().item():.3f}")
```

```
python router_grouped.py
```

Output:

```
no bias      ids: [[13, 14, 4, 5], [7, 13, 12, 14], [13, 6, 12, 7], [11, 10, 2, 1]]
with bias    ids: [[4, 12, 13, 14], [12, 4, 7, 13], [12, 4, 13, 6], [11, 8, 0, 10]]
groups used     : [0, 1, 2, 3] of 4 | per token: [[1, 3], [1, 3], [1, 3], [0, 2]]
weights (right) : [[0.635, 0.48, 0.702, 0.683], [0.637, 0.514, 0.688, 0.661], [0.561, 0.543,
weights (bug)   : [[0.798, 0.685, 0.516, 0.501], [0.785, 0.692, 0.522, 0.501], [0.742, 0.729
max weight error from leaking the bias: 0.273
```

Two things to read off it. First, the bias changes **which** experts are chosen. The boosted experts 0, 4, 8, and 12 move into the top-k, and each token uses exactly two of the four groups. Second, computing the gate weights from the biased scores changes the layer output by up to 0.27 in weight per expert. Nothing crashes and every shape is right; the model just gets measurably worse. Fused router kernels (vLLM's `grouped_topk`, SGLang's biased group top-k) exist to do this whole block in one launch, and their unit tests check exactly this distinction.

---

5. The indirection table: histogram, pad, scan, scatter

A tiled GEMM wants every `BLOCK_M`-row tile to use **one** expert's weights. So each expert's run of rows is padded up to a multiple of `BLOCK_M` with a sentinel row index, and a second array records the expert that owns each tile. This is the contract of vLLM's `moe_align_block_size`:

- `sorted_ids[p]`: the flat row r at padded position p, or the sentinel $T \cdot k$ for padding;
- `expert_ids[p // BLOCK_M]`: the expert for that tile;
- the padded total, so that tiles past the end can exit at once.

```
# file: moe_align.py
# Build the block-aligned indirection table a fused MoE GEMM consumes (same contract as vLLM
# moe_align_block_size): sort the T*k (token, slot) pairs by expert, pad each expert's run t
# multiple of BLOCK_M with a sentinel, and record which expert owns each BLOCK_M-row tile.
import torch


def moe_align(topk_ids, block_m, num_experts):
    flat = topk_ids.flatten().to(torch.int64)                  # flat row r = token * k + slo
    n = flat.numel()
    counts = torch.bincount(flat, minlength=num_experts)       # 1. histogram
    padded = (counts + block_m - 1) // block_m * block_m       # 2. round each run up to BLOC
    starts = torch.cumsum(padded, 0) - padded                  # 3. exclusive scan -> run sta
    total = int(padded.sum())

    sorted_ids = torch.full((total,), n, dtype=torch.int32)    # sentinel n = "padding row"
    order = torch.argsort(flat, stable=True)                   # rows grouped by expert, stab
    rank = torch.arange(n) - (torch.cumsum(counts, 0) - counts)[flat[order]]
    sorted_ids[starts[flat[order]] + rank] = order.to(torch.int32)   # 4. scatter into the r

    expert_ids = torch.repeat_interleave(torch.arange(num_experts, dtype=torch.int32),
                                         padded // block_m)    # one entry per BLOCK_M tile
    return sorted_ids, expert_ids, total


if __name__ == "__main__":
    # The worked example from vLLM's moe_align_block_size docstring.
    topk_ids = torch.tensor([[2, 3, 4], [1, 2, 4], [1, 3, 4], [1, 2, 3]])
    sorted_ids, expert_ids, total = moe_align(topk_ids, block_m=4, num_experts=5)
    print("flat expert of each row :", topk_ids.flatten().tolist())
    print("sorted_ids  :", sorted_ids.tolist())
    print("expert_ids  :", expert_ids.tolist())
    print("rows after padding:", total, " useful:", topk_ids.numel(),
          f" waste: {1 - topk_ids.numel() / total:.0%}")
```

```
python moe_align.py
```

Output:

```
flat expert of each row : [2, 3, 4, 1, 2, 4, 1, 3, 4, 1, 2, 3]
sorted_ids  : [3, 6, 9, 12, 0, 4, 10, 12, 1, 7, 11, 12, 2, 5, 8, 12]
expert_ids  : [1, 2, 3, 4]
rows after padding: 16  useful: 12  waste: 25%
```

This reproduces the worked example in vLLM's `moe_align_block_size` docstring exactly. Expert 0 receives no rows, so it gets no tile and its weights are never read. Experts 1 to 4 have 3 rows each, padded to 4, and the sentinel `12` marks the padding.

On a GPU, the four numbered steps are one small kernel. The histogram uses shared-memory atomics, the scan over E counts runs inside one CTA (with $E \leq 384$ it fits easily), and each row is scattered to `start[e] + rank`. vLLM and SGLang ship CUDA versions of this kernel because, at small batch sizes, its cost is not negligible next to the GEMMs. The `stable=True` sort is a choice made here, not a requirement. Keeping rows in token order within an expert makes the gathered loads more local and the output deterministic.

---

6. A fused grouped GEMM in Triton

The kernel is an ordinary tiled GEMM with three changes:

1. **Tile to expert.** Program `pid_m` reads `expert_ids[pid_m]` and offsets the weight pointer by `expert * stride_be`.
2. **Gathered A rows.** Row indices come from `sorted_ids`. For GEMM1, the input has one row per *token*, shared by its k slots, so the kernel loads row `r // TOP_K`. For GEMM2, the input already has one row per flat row, so `TOP_K=1`.
3. **Scattered C rows, with the router weight folded in.** Results are stored at row `r`, so the output comes out in token order. GEMM2 multiplies by `topk_w[r]` in the epilogue, which leaves the combine as a plain sum over k.

Padding rows (`r >= num_rows`) are masked on load and on store. They cost tensor-core work but no memory traffic.

```
# file: fused_moe_triton.py
# A fused-MoE grouped GEMM in Triton: one launch computes every expert's GEMM by reading a
# block-aligned indirection table (moe_align.py). Each program owns BLOCK_M rows that all be
# ONE expert, gathers their token rows from A, multiplies by that expert's weight tile, and
# the result to the rows' original (token, slot) positions. No permuted copy of A is ever wr
# Runs on a GPU, or on CPU for correctness work with:  TRITON_INTERPRET=1 python fused_moe_t
import os

import torch
import torch.nn.functional as F
import triton
```

```
import triton.language as tl

from moe_align import moe_align
from moe_reference import make_layer, moe_loop, route


@triton.jit
def moe_gemm_kernel(A, B, C, SORTED_IDS, EXPERT_IDS, TOPK_W,
                    num_rows, padded_rows, N, K,
                    stride_am, stride_be, stride_bn, stride_cm,
                    TOP_K: tl.constexpr, MUL_ROUTED_WEIGHT: tl.constexpr,
                    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    if pid_m * BLOCK_M >= padded_rows:            # grid is sized for the worst case; traili
        return

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    rows = tl.load(SORTED_IDS + offs_m)           # flat (token*TOP_K + slot), or the sentin
    row_ok = rows < num_rows
    a_rows = rows // TOP_K                        # GEMM1: A holds one row per token, shared
    expert = tl.load(EXPERT_IDS + pid_m)          # every row of this tile uses the same exp

    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)
    a_ptrs = A + a_rows[:, None] * stride_am + offs_k[None, :]
    b_ptrs = B + expert * stride_be + offs_n[None, :] * stride_bn + offs_k[:, None]   # B[e]

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)
    for k0 in range(0, K, BLOCK_K):
        k_ok = (k0 + offs_k) < K
        a = tl.load(a_ptrs, mask=row_ok[:, None] & k_ok[None, :], other=0.0)
        b = tl.load(b_ptrs, mask=k_ok[:, None] & (offs_n[None, :] < N), other=0.0)
        acc += tl.dot(a, b)
        a_ptrs += BLOCK_K
        b_ptrs += BLOCK_K

    if MUL_ROUTED_WEIGHT:                         # fold the router weight into the down pro
        w = tl.load(TOPK_W + rows, mask=row_ok, other=0.0)
        acc = acc * w[:, None]

    c_ptrs = C + rows[:, None] * stride_cm + offs_n[None, :]
    tl.store(c_ptrs, acc.to(C.dtype.element_ty), mask=row_ok[:, None] & (offs_n[None, :] < N


def moe_gemm(a, b, out, sorted_ids, expert_ids, padded, topk_w, top_k, mul_w,
             block_m, block_n=32, block_k=32):
```

```
    """a: [rows_a, K]; b: [E, N, K] (nn.Linear layout); out: [T*k, N] written at the flat ro
    E, N, K = b.shape
    grid = (expert_ids.numel(), triton.cdiv(N, block_n))
    moe_gemm_kernel[grid](a, b, out, sorted_ids, expert_ids, topk_w,
                          out.shape[0], padded, N, K,
                          a.stride(0), b.stride(0), b.stride(1), out.stride(0),
                          TOP_K=top_k, MUL_ROUTED_WEIGHT=mul_w,
                          BLOCK_M=block_m, BLOCK_N=block_n, BLOCK_K=block_k)


def fused_moe(x, w_gate, w1, w3, w2, top_k, block_m=16):
    T, H = x.shape
    E, I, _ = w1.shape
    topk_w, topk_ids = route(x, w_gate, top_k)
    sorted_ids, expert_ids, padded = moe_align(topk_ids, block_m, E)
    sorted_ids, expert_ids = sorted_ids.to(x.device), expert_ids.to(x.device)

    w13 = torch.cat([w1, w3], dim=1).contiguous()                  # [E, 2I, H]: gate and u
    h = torch.empty(T * top_k, 2 * I, device=x.device, dtype=x.dtype)
    moe_gemm(x, w13, h, sorted_ids, expert_ids, padded, topk_w, top_k, False, block_m)
    act = F.silu(h[:, :I]) * h[:, I:]                              # [T*k, I]  (fused in pr

    y = torch.empty(T * top_k, H, device=x.device, dtype=x.dtype)
    moe_gemm(act, w2, y, sorted_ids, expert_ids, padded, topk_w.flatten().contiguous(), 1, T
    return y.view(T, top_k, H).sum(dim=1), padded                  # combine the k weighted


def grouped_mm_moe(x, w_gate, w1, w3, w2, top_k):
    """Third implementation: explicit permute -> torch grouped GEMM with offsets -> un-permu
    T, H = x.shape
    E = w1.shape[0]
    topk_w, topk_ids = route(x, w_gate, top_k)
    flat = topk_ids.flatten()
    order = torch.argsort(flat, stable=True)                       # rows grouped by expert
    xs = x[order // top_k]                                         # the permuted copy fuse
    offs = torch.cumsum(torch.bincount(flat, minlength=E), 0).to(torch.int32)
    h = F.silu(F.grouped_mm(xs, w1.transpose(-2, -1), offs=offs)) * \
        F.grouped_mm(xs, w3.transpose(-2, -1), offs=offs)
    ys = F.grouped_mm(h, w2.transpose(-2, -1), offs=offs) * topk_w.flatten()[order, None]
    out = torch.zeros_like(x)
    out.index_add_(0, order // top_k, ys)                          # un-permute + combine
    return out


if __name__ == "__main__":
    device = "cpu" if os.environ.get("TRITON_INTERPRET") == "1" else "cuda"
    H, I = 64, 96
```

```
    cases = [  # name, tokens, experts, top_k, router bias toward expert 0
        ("uniform E=8 k=2", 64, 8, 2, 0.0),
        ("skewed  E=16 k=4", 50, 16, 4, 3.0),
        ("decode  T=1 k=2", 1, 8, 2, 0.0),
        ("top-1   E=8 k=1", 37, 8, 1, 0.0),
    ]
    for name, T, E, k, bias in cases:
        torch.manual_seed(1)
        wg, w1, w3, w2 = make_layer(H, I, E, seed=2)
        u = torch.nn.functional.normalize(torch.randn(H), dim=0)
        wg[0] += bias * u                                   # make expert 0 popular
        x = torch.randn(T, H) + 1.5 * u                     # its logit rises by ~1
        ref, ids = moe_loop(x, wg, w1, w3, w2, k)
        got, padded = fused_moe(*(t.to(device) for t in (x, wg, w1, w3, w2)), k)
        gmm = grouped_mm_moe(x, wg, w1, w3, w2, k)
        counts = torch.bincount(ids.flatten(), minlength=E)
        print(f"{name}: loads={counts.tolist()}  rows {T*k} -> padded {padded}  "
              f"|fused-ref|={(got.cpu() - ref).abs().max().item():.1e}  "
              f"|grouped_mm-ref|={(gmm - ref).abs().max().item():.1e}")
```

```
TRITON_INTERPRET=1 python fused_moe_triton.py
```

Output (Triton 3.8.0, PyTorch 2.14.1 CPU, interpreter mode):

```
uniform E=8 k=2: loads=[18, 8, 24, 21, 15, 12, 21, 9]  rows 128 -> padded 192  |fused-ref|=1
skewed  E=16 k=4: loads=[43, 5, 20, 11, 7, 7, 12, 10, 8, 15, 7, 5, 14, 20, 7, 9]  rows 200 -
decode  T=1 k=2: loads=[1, 0, 0, 0, 0, 1, 0, 0]  rows 2 -> padded 32  |fused-ref|=3.0e-07  |
top-1   E=8 k=1: loads=[5, 1, 11, 7, 4, 3, 6, 0]  rows 37 -> padded 112  |fused-ref|=9.5e-07
```

What the test matrix covers:

Case	Why
Uniform, $E = 8$, $k = 2$	The common case; every expert has several tiles' worth of rows or a partial tile
Skewed, $E = 16$, $k = 4$	Expert 0 takes 43 of 50 tokens. Multi-tile experts next to experts with 5 rows
Decode, $T = 1$	Six experts have zero rows, so they get no tiles. 2 useful rows become 32 padded rows
Top-1, $T = 37$	`TOP_K=1` in GEMM1 (no row sharing). Expert 7 is empty. T is not a multiple of anything
Three implementations	Loop, fused Triton kernel, and `F.grouped_mm` with explicit permute and un-permute

The fused kernel matches the loop to about 10^{-6} in FP32: the same sums, accumulated in a different

order. `F.grouped_mm` is PyTorch's public grouped-GEMM API. Its documentation targets BF16 on SM80+ CUDA GPUs. The CPU build used here also accepted FP32 inputs, which makes it a useful third opinion while developing.

The interpreter executes tile semantics with NumPy. It proves the **indexing** is right, which is where MoE kernels break. It says nothing about speed.

---

7. Compile for Hopper, and what the compiler tells you about decode

```
# file: compile_check.py
# Ahead-of-time compile the fused-MoE GEMM for an H100 (sm_90) on a machine with no GPU,
# once per specialisation the layer launches (GEMM1: TOP_K=k, no weight; GEMM2: TOP_K=1, wei
import triton
from triton.backends.compiler import GPUTarget
from triton.compiler import ASTSource

from fused_moe_triton import moe_gemm_kernel

target = GPUTarget("cuda", 90, 32)
sig = {"A": "*bf16", "B": "*bf16", "C": "*bf16", "SORTED_IDS": "*i32", "EXPERT_IDS": "*i32",
       "TOPK_W": "*fp32", "num_rows": "i32", "padded_rows": "i32", "N": "i32", "K": "i32",
       "stride_am": "i32", "stride_be": "i32", "stride_bn": "i32", "stride_cm": "i32",
       "TOP_K": "constexpr", "MUL_ROUTED_WEIGHT": "constexpr",
       "BLOCK_M": "constexpr", "BLOCK_N": "constexpr", "BLOCK_K": "constexpr"}
for name, top_k, mul_w, bm in [("gemm1 (gate+up)", 8, False, 64), ("gemm2 (down)   ", 1, Tru
                               ("gemm1 decode   ", 8, False, 16)]:
    src = ASTSource(fn=moe_gemm_kernel, signature=sig,
                    constexprs={"TOP_K": top_k, "MUL_ROUTED_WEIGHT": mul_w,
                                "BLOCK_M": bm, "BLOCK_N": 128, "BLOCK_K": 64})
    k = triton.compile(src, target=target)
    ptx = k.asm["ptx"]
    print(f"{name} BLOCK_M={bm:<3} num_warps={k.metadata.num_warps} shared={k.metadata.share
          f"mma={'wgmma' if 'wgmma' in ptx else ('mma.sync' if 'mma.sync' in ptx else 'none'
          f"ptx_lines={ptx.count(chr(10))}")
```

```
python compile_check.py
```

Output (Triton 3.8.0 on a CPU-only Linux box):

```
gemm1 (gate+up) BLOCK_M=64  num_warps=4 shared= 24576 B  mma=wgmma  ptx_lines=3937
gemm2 (down)    BLOCK_M=64  num_warps=4 shared= 24576 B  mma=wgmma  ptx_lines=3936
gemm1 decode    BLOCK_M=16  num_warps=4 shared= 18432 B  mma=mma.sync  ptx_lines=2294
```

All three specializations lower to `sm_90` code. The last column is the useful part. With `BLOCK_M=64`, Triton emits Hopper **`wgmma`** warpgroup MMAs. With `BLOCK_M=16`, it falls back to Ampere-style **`mma.sync`**, because `wgmma` works on tiles that are 64 rows tall per warpgroup. That is the structural problem of MoE decode in one line:

- At decode, an active expert typically has **1 to 4 rows**. A 64-row tile is more than 90% padding (section 8), and a 16-row tile cannot use the Hopper MMA path at all.
- At those sizes the layer is bandwidth-bound anyway, so wasted MMA rows cost little. What matters is that each active expert's weights are streamed **once**, at full HBM bandwidth, by enough CTAs to cover the SMs.

That is why production stacks keep separate MoE kernels or configurations for small batches. Common approaches are tuned configs with small `BLOCK_M` (vLLM keys its tuned Triton configs by E, N, and GPU), a per-token path (vLLM skips alignment when $T \cdot k \cdot 4 \le E$ and gives each routed row its own tile), **swap-AB** schedules that put the expert's large N dimension on the MMA's M side, and fixed-capacity "masked" layouts that keep shapes static for CUDA Graphs. The ahead-of-time compile is cheap enough to run in the agent loop for every candidate tile shape. Grep the PTX for the instruction you expect before paying for GPU time.

---

8. Shapes decide everything: padding, stragglers, and the roofline

One active expert in FP8 holds $3HI$ bytes of weights and does $2 \cdot n_e \cdot 3HI$ FLOPs for n_e rows. Its arithmetic intensity against weight bytes is simply

$$\text{FLOP per weight byte} \approx 2\,n_e \quad \text{(FP8 weights)}, \qquad \approx n_e \quad \text{(BF16 weights)}.$$

An H100 SXM's ridge point is roughly 1979 TFLOP/s/3.35 TB/s $\approx$ 590 FLOP/byte for dense FP8. An expert needs **hundreds of rows** before it becomes compute-bound.

```
# file: moe_shapes.py
# Back-of-envelope numbers for one MoE layer: tokens per expert, padding waste of a block-al
# grouped GEMM, and arithmetic intensity of the expert weights (the number that decides whet
# the layer is bandwidth-bound or compute-bound).
import numpy as np

rng = np.random.default_rng(0)


def routed_counts(tokens, k, E, skew):
    """Expert loads for `tokens` tokens choosing k distinct experts; skew=0 is uniform, >0 i
    p = 1.0 / np.arange(1, E + 1) ** skew
    p /= p.sum()
    counts = np.zeros(E, dtype=np.int64)
    for _ in range(tokens):
```

```
        counts[rng.choice(E, size=k, replace=False, p=p)] += 1
    return counts


def padded_rows(counts, block_m):
    return int(((counts + block_m - 1) // block_m * block_m).sum())


# DeepSeek-V3-like layer: hidden 7168, expert intermediate 2048, 256 routed experts, top-8.
H, I, E, K = 7168, 2048, 256, 8
w_bytes = 3 * H * I * 1                      # one expert's W1, W3, W2 in FP8 (1 byte/param)
print(f"one expert: {3*H*I/1e6:.1f} M params = {w_bytes/2**20:.0f} MiB in FP8\n")
print(f"{'tokens':>6} {'skew':>4} {'tok/exp':>7} {'active':>6} {'max':>5} {'BLOCK_M':>7} {'r
      f"{'FLOP/B(w)':>9}")
for tokens in (1, 16, 128, 1024, 8192):
    for skew in (0.0, 1.0):
        counts = routed_counts(tokens, K, E, skew)
        active = int((counts > 0).sum())
        for block_m in (16, 64):
            rows = padded_rows(counts, block_m)
            useful = tokens * K
            # FLOPs actually needed vs bytes of expert weights that must be streamed (active
            ai = 2 * useful * 3 * H * I / (active * w_bytes)
            print(f"{tokens:>6} {skew:>4} {useful/E:>7.2f} {active:>6} {int(counts.max()):>5
                  f"{1-useful/rows:>6.0%} {ai:>9.1f}")
```

```
python moe_shapes.py
```

Output:

```
one expert: 44.0 M params = 42 MiB in FP8

tokens skew tok/exp active   max BLOCK_M    rows  waste FLOP/B(w)
     1  0.0    0.03      8     1      16     128    94%       2.0
     1  0.0    0.03      8     1      64     512    98%       2.0
     1  1.0    0.03      8     1      16     128    94%       2.0
     1  1.0    0.03      8     1      64     512    98%       2.0
    16  0.0    0.50    103     4      16    1648    92%       2.5
    16  0.0    0.50    103     4      64    6592    98%       2.5
    16  1.0    0.50     80    13      16    1280    90%       3.2
    16  1.0    0.50     80    13      64    5120    98%       3.2
   128  0.0    4.00    252    11      16    4032    75%       8.1
   128  0.0    4.00    252    11      64   16128    94%       8.1
   128  1.0    4.00    205   110      16    3632    72%      10.0
   128  1.0    4.00    205   110      64   13248    92%      10.0
  1024  0.0   32.00    256    47      16   10032    18%      64.0
```

```
1024  0.0   32.00   256    47     64   16384   50%    64.0
1024  1.0   32.00   256   774     16   10176   19%    64.0
1024  1.0   32.00   256   774     64   20352   60%    64.0
8192  0.0  256.00   256   308     16   67424    3%   512.0
8192  0.0  256.00   256   308     64   73472   11%   512.0
8192  1.0  256.00   256  6455     16   67504    3%   512.0
8192  1.0  256.00   256  6455     64   73024   10%   512.0
```

Reading the table, row by row:

- **1 to 16 tokens (decode):** 8 to 103 experts are active, each with about one row. Intensity is 2 to 3 FLOP/byte, deep in bandwidth-bound territory. The padding "waste" of 90–98% is irrelevant: those MMA rows are free while the kernel waits on HBM. The real cost is active$\times$42 MiB of weights per layer. Below about 128 tokens, each extra token can wake experts that were idle, so per-token cost falls more slowly than in a dense model. Once nearly every expert is active, extra decode tokens add **no weight bytes** at all, and batching becomes almost free until the ridge.
- **1024 tokens:** every expert is active, with 32 rows on average. Padding to 64 now wastes half the tensor-core work, so `BLOCK_M=16` or 32 is better, and still bandwidth-bound at 64 FLOP/byte.
- **8192 tokens (prefill):** 256 rows per expert reach the ridge, and padding waste drops to 3–11%. This is where `wgmma`-sized tiles, TMA, and warp specialization pay off, as for any large GEMM.
- **Skew (`skew=1.0`):** the hottest expert takes 774 of 1024 and 6455 of 8192 routed rows, so one expert holds most of the batch. A grid with one tile per `BLOCK_M` rows still runs, but the hot expert's tiles form a long tail if tiles are scheduled expert by expert. Persistent kernels that hand out tiles from a global queue, and the load-balancing router from section 4, both exist because of this column.

For expert parallelism across GPUs, the same arithmetic applies per GPU, with E replaced by the local expert count and T by the tokens the all-to-all delivers to that GPU.

---

9. Where this sits in production

Component	Role
vLLM / SGLang `fused_moe` (Triton)	The gather/scatter design of section 6: `moe_align_block_size`, a `fused_moe_kernel` with FP8/INT8/INT4 weight variants, tuned configs per (E, N, GPU), and fused `silu_and_mul` and `moe_sum` kernels

Component	Role
`torch.nn.functional.grouped_mm`	Public PyTorch grouped GEMM over 2-D/3-D operands with `offs`. Used for MoE training in torchtitan
CUTLASS grouped GEMM	Many problem sizes in one persistent launch. Problem shapes and pointer arrays can live on the device
DeepGEMM grouped GEMMs	FP8 "m-grouped" kernels with a **contiguous** layout (each expert's rows padded to the M-block alignment, for prefill and training) and a **masked** layout (fixed capacity per expert plus a valid-row count, for CUDA-Graph decode)
Triton grouped GEMM tutorial	A persistent Triton kernel that walks a device-side list of GEMM problems
MegaBlocks	"Dropless" MoE as block-sparse GEMMs. It removed the capacity-factor token dropping of earlier MoE systems
DeepEP	Expert-parallel all-to-all **dispatch** and **combine**. High-throughput kernels for prefill and training, and low-latency RDMA kernels for decode

The capacity-factor approach deserves a sentence because generated code still produces it: early MoE systems gave every expert a fixed number of slots and **dropped** tokens that overflowed. That gives static shapes, but it silently changes the model's output under load. Modern inference stacks are dropless: every routed row is computed.

---

10. Verification and benchmark plan

Functional

Check	Catches
Loop vs dense vs fused vs grouped-GEMM agree	Index bugs in any one path
Experts with 0 rows, 1 row, `BLOCK_M-1`, `BLOCK_M`, `BLOCK_M+1` rows	Tile-boundary and empty-expert bugs (reading weights of an expert with no tile)
$T = 1$ and $T \cdot k < E$	The decode path, and any per-token fallback
One expert receives every token	Grid sizing, the worst-case padded length, stragglers
Router: biased selection, unbiased weights, renormalization, scale factor	Silent quality bugs (section 4)
Sentinel rows never written; output rows written exactly once per slot	Races and out-of-bounds stores (run `compute-sanitizer` on a GPU)
BF16/FP8 inputs vs FP32 reference with tolerances	Accumulating in low precision; FP8 scale handling

Performance

Metric	Target
Decode: achieved bandwidth = active-expert weight bytes / layer time	Approaches HBM peak
Prefill: achieved TFLOP/s on useful rows only (exclude padding)	Approaches a dense GEMM of the same total size
Alignment + router + combine share of layer time	Small at prefill. Watch it at decode, where it can rival the GEMMs
Sweep over T and routing skew	No cliff where the tile config changes

Benchmark with **real router outputs** captured from the model on real prompts, not `randint` expert IDs. Uniform random routing hides the straggler problem that real traffic creates.

---

11. Prompting an AI assistant for an MoE kernel

Generated MoE code tends to loop over experts in Python or to assume uniform loads. State the contract:

1. **Shapes and layout:** `x [T, H]`, `w13 [E, 2I, H]` and `w2 [E, H, I]` in `nn.Linear` layout, `topk_ids/topk_w [T, k]`, flat row $r = t \cdot k + j$.
2. **Metadata:** produce `sorted_ids`, `expert_ids`, and the padded count with a sentinel. Empty experts get no tiles.
3. **Kernel:** one launch per GEMM, tile-to-expert mapping, gathered A rows (`r // k` for GEMM1), scattered C rows, router weight in the GEMM2 epilogue, FP32 accumulation.
4. **Router semantics** if DeepSeek-style: sigmoid, bias for selection only, group-limited top-k, renormalize, scale.

5. **Tests first:** interpreter-mode tests against a loop *and* a dense reference across the cases in section 10, plus an AOT compile for the target with a check that the expected MMA instruction appears.
6. **Metric:** GB/s of weights at decode, useful TFLOP/s at prefill.

Pitfalls

Pitfall	Result
Python loop over experts	E launches per layer, idle GPU between them
Capacity factor with token dropping	Static shapes, silently different outputs under load
Gate weights from bias-adjusted scores	Plausible but wrong outputs, no error
Forgetting renormalization or `routed_scaling_factor`	Output scaled wrongly by a per-token factor
Padding rows not masked on store	Writes past the output buffer, or over real rows
Launching tiles for empty experts	Weight bytes read for nothing, which hurts most at decode
One tile config for all batch sizes	Prefill-tuned 64-row tiles at decode, or `mma.sync` tiles at prefill
Benchmarking with uniform random routing	Hides hot-expert stragglers

Checkpoint

- Explain why an MoE layer is E ragged GEMMs whose sizes are only known on the device.
- Write the `sorted_ids` / `expert_ids` contract and compute it with a histogram and a scan.
- Describe how a fused kernel gathers A rows for GEMM1 versus GEMM2, and where the router weight is applied.
- State the DeepSeek-V3 routing rule, and which scores the gate weights come from.
- Use "FLOP per weight byte $\approx 2n_e$" to say whether a given MoE layer is bandwidth- or compute-bound.
- Explain why 16-row tiles cannot use Hopper `wgmma`, and name two ways decode kernels deal with small row counts.

Try this

1. Change `moe_align.py` to compute `sorted_ids` without `argsort`: one pass for the histogram, an exclusive scan, and an atomic-style running counter per expert. Check it against the current output.

2. Add an `expert_map` argument for expert parallelism: experts not on this GPU get `expert_ids = -1`, and the kernel skips those tiles. Test with half the experts remote.
3. Fuse `silu(gate) * up` into the GEMM1 epilogue by computing the gate and up halves as two accumulators in the same program. Confirm the interpreter test still passes.
4. Run `compile_check.py` with `BLOCK_M {16, 32, 64, 128}` and record which tiles get `wgmma`, and the shared memory each needs.
5. On a GPU, capture real `topk_ids` from a small MoE model, then sweep $T \in \{1, 8, 64, 512, 4096\}$ and plot achieved GB/s and TFLOP/s for `BLOCK_M=16` and `BLOCK_M=64`.

Summary

An MoE layer replaces one GEMM with many small, ragged, data-dependent GEMMs. Making it fast starts with metadata: a histogram, a scan, and a scatter turn the router's `topk_ids` into a block-aligned table that maps every tile to one expert. A single grouped-GEMM launch then either reads a permuted copy of the activations or gathers rows through the table, and scatters results back to token order with the router weight folded in. Routing semantics are part of correctness, especially the selection-only bias in DeepSeek-style routers. Shape arithmetic tells you which regime you are in: decode streams weights at about 2 FLOP per byte and needs bandwidth-oriented kernels, while prefill reaches the ridge only when experts see hundreds of rows. Verify the indexing on a CPU with the Triton interpreter, compile ahead of time for the target, and benchmark with real routing traces.

Sources (for further reading)

- DeepSeek-AI, "DeepSeek-V3 Technical Report" (arXiv:2412.19437): auxiliary-loss-free balancing with a selection bias, sigmoid gating, node-limited routing. Model `config.json` files on Hugging Face for DeepSeek-V3, Kimi K2, Qwen3-235B-A22B, gpt-oss-120b, and Mixtral-8x7B (expert counts and routing fields).
- vLLM source: `vllm/model_executor/layers/fused_moe/moe_align_block_size.py` (docstring example reproduced in section 5) and `fused_moe.py` (`fused_moe_kernel`, the per-token assignment heuristic, tuned configs).
- PyTorch documentation: `torch.nn.functional.grouped_mm` (2.14); torchtitan MoE (`_run_experts_grouped_mm`).
- Gale, Narayanan, Young, Zaharia, "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts" (MLSys 2023, arXiv:2211.15841).
- Fedus, Zoph, Shazeer, "Switch Transformers" (arXiv:2101.03961): capacity factor and token dropping.
- DeepSeek-AI, DeepGEMM repository (m-grouped contiguous and masked FP8 GEMMs) and DeepEP repository (expert-parallel dispatch and combine kernels).
- NVIDIA CUTLASS documentation: grouped GEMM. PTX ISA: `wgmma.mma_async` shapes (m64nNk16).
- Triton documentation: tutorial "Group GEMM", `TRITON_INTERPRET=1`, and `triton.compile` with `GPUTarget`.

Table-Batched Embeddings in Triton: Gather, Pool, and Run-Length Backward

Table-Batched Embeddings in Triton: Gather, Pool, and Run-Length Backward

Recommendation models spend their accelerator time in two places: dense MLPs, which are GEMMs and therefore familiar ground, and **embedding lookups**, which are not. A ranking model may carry hundreds of sparse features ("pages liked", "videos watched"), each with its own table of millions of rows. Every request turns into a few thousand row gathers, a sum per feature, and, in training, a sparse gradient that has to be folded back into exactly the rows it came from. There is almost no arithmetic. The job is memory traffic and bookkeeping.

The production operator for this is **TBE**, the *table-batched embedding*: one launch performs the lookups and pooling for **all** tables. In October 2026 the PyTorch/Meta team described replacing FBGEMM's templated CUDA TBE kernels with **FBTriton** kernels. They reported a median 1.28× forward speedup over 307 shard configurations on GB200 and up to 2× in backward. Their analysis attributes most of the backward win to *where the code switches strategies*, not to clever instructions.

This chapter builds a small TBE in Triton that you can verify on a laptop: a pooled forward kernel with a tunable **gather width**, and a backward that sorts lookups into **runs** (one per unique row), sends short and long runs to different kernels, and applies exactly one optimizer step per row. Every listing runs under `TRITON_INTERPRET=1` on CPU, and the kernels are then compiled for `sm_90` and `sm_100` without a GPU.

Versions used: **Triton 3.8.0**, **PyTorch 2.14.1+cpu**, Python 3.13.5, Debian 13 x86-64 (no GPU). Timings in the interpreter mean nothing, so this chapter makes no speed claims of its own. The performance numbers quoted are FBTriton's.

---

1. Mental model: what TBE computes

Notation follows the FBTriton write-up: **T** tables, each with **E** rows of width **D**. **B** is the batch size (bags per table), and **L** is the pooling factor (lookups per bag, varying from bag to bag).

```
inputs (flattened, table-major)                one output row per sample
offsets: [t0b0 | t0b1 | ... | t1b0 | ... ]     out[b] = [ pool_t0(b) | pool_t1(b) | ... ]
indices:  3 9 9   1   ...   4 4 7 2   ...
                                               pool_t(b) = Σ_{i   bag(t,b)}  psw_i · W_t[ic
W: all tables concatenated, table_off[t] = first row of table t
```

Pass	Work per lookup	Hard part
Forward	read one D-wide row, add	latency of dependent loads (`idx` → row address → row)
Backward	add one D-wide gradient into its row	many lookups hit the **same** row, and the counts are skewed by orders of magnitude

The backward is a **scatter-add with duplicates**. Its shape depends on the data. With real IDs, a handful of rows (the popular item, the default token) appear in thousands of bags, and most rows appear once. The FBTriton post cites a batch with 83 M lookups, 3 M unique rows, and a hottest row hit 125 K times. Its central variable is the **segment length (SL)**, the number of lookups that touch one row.

---

2. The kernels

Save as `tbe_triton.py`. The file holds four kernels, the host-side planning for the backward, a batch generator with skewed IDs, and a reference built on `torch.nn.functional.embedding_bag`.

```
# tbe_triton.py - table-batched embedding (sum pooling) forward + fused-SGD backward in Trit
# GPU:  python tbe_triton.py        CPU correctness:  TRITON_INTERPRET=1 python tbe_triton.p
import os

import torch
import triton
import triton.language as tl


@triton.jit
def tbe_fwd_kernel(W, TABLE_OFF, IDX, OFFS, PSW, OUT, B, T,
                   D: tl.constexpr, BLOCK_D: tl.constexpr, GATHER: tl.constexpr,
                   BAGS: tl.constexpr, HAS_PSW: tl.constexpr):
    # one program = BAGS consecutive samples; it walks every table for them (no B x T grid)
    cols = tl.arange(0, BLOCK_D)
    cmask = cols < D
    g = tl.arange(0, GATHER)
    for bb in range(BAGS):
        b = tl.program_id(0) * BAGS + bb
        if b < B:
            for t in range(T):
                start = tl.load(OFFS + t * B + b)
                end = tl.load(OFFS + t * B + b + 1)
                row0 = tl.load(TABLE_OFF + t)
```

```
                acc = tl.zeros([BLOCK_D], dtype=tl.float32)          # accumulate in fp32
                for j in range(start, end, GATHER):                   # GATHER independent r
                    lmask = (j + g) < end
                    rows = row0 + tl.load(IDX + j + g, mask=lmask, other=0).to(tl.int64)
                    tile = tl.load(W + rows[:, None] * D + cols[None, :],
                                   mask=lmask[:, None] & cmask[None, :], other=0.0).to(tl.fl
                    if HAS_PSW:
                        tile *= tl.load(PSW + j + g, mask=lmask, other=0.0)[:, None]
                    acc += tl.sum(tile, axis=0)
                tl.store(OUT + b * (T * D) + t * D + cols, acc.to(OUT.dtype.element_ty), mas


@triton.jit
def short_run_kernel(W, GRAD_OUT, RUN_ROW, RUN_START, RUN_LEN, SORTED_POS, POS_BAG, POS_TABL
                     SHORT_IDS, lr, T, D: tl.constexpr, BLOCK_D: tl.constexpr):
    # one program owns one run (= one unique row): gather grads, sum, update. Plain store, n
    r = tl.load(SHORT_IDS + tl.program_id(0))
    cols = tl.arange(0, BLOCK_D)
    cmask = cols < D
    start = tl.load(RUN_START + r)
    n = tl.load(RUN_LEN + r)
    acc = tl.zeros([BLOCK_D], dtype=tl.float32)
    for k in range(n):
        p = tl.load(SORTED_POS + start + k)
        b = tl.load(POS_BAG + p).to(tl.int64)
        t = tl.load(POS_TABLE + p)
        acc += tl.load(GRAD_OUT + b * (T * D) + t * D + cols, mask=cmask, other=0.0).to(tl.f
    row = tl.load(RUN_ROW + r).to(tl.int64)
    w = tl.load(W + row * D + cols, mask=cmask).to(tl.float32)
    tl.store(W + row * D + cols, (w - lr * acc).to(W.dtype.element_ty), mask=cmask)


@triton.jit
def long_run_accum_kernel(GRAD_OUT, WS, CHUNK_SLOT, CHUNK_START, CHUNK_END,
                          SORTED_POS, POS_BAG, POS_TABLE, T, D: tl.constexpr, BLOCK_D: tl.co
    # one program = one fixed-size chunk of a long run (split-K); partials meet in a workspa
    c = tl.program_id(0)
    cols = tl.arange(0, BLOCK_D)
    cmask = cols < D
    acc = tl.zeros([BLOCK_D], dtype=tl.float32)
    for k in range(tl.load(CHUNK_START + c), tl.load(CHUNK_END + c)):
        p = tl.load(SORTED_POS + k)
        b = tl.load(POS_BAG + p).to(tl.int64)
        t = tl.load(POS_TABLE + p)
        acc += tl.load(GRAD_OUT + b * (T * D) + t * D + cols, mask=cmask, other=0.0).to(tl.f
    slot = tl.load(CHUNK_SLOT + c).to(tl.int64)
    tl.atomic_add(WS + slot * D + cols, acc, mask=cmask)
```

```
@triton.jit
def long_run_apply_kernel(W, WS, LONG_ROW, lr, D: tl.constexpr, BLOCK_D: tl.constexpr):
    s = tl.program_id(0)
    cols = tl.arange(0, BLOCK_D)
    cmask = cols < D
    row = tl.load(LONG_ROW + s).to(tl.int64)
    g = tl.load(WS + s * D + cols, mask=cmask)
    w = tl.load(W + row * D + cols, mask=cmask).to(tl.float32)
    tl.store(W + row * D + cols, (w - lr * g).to(W.dtype.element_ty), mask=cmask)


def tbe_forward(W, table_off, idx, offs, B, T, D, psw=None, gather=4, bags=2):
    out = torch.empty(B, T * D, dtype=torch.float32, device=W.device)
    grid = (triton.cdiv(B, bags),)
    tbe_fwd_kernel[grid](W, table_off, idx, offs, psw if psw is not None else W, out, B, T,
                         D=D, BLOCK_D=triton.next_power_of_2(D), GATHER=gather, BAGS=bags,
                         HAS_PSW=psw is not None)
    return out


def tbe_backward_sgd(W, grad_out, table_off, idx, offs, B, T, D, lr, threshold=8, chunk=8):
    """Sum gradients per unique (table,row), apply one SGD step per row. Returns path stats.
    dev = W.device
    lengths = offs[1:] - offs[:-1]                                      # length of each (t, b
    seg = torch.arange(T * B, device=dev).repeat_interleave(lengths)  # which bag each looku
    pos_table = (seg // B).to(torch.int32)
    pos_bag = (seg % B).to(torch.int32)
    linear = table_off[pos_table.long()] + idx.long()                   # global row id per lo
    # "transpose": sort lookups by row and run-length encode -> one run per unique row
    sorted_rows, sorted_pos = torch.sort(linear, stable=True)
    run_row, run_len = torch.unique_consecutive(sorted_rows, return_counts=True)
    run_start = torch.cumsum(run_len, 0) - run_len
    is_long = run_len >= threshold
    short_ids = torch.nonzero(~is_long).flatten().to(torch.int32)
    long_ids = torch.nonzero(is_long).flatten()
    BLOCK_D = triton.next_power_of_2(D)
    sp = sorted_pos.to(torch.int32)
    if len(short_ids):
        short_run_kernel[(len(short_ids),)](W, grad_out, run_row, run_start, run_len.to(torc
                                            sp, pos_bag, pos_table, short_ids, lr, T,
                                            D=D, BLOCK_D=BLOCK_D)
    n_chunks = 0
    if len(long_ids):
        lens, starts = run_len[long_ids], run_start[long_ids]
        per = (lens + chunk - 1) // chunk                                # chunks per long ru
```

```
        slot = torch.arange(len(long_ids), device=dev).repeat_interleave(per)
        k = torch.arange(int(per.sum()), device=dev) - (torch.cumsum(per, 0) - per).repeat_i
        c_start = starts.repeat_interleave(per) + k * chunk
        c_end = torch.minimum(c_start + chunk, (starts + lens).repeat_interleave(per))
        n_chunks = len(c_start)
        ws = torch.zeros(len(long_ids), D, dtype=torch.float32, device=dev)
        long_run_accum_kernel[(n_chunks,)](grad_out, ws, slot,
                                           c_start, c_end, sp, pos_bag, pos_table, T,
                                           D=D, BLOCK_D=BLOCK_D)
        long_run_apply_kernel[(len(long_ids),)](W, ws, run_row[long_ids], lr, D=D, BLOCK_D=B
    return dict(lookups=len(idx), unique_rows=len(run_row), short_runs=len(short_ids),
                long_runs=len(long_ids), long_chunks=n_chunks, max_run=int(run_len.max()))


def make_batch(T, E, B, D, max_L, dtype, dev, seed=0):
    g = torch.Generator().manual_seed(seed)
    table_off = torch.arange(T, dtype=torch.int64) * E
    W = (torch.randn(T * E, D, generator=g) * 0.1).to(dtype)
    lengths = torch.randint(0, max_L + 1, (T * B,), generator=g)      # table-major: t*B + b
    offs = torch.zeros(T * B + 1, dtype=torch.int64)
    offs[1:] = torch.cumsum(lengths, 0)
    # skewed ids: a few rows are very hot, like real recommendation features
    u = torch.rand(int(offs[-1]), generator=g)
    idx = torch.clamp((-torch.log1p(-u) / 0.15).long(), max=E - 1).to(torch.int32)  # Exp(0.
    to = lambda x: x.to(dev)
    return to(W), to(table_off), to(idx), to(offs)


def reference(W, table_off, idx, offs, B, T, D, psw=None):
    outs = []
    for t in range(T):
        o = offs[t * B: (t + 1) * B + 1]
        rows = W[table_off[t]: table_off[t] + (table_off[1] - table_off[0])]
        outs.append(torch.nn.functional.embedding_bag(
            idx[o[0]:o[-1]].long(), rows, (o[:-1] - o[0]), mode="sum",
            per_sample_weights=None if psw is None else psw[o[0]:o[-1]]))
    return torch.cat(outs, dim=1)


if __name__ == "__main__":
    dev = "cpu" if os.environ.get("TRITON_INTERPRET") == "1" else "cuda"
    torch.manual_seed(0)
    T, E, B, D, L = 3, 50, 16, 24, 12
    for dtype in (torch.float32, torch.float16):
        W, toff, idx, offs = make_batch(T, E, B, D, L, dtype, dev)
        psw = torch.rand(len(idx), device=dev)
        out = tbe_forward(W, toff, idx, offs, B, T, D)
```

```
        ref = reference(W.float(), toff, idx, offs, B, T, D)
        outw = tbe_forward(W, toff, idx, offs, B, T, D, psw=psw)
        refw = reference(W.float(), toff, idx, offs, B, T, D, psw=psw)
        print(f"{str(dtype):14s} fwd max|err| = {(out - ref).abs().max():.2e}    "
              f"weighted = {(outw - refw).abs().max():.2e}")

    # backward + SGD against autograd
    W, toff, idx, offs = make_batch(T, E, B, D, L, torch.float32, dev, seed=1)
    Wref = W.clone().requires_grad_(True)
    grad_out = torch.randn(B, T * D, device=dev)
    reference(Wref, toff, idx, offs, B, T, D).backward(grad_out)
    lr = 0.05
    expected = Wref.detach() - lr * Wref.grad
    stats = tbe_backward_sgd(W, grad_out, toff, idx, offs, B, T, D, lr)
    print("bwd+SGD max|err| =", f"{(W - expected).abs().max():.2e}", stats)
```

What to notice in the forward:

- **Grid.** `cdiv(B, BAGS)` programs, and each program loops over all T tables, as in FBTriton's generic path. There is no `B × T` grid, so there are fewer, longer-lived programs, and offsets for the same sample are read close together.
- **Gather width.** Each loop iteration loads `GATHER` indices and then a `[GATHER, BLOCK_D]` tile of rows: `GATHER` independent row loads in flight. That hides the latency of the dependent `idx → row` chain. The cost is registers, because the pointer tile `rows[:, None] * D + cols[None, :]` holds `GATHER × BLOCK_D` 64-bit addresses. FBTriton measured this exact cliff on B200: dropping the width from 8 to 2 cut registers from 184 to 64 and roughly quadrupled occupancy in its short-run backward tier. The right width is a per-target **config value**.
- **Accumulate in FP32.** The table can be FP16 or BF16, but `acc` is FP32. FBTriton goes further and accumulates FP32 tables in FP64 for large D.

What to notice in the backward:

- **The transpose.** `torch.sort` plus `unique_consecutive` turns "lookups in batch order" into **runs**: one unique row, plus the positions of every lookup that touched it. FBTriton calls this `transpose_embedding_input`, and in one optional mode it hoists the step into the forward pass.
- **Short runs** (`SL < threshold`): one program owns the whole run. It sums the gradients in registers, applies the update, and does a **plain store**. No atomics are needed, because no other program touches that row.
- **Long runs**: split into fixed-size chunks (split-K). Each chunk program writes its partial sum with `tl.atomic_add` into a per-run workspace row, and a second kernel applies the update. FBTriton uses 256-lookup chunks and a threshold of `SL  256`. On Blackwell it fuses the two kernels with a device-scope fence and a countdown, so that the last chunk applies the update.

---

3. Verify on CPU

```
pip install torch triton            # CPU wheels are enough for interpreter mode
TRITON_INTERPRET=1 python tbe_triton.py
```

```
torch.float32  fwd max|err| = 1.79e-07   weighted = 1.19e-07
torch.float16  fwd max|err| = 0.00e+00   weighted = 5.96e-08
bwd+SGD max|err| = 1.79e-07 {'lookups': 313, 'unique_rows': 67, 'short_runs': 54, 'long_runs
```

Forward results match `embedding_bag(mode="sum")` table by table, with and without per-sample weights, for FP32 and FP16 tables. The backward is checked the strict way: autograd computes the gradient of the reference forward, and `W - lr * grad` is compared with what the Triton kernels wrote into `W`. The skewed generator produced 313 lookups but only 67 unique rows, with the hottest row touched 20 times.

The split point must not change the answer. `thresholds.py` runs the same batch with every lookup sent through the short path, a mixed split, a fine split, and every lookup sent through the atomic path:

```
# thresholds.py - same batch, different short/long split points; the answer must not change
import os

import torch

from tbe_triton import make_batch, reference, tbe_backward_sgd

dev = "cpu" if os.environ.get("TRITON_INTERPRET") == "1" else "cuda"
T, E, B, D, L, lr = 3, 50, 16, 24, 12, 0.05
W0, toff, idx, offs = make_batch(T, E, B, D, L, torch.float32, dev, seed=1)
grad_out = torch.randn(B, T * D, device=dev, generator=torch.Generator().manual_seed(2))
Wref = W0.clone().requires_grad_(True)
reference(Wref, toff, idx, offs, B, T, D).backward(grad_out)
expected = Wref.detach() - lr * Wref.grad

for threshold, chunk in [(10**9, 8), (8, 8), (4, 2), (1, 1)]:
    W = W0.clone()
    s = tbe_backward_sgd(W, grad_out, toff, idx, offs, B, T, D, lr, threshold=threshold, chu
    print(f"threshold={threshold:<10} chunk={chunk}  short={s['short_runs']:<3} long={s['lon
          f"chunks={s['long_chunks']:<4} max|err|={(W - expected).abs().max():.1e}")
```

```
threshold=1000000000 chunk=8  short=67  long=0   chunks=0    max|err|=1.8e-07
threshold=8          chunk=8  short=54  long=13  chunks=26   max|err|=2.4e-07
threshold=4          chunk=2  short=39  long=28  chunks=132  max|err|=2.4e-07
threshold=1          chunk=1  short=0   long=67  chunks=313  max|err|=1.8e-07
```

All four routings agree to FP32 rounding. On a GPU the four runs differ only in speed, and

the threshold becomes a **tuning knob**. That is the FBTriton result in miniature. Their CUDA baseline switched to a cooperative CTA-per-row kernel at `SL = 32`, while the Triton version kept simple streaming programs up to `SL = 256`. In that band Triton measured 3,948 GB/s against CUDA's 678 GB/s on the same shape.

---

4. Compile for GPUs you do not have

The interpreter checks semantics, not lowering. Lower the kernels for Hopper and Blackwell ahead of time:

```
# compile_check.py - lower the TBE kernels to cubin for GPUs this machine does not have
import triton
from triton.backends.compiler import GPUTarget
from triton.compiler import ASTSource

from tbe_triton import long_run_accum_kernel, short_run_kernel, tbe_fwd_kernel

fwd_sig = {"W": "*fp16", "TABLE_OFF": "*i64", "IDX": "*i32", "OFFS": "*i64", "PSW": "*fp32",
           "OUT": "*fp32", "B": "i32", "T": "i32", "D": "constexpr", "BLOCK_D": "constexpr",
           "GATHER": "constexpr", "BAGS": "constexpr", "HAS_PSW": "constexpr"}
short_sig = {"W": "*fp32", "GRAD_OUT": "*fp32", "RUN_ROW": "*i64", "RUN_START": "*i64",
             "RUN_LEN": "*i32", "SORTED_POS": "*i32", "POS_BAG": "*i32", "POS_TABLE": "*i32"
             "SHORT_IDS": "*i32", "lr": "fp32", "T": "i32", "D": "constexpr", "BLOCK_D": "co
accum_sig = {"GRAD_OUT": "*fp32", "WS": "*fp32", "CHUNK_SLOT": "*i64",
             "CHUNK_START": "*i64", "CHUNK_END": "*i64", "SORTED_POS": "*i32", "POS_BAG": "*
             "POS_TABLE": "*i32", "T": "i32", "D": "constexpr", "BLOCK_D": "constexpr"}

for cc in (90, 100):
    target = GPUTarget("cuda", cc, 32)
    for name, fn, sig, ce in [
        ("fwd gather=4", tbe_fwd_kernel, fwd_sig, dict(D=128, BLOCK_D=128, GATHER=4, BAGS=2,
        ("fwd gather=8", tbe_fwd_kernel, fwd_sig, dict(D=128, BLOCK_D=128, GATHER=8, BAGS=2,
        ("short_run", short_run_kernel, short_sig, dict(D=128, BLOCK_D=128)),
        ("long_run_accum", long_run_accum_kernel, accum_sig, dict(D=128, BLOCK_D=128)),
    ]:
        k = triton.compile(ASTSource(fn=fn, signature=sig, constexprs=ce), target=target)
        ptx = k.asm["ptx"]
        atomics = ptx.count("red.global") + ptx.count("atom.global")
        print(f"sm_{cc:<3} {name:15s} stages={','.join(k.asm.keys())}  atomics_in_ptx={atomi
```

```
sm_90  fwd gather=4    stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
sm_90  fwd gather=8    stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
sm_90  short_run       stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
```

```
sm_90  long_run_accum  stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=1
sm_100 fwd gather=4    stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
sm_100 fwd gather=8    stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
sm_100 short_run       stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=0
sm_100 long_run_accum  stages=source,ttir,ttgir,llir,ptx,cubin  atomics_in_ptx=1
```

Every kernel reaches a cubin for both targets. The PTX of the long-run accumulator contains its atomic reduction, and the other three contain none, which is a cheap structural check that the short-run path really is atomic-free. An agent loop can run this file in seconds as a gate before any GPU time is spent.

---

5. The trap: one optimizer step per row, not per lookup

A tempting shortcut is a backward that skips the sort and the runs: one program per lookup, each applying its own gradient to its row. With **SGD** it even passes tests (setting the race aside), because the SGD update is linear in the gradient. With any stateful optimizer it is wrong, even with perfect atomics. FBTriton's production configuration uses exact **row-wise Adagrad**:

```python
# adagrad_trap.py - why TBE backward sums per row *before* the optimizer step
import torch

torch.manual_seed(0)
D, lr, eps = 4, 0.1, 1e-8
w0 = torch.randn(D)
grads = [torch.randn(D) for _ in range(5)]      # 5 bags in one batch touched the same row

def rowwise_adagrad(w, state, g):
    state = state + (g * g).mean()              # row-wise: one accumulator per row
    return w - lr * g / (state.sqrt() + eps), state

# correct: one update with the summed gradient (what autograd + optimizer.step() does)
w_ok, _ = rowwise_adagrad(w0, torch.tensor(0.0), sum(grads))

# wrong: one update per lookup, as a naive "scatter the optimizer" kernel would do
w_bad, s = w0, torch.tensor(0.0)
for g in grads:
    w_bad, s = rowwise_adagrad(w_bad, s, g)

print("one step, summed grad :", w_ok)
print("five per-lookup steps :", w_bad)
print("max |difference|      :", f"{(w_ok - w_bad).abs().max():.3f}")

# with plain SGD the two agree (the update is linear in g), which is why SGD tests hide the
```

```
sgd_ok = w0 - lr * sum(grads)
sgd_bad = w0.clone()
for g in grads:
    sgd_bad -= lr * g
print("SGD max |difference|  :", f"{(sgd_ok - sgd_bad).abs().max():.1e}")
```

```
one step, summed grad : tensor([ 1.7126, -0.3612, -2.1057,  0.5439])
five per-lookup steps : tensor([ 1.7876, -0.2906, -2.0869,  0.5086])
max |difference|      : 0.075
SGD max |difference|  : 3.0e-08
```

The fix is the structure of Section 2: **reduce first, then update once**. That is why the run-length transpose is not optional bookkeeping. It defines the semantics. The same reasoning applies to the long-run path. The update may run only after *every* chunk's partial has landed, so the two-kernel split here (or a fence plus countdown in a fused kernel) is a correctness requirement, not a performance detail. Validate a TBE backward with a stateful optimizer, never with SGD alone.

---

6. Prompting an AI assistant for a TBE kernel

Kernels in this family fail in data-dependent ways, so put the data in the prompt and the checks in the harness:

1. **State the semantics exactly.** Give the layout (table-major `offsets` of length `T·B+1`, `table_off`, the output shape `B × T·D`), the pooling mode, per-sample weights, and the accumulation dtype.
2. **Give the reference.** `embedding_bag` per table for the forward, and autograd plus the *real* optimizer for the backward.
3. **Demand skew in the tests.** Uniform random IDs produce runs of length 1 and never exercise the long path. Use a heavy-tailed generator and assert that both paths ran (`short_runs > 0 and long_runs > 0`).
4. **Ask for config knobs**, not constants: gather width, bags per program, threshold, chunk size. They are what you will tune per GPU.
5. **Gate on the CPU first.** Interpreter equivalence, then the threshold sweep, then the cross-target compile. Only then benchmark on hardware.

Pitfalls

- **Index overflow.** `row * D` in int32 overflows once a table holds more than 2^{31} elements. Cast row ids to int64 before multiplying (the kernels above do). Int32 *indices* are fine and halve index traffic when the linearized row range fits.

- **Masked-lane waste.** `BLOCK_D` is a `constexpr` set to `next_pow2(max D)`. When tables have mixed widths, narrow tables waste most of each tile. FBTriton buckets runs by `next_pow2(D)` and launches each bucket with its own block size.
- **Host syncs.** Asking "how many long runs?" with `.item()` costs a stream synchronize on every backward. Production code classifies runs on the device and passes the counts to the next kernels as device pointers. The host-side planning here keeps the chapter readable.
- **Out-of-range ids.** A bad index reads another table's row without a word. Validate in a bounds-check step (FBTriton fuses it into an eligible forward), or at least in debug builds.
- **Testing only SGD.** See Section 5.

Checkpoint

1. Why can the short-run kernel use a plain store while the long-run path needs `atomic_add`?
2. What does `GATHER` trade against, and why is its best value target-specific?
3. Show that per-lookup SGD equals per-row SGD, and identify the exact step that fails for row-wise Adagrad.
4. Why does uniform random test data give false confidence in a TBE backward?

Try this

- Add mean pooling (divide by the bag length, and define what an empty bag returns), then compare with `embedding_bag(mode="mean")`.
- Implement row-wise Adagrad in `short_run_kernel` and `long_run_apply_kernel` (one FP32 state per row) and check it against `torch.optim.Adagrad` applied to the reference's gradient.
- Fuse the long path. Have `long_run_accum_kernel` decrement a per-run counter after its `atomic_add`, and let the program that sees `1` apply the update. Then explain why, on real hardware, that program needs a device-scope memory fence before reading the workspace.
- On a GPU, sweep `GATHER {1, 2, 4, 8}` and `threshold {32, 64, 128, 256}` on a skewed batch with D = 128, and plot GB/s.

Summary

TBE is a gather-and-reduce problem whose difficulty lives in the data: dependent loads in the forward, and skewed duplicate rows in the backward. The forward hides latency with several independent row loads per iteration and accumulates in FP32. The backward sorts lookups into per-row runs, sends short runs to atomic-free programs and long runs to split-K chunks with an atomic merge, and applies exactly one optimizer update per row. Every strategy switch is a config value, which is where FBTriton found its speedups. The interpreter plus a threshold sweep plus a cross-target compile verifies all of this before a GPU is involved.

Sources (for further reading)

- PyTorch blog, "Modernizing Table Batched Embeddings with FBTriton" (2026-10-06): https://pytorch.org/blog/modernizing-table-batched-embeddings-with-fbtriton/. Grid, gather width, run routing at SL 256, register/occupancy table, GB200 results, analysis versus CUDA at SL 32.
- FBGEMM documentation, Table Batched Embedding operators: https://pytorch.org/FBGEMM/.
- PyTorch documentation, `torch.nn.functional.embedding_bag`: https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.embedding_bag.html.
- Triton documentation: `tl.atomic_add`, interpreter mode (`TRITON_INTERPRET=1`): https://triton-lang.org/main/python-api/triton.language.html.

Part V: AI-Written Kernels

DeepSeek: FlashMLA & DeepGEMM

DeepSeek: FlashMLA & DeepGEMM

DeepSeek stunned the AI industry by open-sourcing compact, ultra-efficient kernels that outcompeted monolithic enterprise libraries.

FlashMLA: Multi-Head Latent Attention

DeepSeek-V3 and R1 utilize Multi-Head Latent Attention (MLA) to compress KV caches into low-dimensional latent spaces. Standard attention kernels cannot handle this asymmetrical projection efficiently. FlashMLA is custom-tuned to decode latent vectors directly into shared memory, cutting KV memory bandwidth consumption by over 70%.

DeepGEMM: Clean Architecture over Brute Force

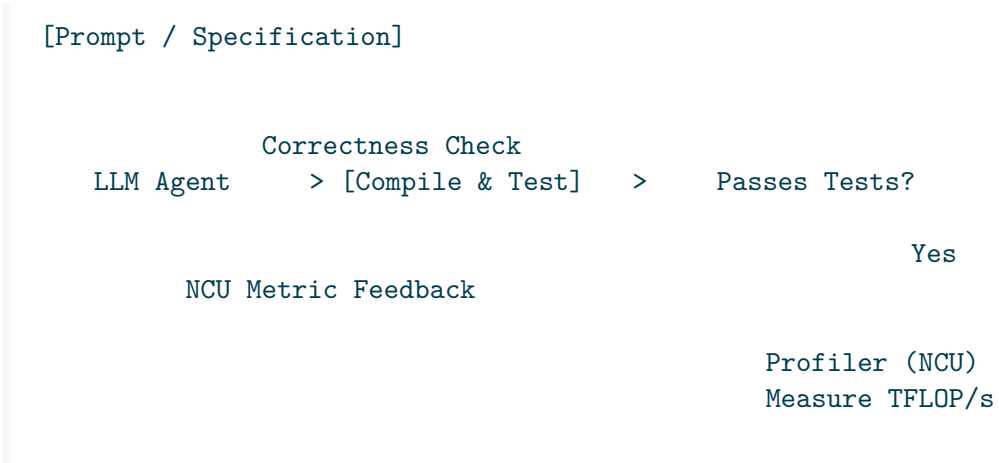
In fewer than 300 lines of readable, modular CUDA code, `DeepGEMM` implements fine-grained FP8 matrix multiplication with grouped scaling. By avoiding deeply nested template metaprogramming and focusing purely on optimal software pipelining and TMA scheduling, DeepGEMM matches and often beats commercial closed-source GEMM kernels.

LLM-Generated Kernels: The Agent + Profiler Loop

LLM-Generated Kernels: The Agent + Profiler Loop

We are entering a transformative era where AI models generate, optimize, and verify GPU kernels autonomously.

The Agentic Optimization Loop



Google DeepMind AlphaEvolve & KernelBench

- **KernelBench:** An automated evaluation benchmark testing whether language models can generate numerically correct and high-performance CUDA/Triton kernels across realistic neural network primitives.
- **DeepMind AlphaEvolve:** An evolutionary coding agent combining LLM code mutations with real hardware profiler feedback. AlphaEvolve discovered non-trivial memory layout schedules that sped up production attention kernels by ~32%.

The Role of the Human Kernel Engineer

The engineer's job is not becoming obsolete; it is elevating: - Defining invariants, numerical bounds, and correctness testbenches. - Providing architectural guidance and identifying hardware bottlenecks. - Interpreting profiler telemetry to steer the agent's exploration away from local minima.

Part VI: Kernel Engineering Capstone Projects

Crusoe Capstone Specifications & Problem Tracks

Crusoe Capstone Specifications & Problem Tracks

The capstone project requires building a production-ready GPU kernel addressing genuine infrastructure challenges.

Problem Track 1: Fused Sparse Attention

Implement a fused attention kernel supporting block-sparse patterns (e.g. BigBird or sliding window + global tokens). The kernel must: - Skip computation of zero-weight blocks entirely. - Maintain online softmax correctness across sparse blocks. - Outperform standard dense PyTorch attention by at least 2x on sequences >16k.

Problem Track 2: Quantized FP4/FP8 GEMM with Dynamic Dequantization

Build a high-throughput matrix multiply kernel that ingests FP8 or INT4 quantized weights and dequantizes on-the-fly inside SM registers before execution.

Problem Track 3: High-Throughput Speculative Decoding Verification Kernel

Design a batched verification kernel for speculative decoding that verifies K draft tokens concurrently against a target model distribution in a single fused pass.

Verification, Profiling, and Benchmarking Harness

Verification, Profiling, and Benchmarking Harness

Every kernel submission must pass a three-tiered validation suite:

1. Correctness Verification

Compare kernel output tensors against double-precision PyTorch / FP64 reference outputs across edge-case dimensions, non-multiple-of-32 matrix sizes, and extreme numerical scales (10^{-5} to 10^{5}). Maximum allowable relative error:

$$\frac{\|Y_{\text{kernel}} - Y_{\text{ref}}\|_2}{\|Y_{\text{ref}}\|_2} < 10^{-3}$$

2. CUDA Event Benchmarking

Measure kernel execution time using `cudaEventRecord` over 1,000 warm iterations, discarding outliers:

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
for (int i = 0; i < 1000; ++i) {
    launch_kernel<<<grid, block>>>(...);
}
cudaEventRecord(stop);
cudaEventSynchronize(stop);

float ms = 0;
cudaEventElapsedTime(&ms, start, stop);
printf("Average execution time: %f ms\n", ms / 1000.0f);
```

Demo Day Presentation & Portfolio

Demo Day Presentation & Portfolio

Presenting your work effectively is as critical as writing fast code.

Deliverables Checklist

1. **GitHub Repository:** Clean, modular repository containing your kernel source, CMake / setup.py build configuration, and automated test runners.
2. **Performance Roofline Graph:** A generated plot showing where your kernel sits relative to theoretical memory and compute ceilings across varying matrix dimensions.
3. **Speedup Comparison Table:** Side-by-side throughput (TFLOP/s) comparisons against naive CUDA, PyTorch `torch.compile`, and official vendor libraries (cuBLAS / cuDNN).
4. **Technical Defense:** Prepare concise explanations for architectural decisions: tile sizes, shared memory allocations, register counts per thread, and how AI-assisted iteration shaped the final solution.